

Chapter 16:

Visualizing Neural Networks

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

July 14/15/21, 2026

Impact of Deep Learning

- **Turing Award 2018** to Yoshua Bengio, Geoffrey Hinton and Yann LeCun, for „breakthroughs that have made deep neural networks a critical component of computing“
- **Nobel Prize in Physics 2024** to John Hopfield and Geoffrey Hinton, for „foundational discoveries and inventions that enable machine learning with artificial neural networks“



Ill. Niklas Elmehed © Nobel Prize Outreach
John J. Hopfield



Ill. Niklas Elmehed © Nobel Prize Outreach
Geoffrey Hinton

<https://www.acm.org/articles/bulletins/2019/march/turing-award-2018>

<https://www.nobelprize.org/prizes/physics/2024/summary/>

Deep Learning for Image Classification

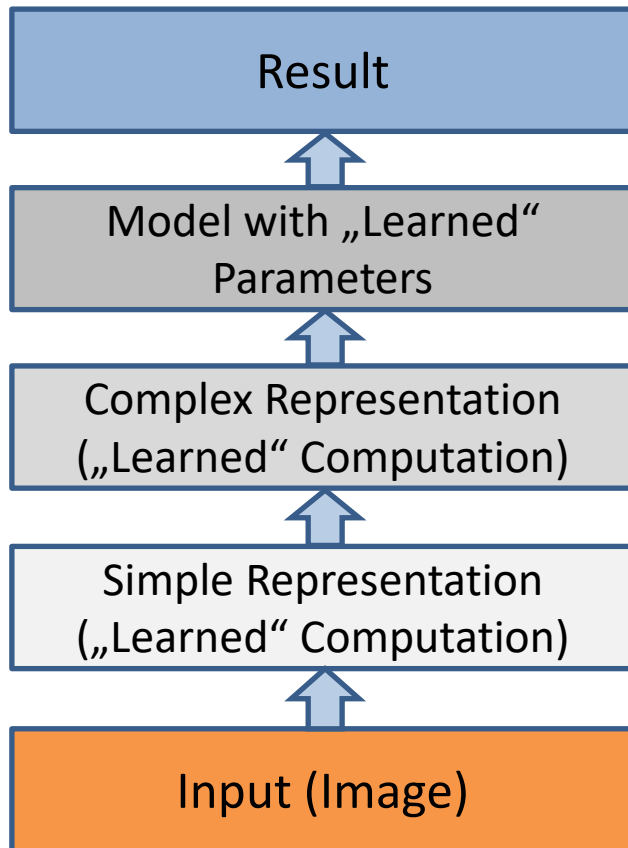
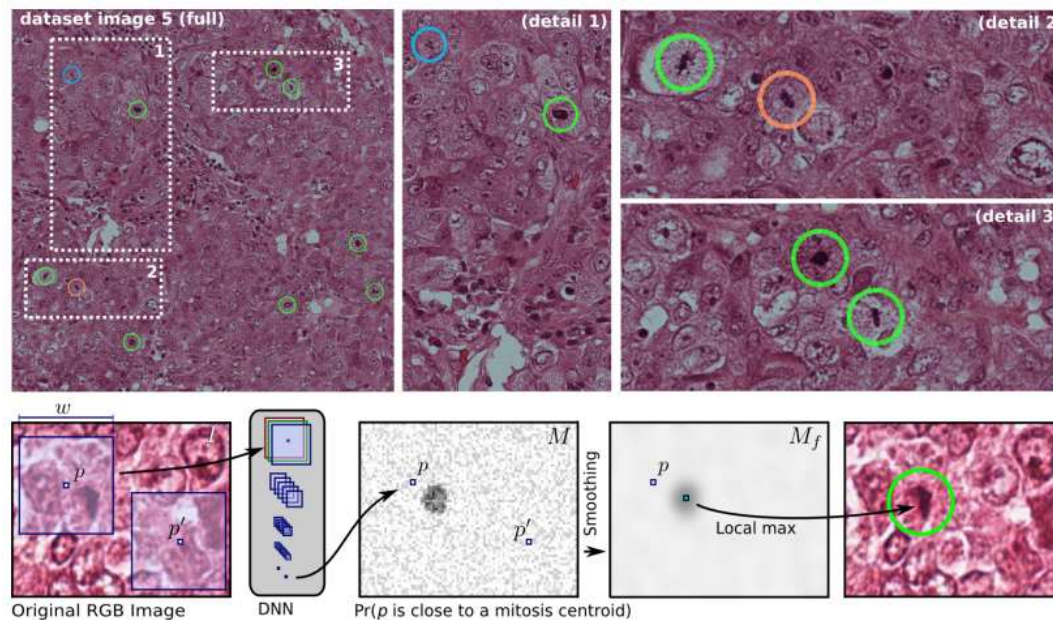


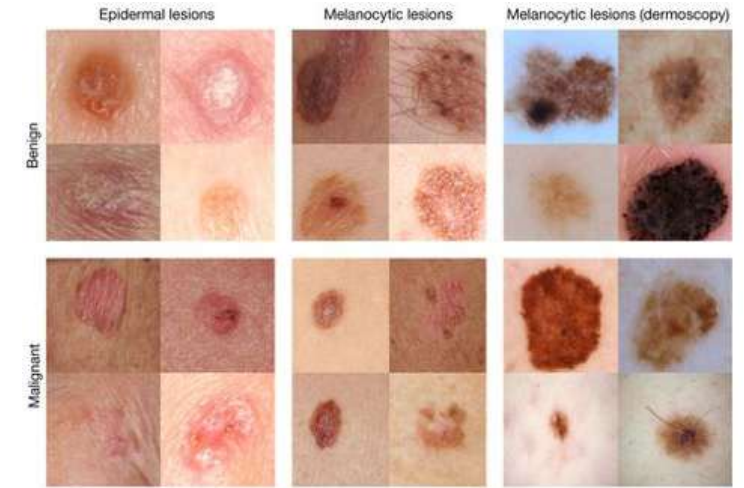
Image from: Olga Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge" Int'l J. of Computer Vision 115(3):211-252, 2015

Deep Learning in Medical Imaging: Academic Success

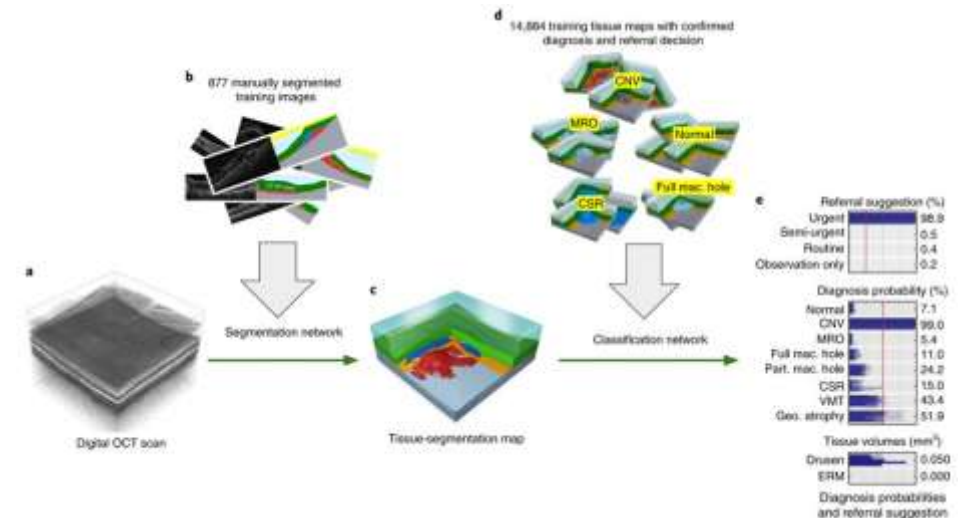
We are starting to see human expert level accuracy in various applications



[Cireřan et al., MICCAI 2013]



[Esteva et al., Nature 2017]



[De Fauw et al., Nature Medicine 2018]

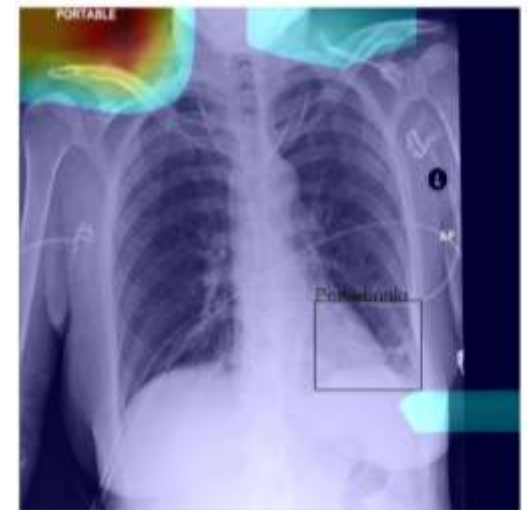
Deep Learning in Medical Imaging: Commercialization

- In 2018, IDx-DR, which detects diabetic retinopathy, becomes the first **FDA-approved autonomous diagnostic system**.
- **May 2026:** More than 1400 FDA-approved AI-enabled medical devices



Why Visualize: Building Confidence / Trust

- **Clever Hans effect:** A neural network might produce the correct result *for the wrong reason*
- *Examples:*
 - Use of a portable X-ray indicates that patient was too sick to move
 - Presence of drain tube indicates treatment of a pneumothorax, but is not a useful basis for making a diagnosis
- **Goal in Section 16.3:** Reveal which parts of the input determine the output



Original image



Grad-CAM overlay



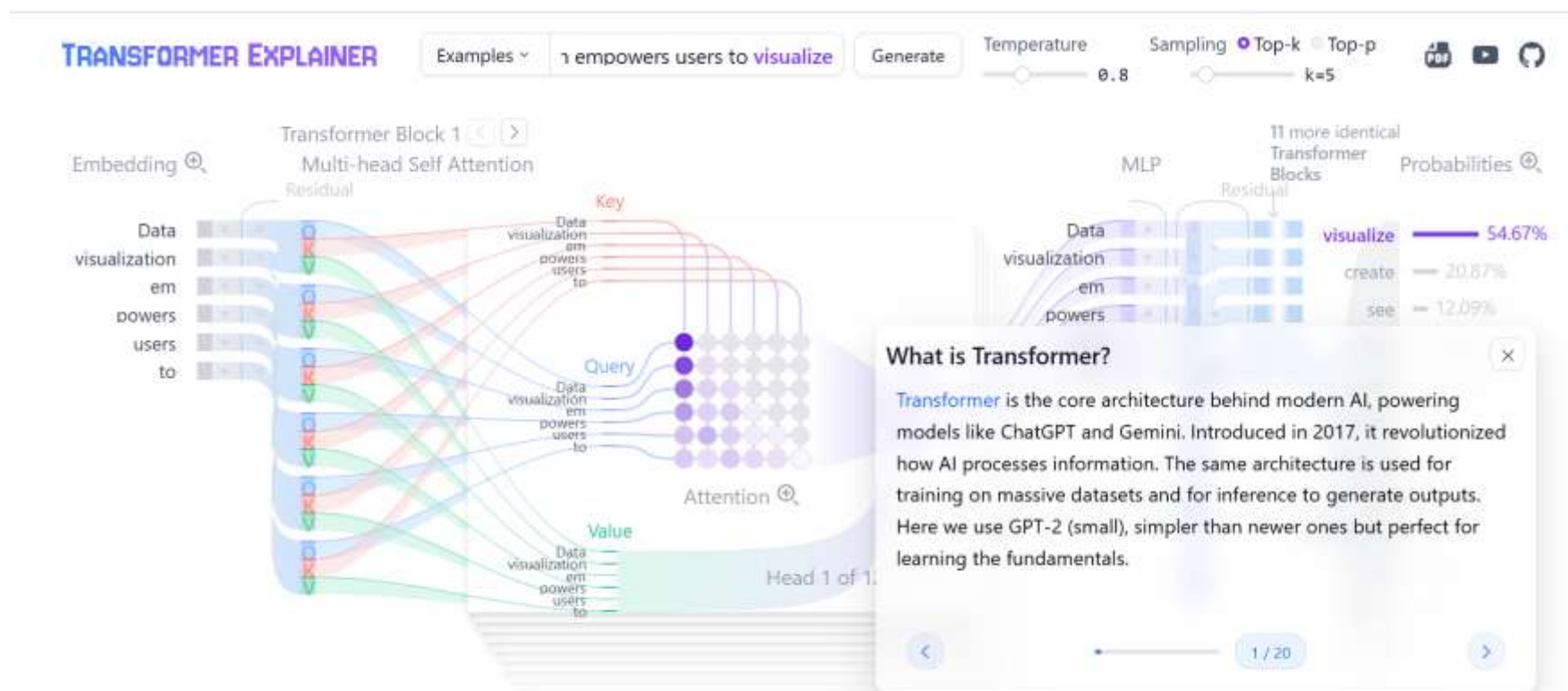
Why Visualize: Understanding the Abstraction

- **“Black box” problem:** The power of deep neural networks derives from their ability to **learn a hierarchy of suitable abstractions** of complex data, but what exactly are they?
- **Goal in Section 16.4:** Reveal what certain network layers or specific neurons represent



Why Visualize: Understanding Basic Principles

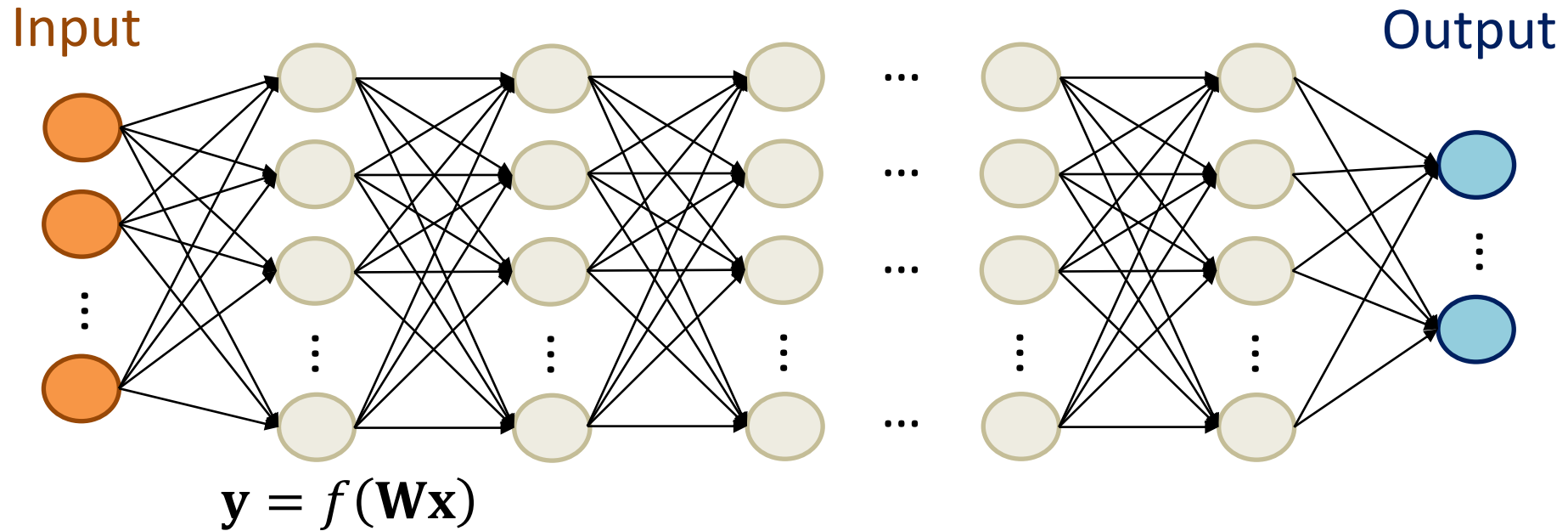
- Building a **general understanding** of how different neural network architectures work is another visualization goal
 - Will not be covered here in detail due to time constraints



Section 16.1:

Deep Neural Networks

Deep Neural Networks



Deep learning is typically implemented using **neural networks**

Artificial Neuron: Formal Definition

Artificial neurons can be expressed by the following mathematical equation:

$$y_k = f \left(\sum_{j=1}^m w_{kj} x_j + b_k \right)$$

- y_k is the output of the k th neuron
- x_j are the inputs
- w_{kj} are the weights
- b_k is a bias term
- f is a nonlinear transfer (activation) function

Artificial Neuron: Alternative Equations

A common alternative notation sets $x_0 := 1$, $w_{k0} := b_k$, so that

$$y_k = f \left(\sum_{j=0}^m w_{kj} x_j \right)$$

In matrix notation:

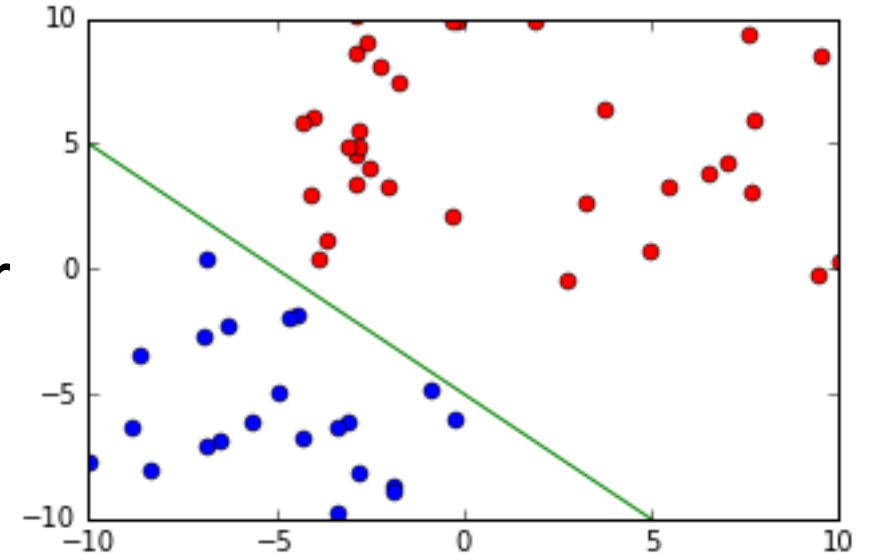
$$\mathbf{y} = f(\mathbf{W}\mathbf{x})$$

- f is applied element-wise
- weight matrix $\mathbf{W}$ collects w_{kj}

Quiz: What does this trick remind you of?

Binary Activation Function

- Pre-activation $\alpha = \sum x_i w_i + b$
- Simplest activation function is binary, so that a neuron is either firing or not; $f(\alpha) = \begin{cases} 1, & \alpha > 0 \\ 0, & \text{o.w.} \end{cases}$
- In this case, a neuron is a linear classifier
- Nonlinear decision boundaries achieved by stacking neurons in layers

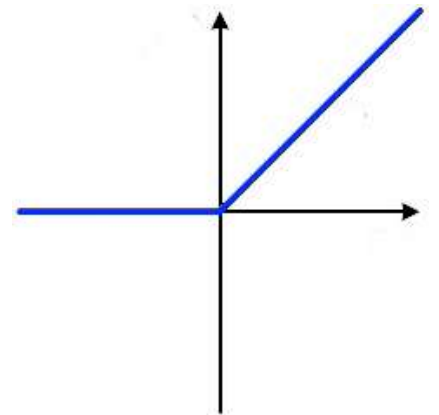


Rectified Linear Unit

- **Problem:** Thresholding does not permit gradient-based optimization of weights

Various alternatives are used in practice. Popular example:

- **Rectified Linear Unit (ReLU)** $f(\alpha) = \max(\alpha, 0)$
 - Simple and computationally efficient
 - Gradient-based learning effective over the full range for which the unit is active



Linear Image Classification

- Trying to classify an image with a single layer of neurons would amount to finding a linear function that produces scores for different classes

$$f(\mathbf{x}, \mathbf{W}, \mathbf{b}) = \mathbf{W}\mathbf{x} + \mathbf{b} = \mathbf{s}$$

$\mathbf{x}$: Input image

$\mathbf{W}$: Parameters (weights)

$\mathbf{b}$: Bias (offset)

$\mathbf{s}$: Score vector for different classes



$\mathbf{x}$

$f(\mathbf{x}, \mathbf{W}, \mathbf{b})$

-2.5	Cat
2	Dog
3	Bird

Training Linear Image Classification

- We would attempt to find W and b , so that for arbitrary input x_i , the score for the correct class is maximum



[64x64x3]

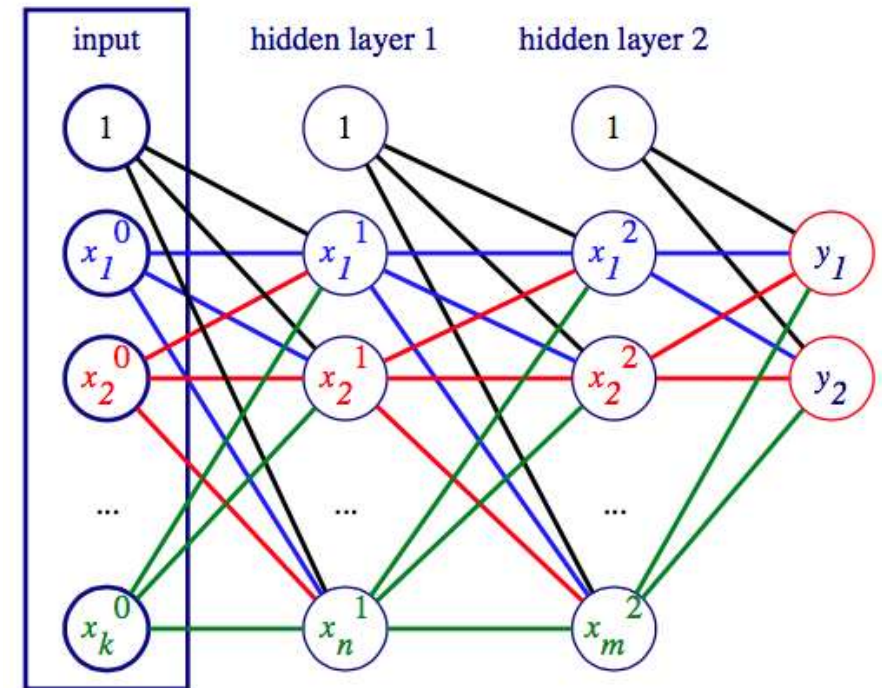
$$f(x_i, W, b) = Wx_i + b$$

W				x_i		b			
2.3	5	0	...	3	+	-1.5	=	-2.5	Cat
-1	-3	1	...	231		2		2	Dog
0.5	2	-1	...	21		1		1	Bird
				$\vdots$					

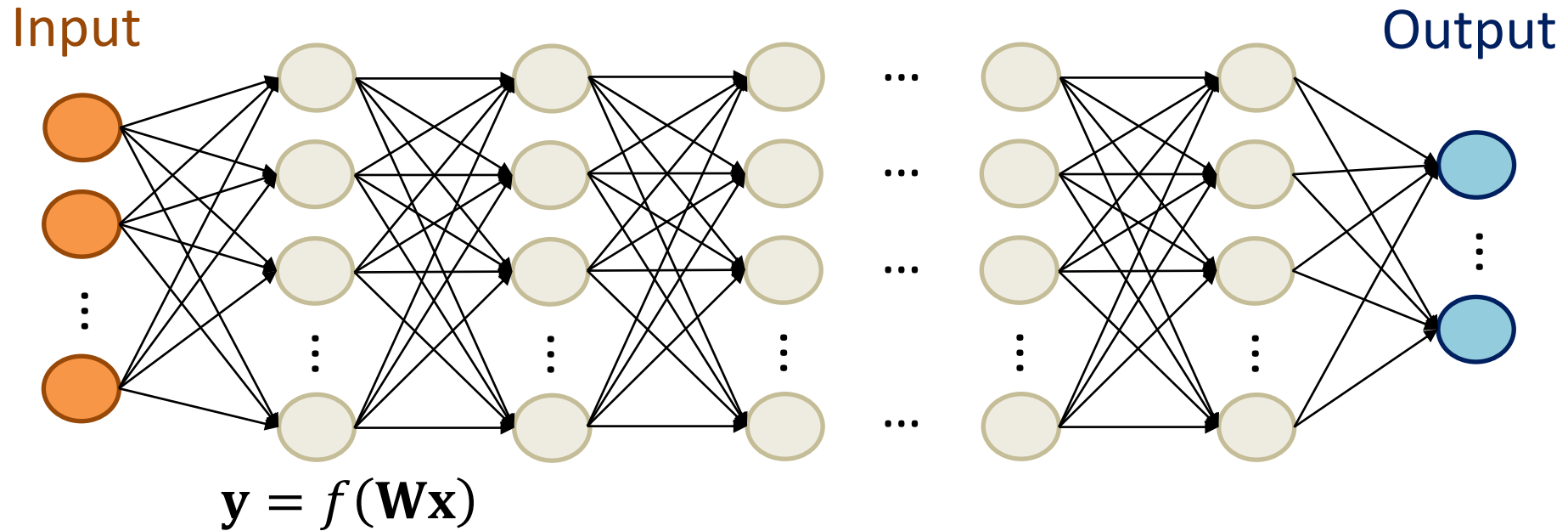
Hidden Layers

- To learn complex functions in a hierarchical manner, we use additional layers between the input and the output. These extra layers are called **hidden layers**

- *Example:* 3-layer neural network with two fully connected hidden layers and two outputs
 - Each connection has its own weight
 - On top is the always-on bias neuron
- In principle, it is possible to have as many hidden layers as desired
 - Training them can become hard



Deep Neural Networks



- In **feedforward neural networks**, neurons are arranged in layers so that the output of earlier layers is only used in later layers
- Complex tasks like image classification require **deep networks**
 - Networks with only one hidden layer are *shallow*; those with two or more hidden layers may be called *deep*. It is not unusual to have >100 layers.

Overview: Training a Neural Network

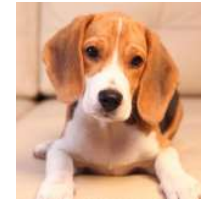
- **Training a network** finds suitable weights for its neurons:
 1. Select a loss function
 2. Initialize weights
 3. Update weights using **backpropagation**.
 - Feed a batch of training samples through the network, compute loss
 - Compute partial derivatives of loss with respect to weights
 - Update weights with a certain learning rate along negative gradient
 4. Repeat step 3 for a number of epochs
 - **Epoch** usually denotes one pass over all training data, sometimes defined as a fixed number of weight updates
 - Learning rate often high initially, reduced over time
 - We use a validation loss to decide when to stop, often also to adapt the learning rate

Probabilistic Predictions via Softmax

Softmax converts raw scores s_j into “class probabilities”

$$P(y = k | \mathbf{x} = \mathbf{x}_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

For $\mathbf{x}_i$ being feature vector of sample i
 y being correct class label for input $\mathbf{x}$



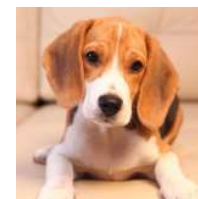
Cat	3.0	3.0	1.0
Dog	1.0	-5.0	3.2
Bird	-2.5	1.5	-5.0

3.0		20.08		0.87
1.0	→ exp	2.7	→ normalize	0.12
-2.5		0.08		0.00

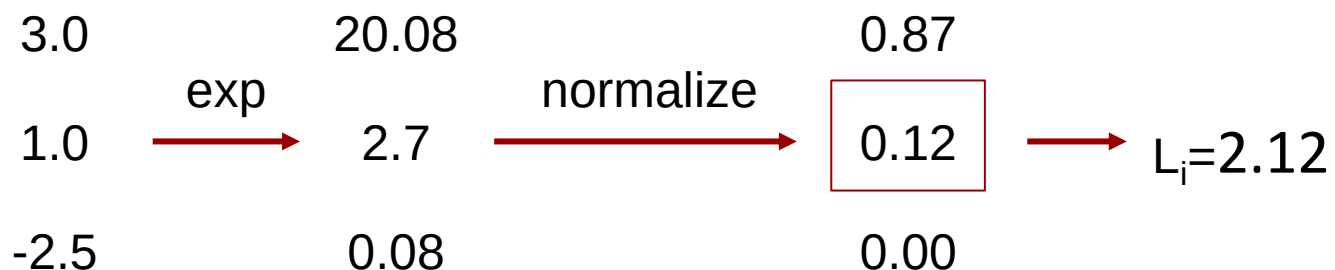
Cross-Entropy Loss

Given softmax conversion, minimizing the negative log-likelihood of the true class y_i amounts to a cross-entropy loss:

$$L_i = -\ln \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right)$$



Cat	3.0	3.0	1.0
Dog	1.0	-5.0	3.2
Bird	-2.5	1.5	-5.0



Optimization

- Given a random initialization of the parameters, we need to slowly change them to minimize the loss function

- **Gradient Descent**

while true:

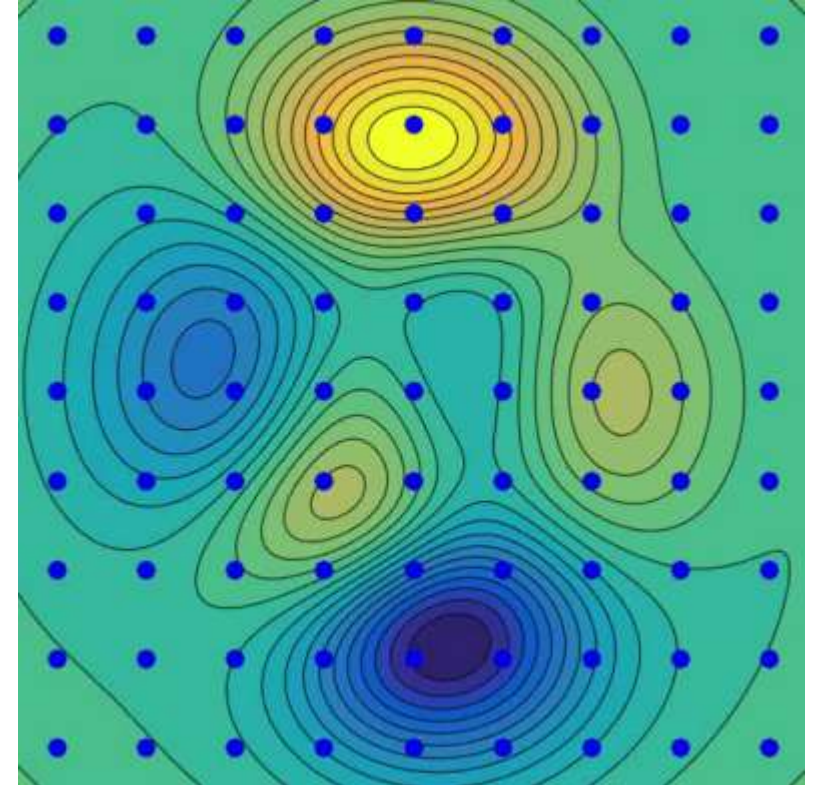
weights_grad = eval_grad(loss_fun, data, weights)

*weights += -step_size * weights_grad*

Step size: also called learning rate

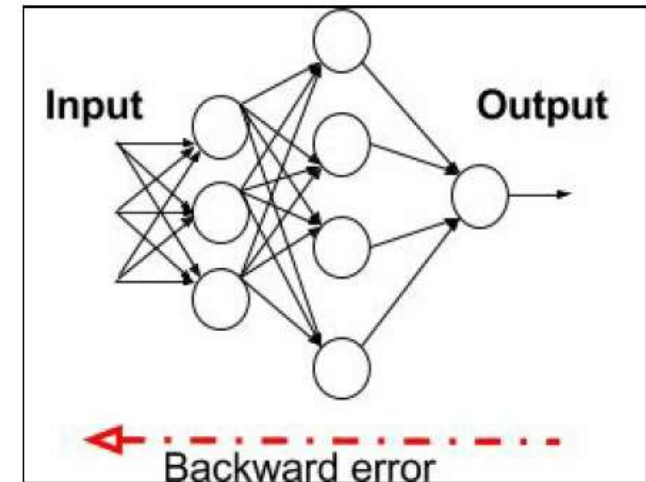
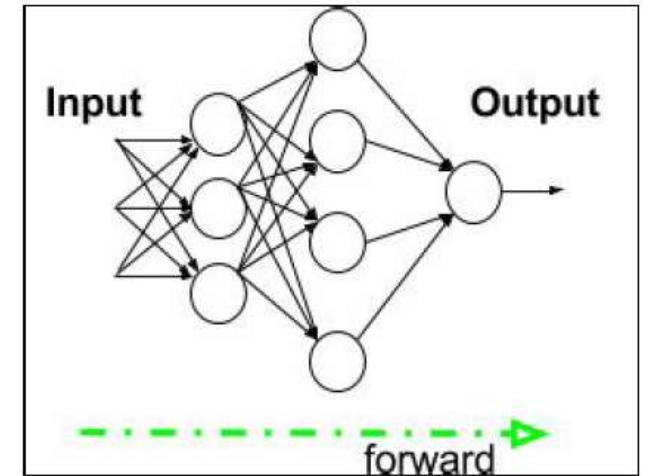
- **Stochastic (Mini-batch) Gradient Descent**

Only use a small, randomly selected part of the training set to compute gradient. Improves gradient estimation speed but causes inaccuracies (“noise”).

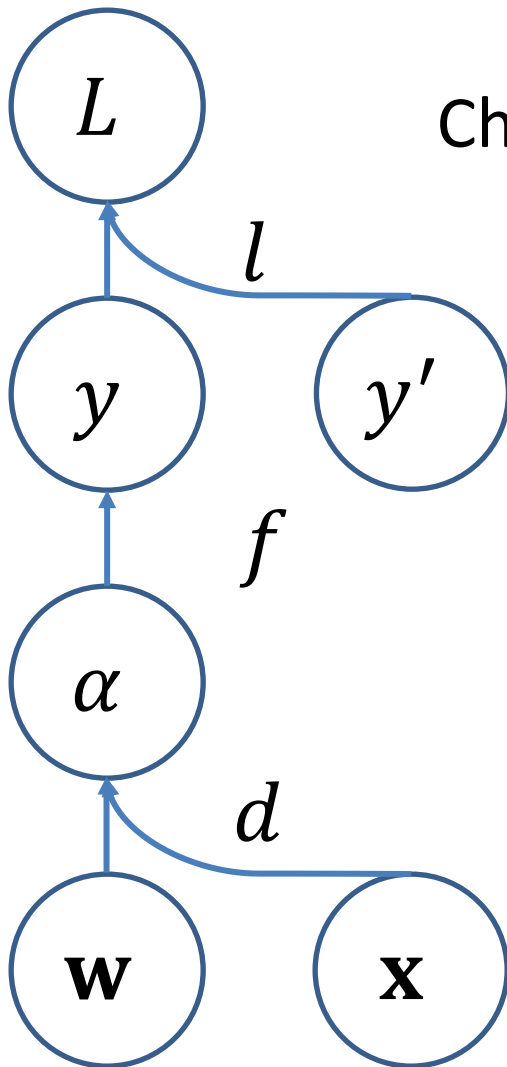


Main Idea Behind Backpropagation

- **Forward pass:** Values are propagated *forward* from the input stage through the hidden stages to the output stage where a prediction is made and the loss is computed. Intermediate results are stored.
- **Backward pass:** Partial derivatives of the loss with respect to the weights are computed using the chain rule, propagating values from the final loss back towards the input layer. Information from this step is used to adjust the neural network weights during training.



Minimal Example: Backpropagation



Chain rule:

$$\begin{aligned}\nabla_{\mathbf{w}} L &= \frac{\partial L}{\partial y} \frac{\partial y}{\partial \alpha} \nabla_{\mathbf{w}} \alpha \\ &= l_y(y, y') f_{\alpha}(\alpha) d_{\mathbf{w}}(\mathbf{w}, \mathbf{x}) \\ &= l_y(f(d(\mathbf{w}, \mathbf{x})), y') f_{\alpha}(d(\mathbf{w}, \mathbf{x})) d_{\mathbf{w}}(\mathbf{w}, \mathbf{x})\end{aligned}$$

Backpropagation algorithm builds up this product step by step, avoiding re-computation by storing intermediate results

Summary

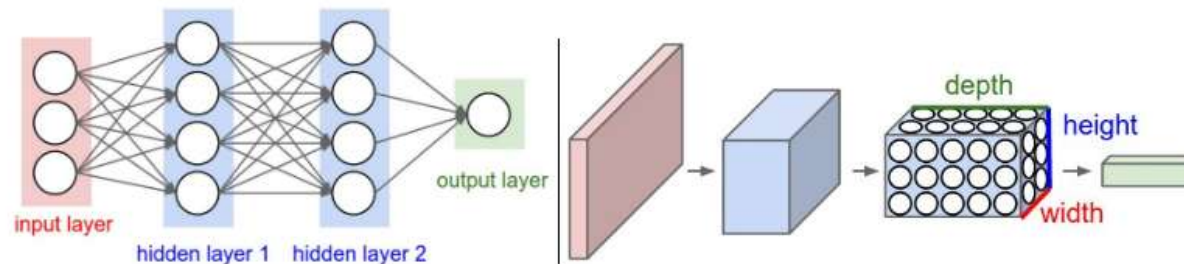
- **Neural Networks** are a popular representation of learnable functions $f(\mathbf{x})$, especially for high dimensional inputs
- **Artificial neurons** are their building blocks
 - Linear map with nonlinear activation $\mathbf{y} = f(\mathbf{W}\mathbf{x})$
 - Single-layer neural networks are linear classifiers
 - **Deep neural networks** permit learning nonlinear functions
 - Training requires suitable **activation function** such as ReLU
- **Stochastic gradient descent** can be used for training
 - **Backpropagation** algorithm for computing gradient of the loss

Section 16.2:

Convolutional Neural Networks

Convolutional Neural Network

- Convolutional neural networks (CNNs/ConvNets) are specially adapted to work with inputs on grids, such as images
- Regular Neural Nets do not scale well to large images. Full connectivity is wasteful and the huge number of parameters would quickly lead to overfitting.
- Instead, the weights in a CNN layer define a convolution kernel that is applied to the input of the layer.
 - Well-suited for tasks with translation invariance, e.g., we would like to recognize objects no matter where in the image they are



Discrete Convolution

- *Reminder:* We learned about 2D convolution in Section 8.4:

$$f(\mathbf{x}) = (h * g)(\mathbf{x}) = \iint h(\xi)g(\mathbf{x} - \xi)d\xi$$

- Given the discrete nature of digital images, we require a corresponding definition for discrete functions h and g :

$$f(i, j) = (h * g)(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k h(u, v) \cdot g(i - u, j - v)$$

- Assumes that h is a convolution kernel of size $(2k + 1)$

Image Convolution: Example 1



Original

*

0	0	0
0	1	0
0	0	0

=



Identical Image

Image Convolution: Example 2



Original



0	0	0
1	0	0
0	0	0



Shifted Left by
1 Pixel

Image Convolution: Example 3



Original

*

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

=



Blurred
(Mean Filter)

Image Convolution: Example 4



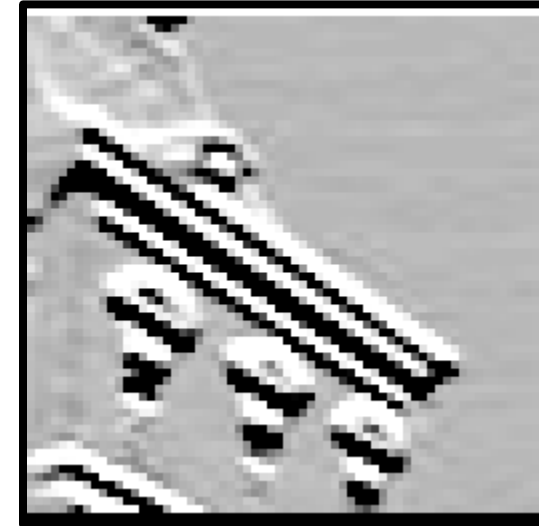
Original

*

$\frac{1}{8}$

-1	-2	-1
0	0	0
1	2	1

=



Horizontal
Edge Detector

Note on Convolution vs. Cross-Correlation

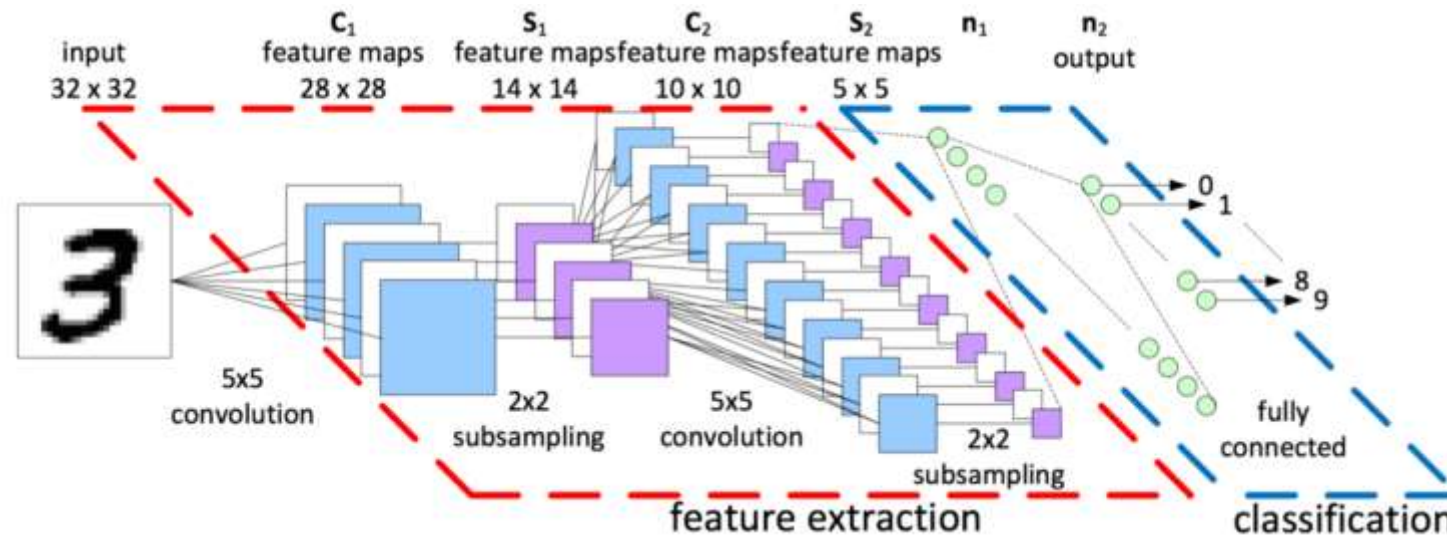
- **Cross-correlation** of g and h also produces a function f
 - Almost the same as convolution, except that h does not get flipped

$$f(i, j) = (h \otimes g)(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k h(u, v) \cdot g(i + u, j + v)$$

- Unlike cross-correlation, convolution is *commutative* and *associative*
- In CNN literature, flipping is often omitted, but the operation is still (incorrectly) called “convolution”
 - Can view this issue as learning a flipped convolution kernel h ; in this sense, imprecise use of terminology does not make any practical difference
 - In this chapter, we will stick with CNN terminology

Building Blocks of CNNs

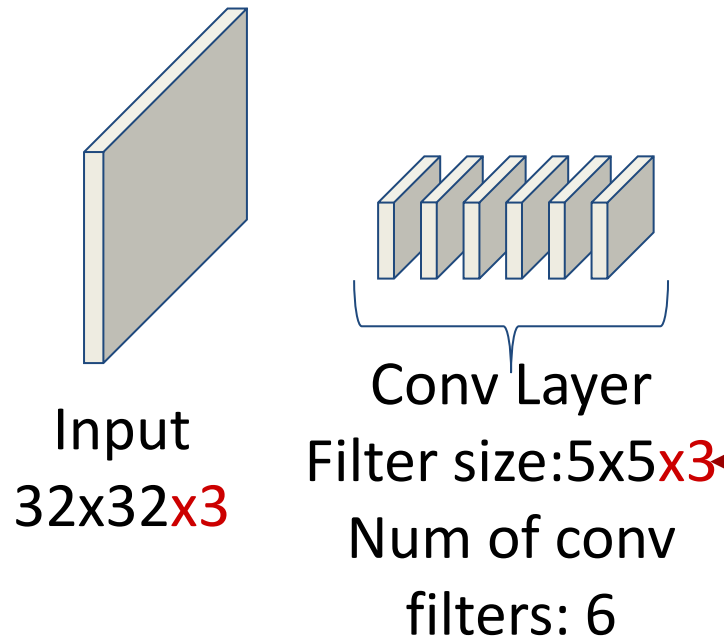
- Example of a simple CNN for handwritten digit recognition:



- Common building blocks of a CNN:
 - Convolution layers
 - Activation layers (e.g., ReLU)
 - Pooling layers

Convolution Layer

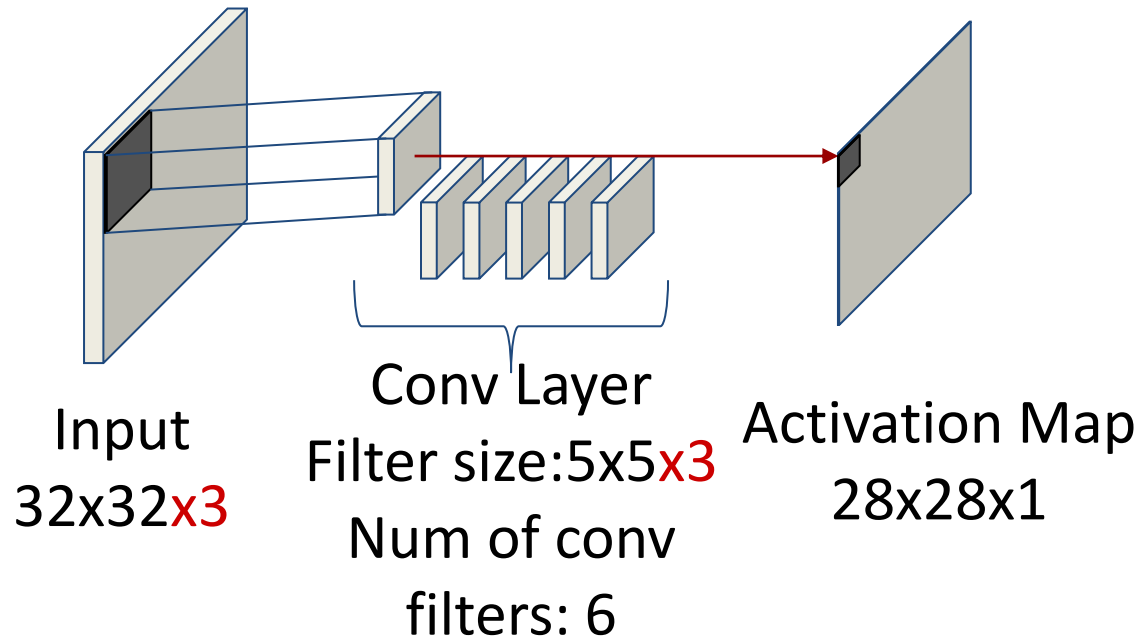
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Each convolution kernel is extended to match the depth of the input volume. In this example, the input has 3 channels. Therefore, each conv filter is extended accordingly.

Convolution Layer

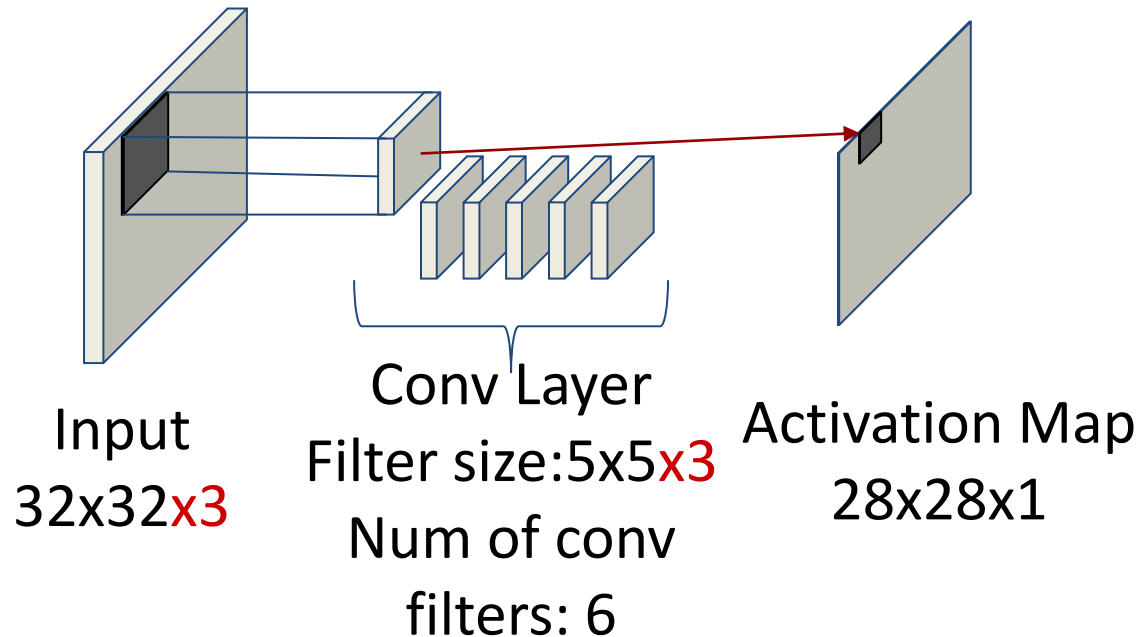
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Activations maps are generated by convolving conv filters with the input volume. That is to slide the filter over the input volume and performing a dot product.

Convolution Layer

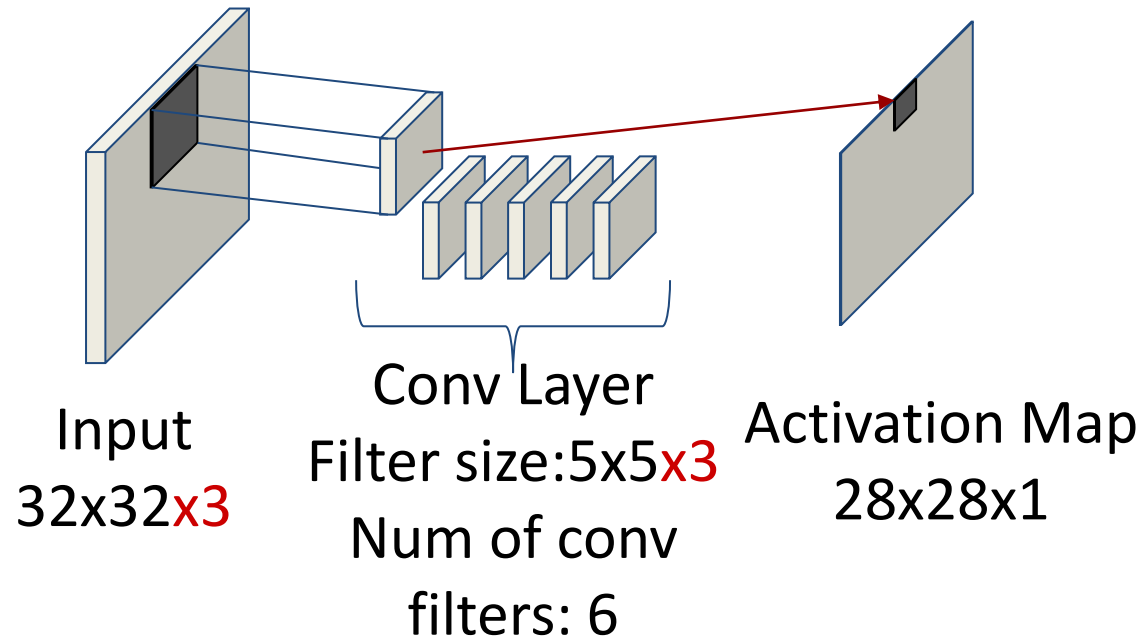
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Activations maps are generated by convolving conv filters with the input volume. That is to slide the filter over the input volume and performing a dot product.

Convolution Layer

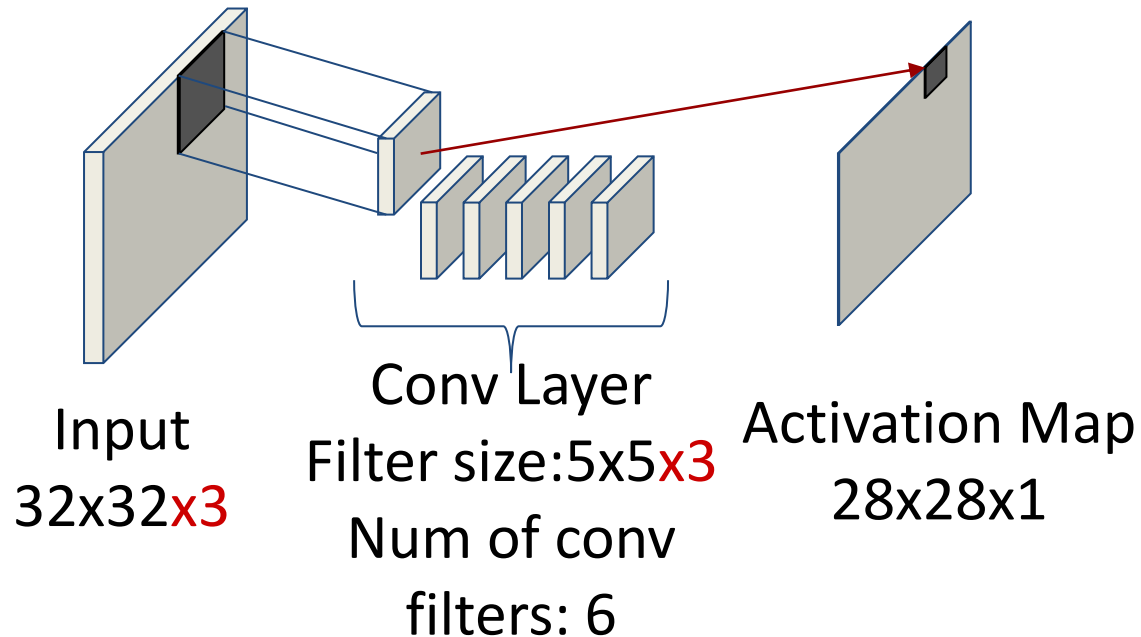
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Activations maps are generated by convolving conv filters with the input volume. That is to slide the filter over the input volume and performing a dot product.

Convolution Layer

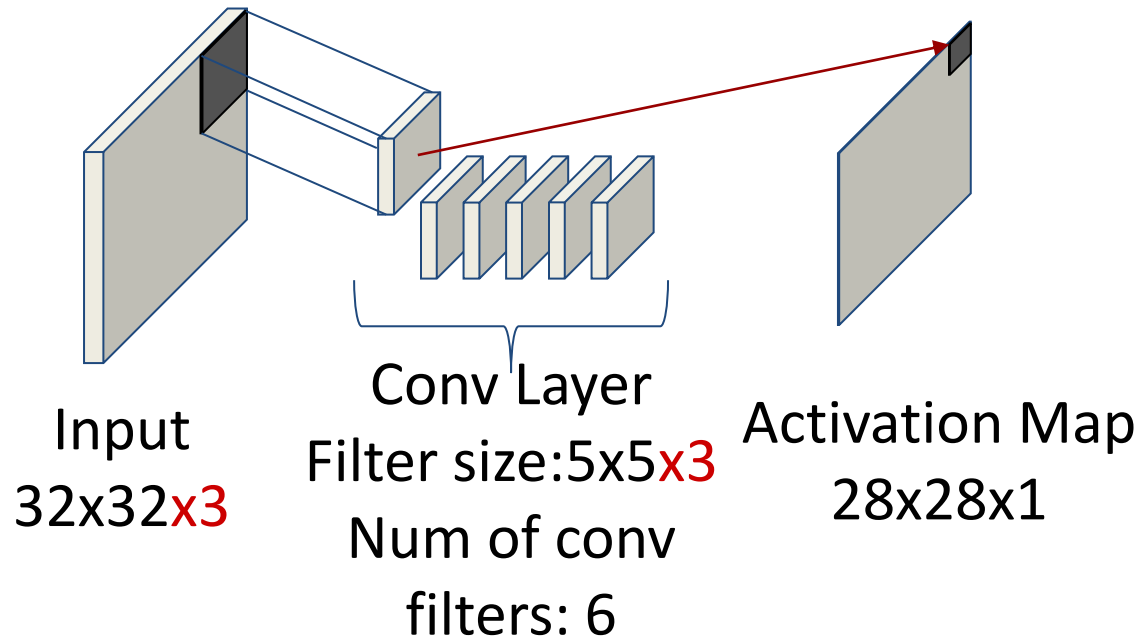
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Activations maps are generated by convolving conv filters with the input volume. That is to slide the filter over the input volume and performing a dot product.

Convolution Layer

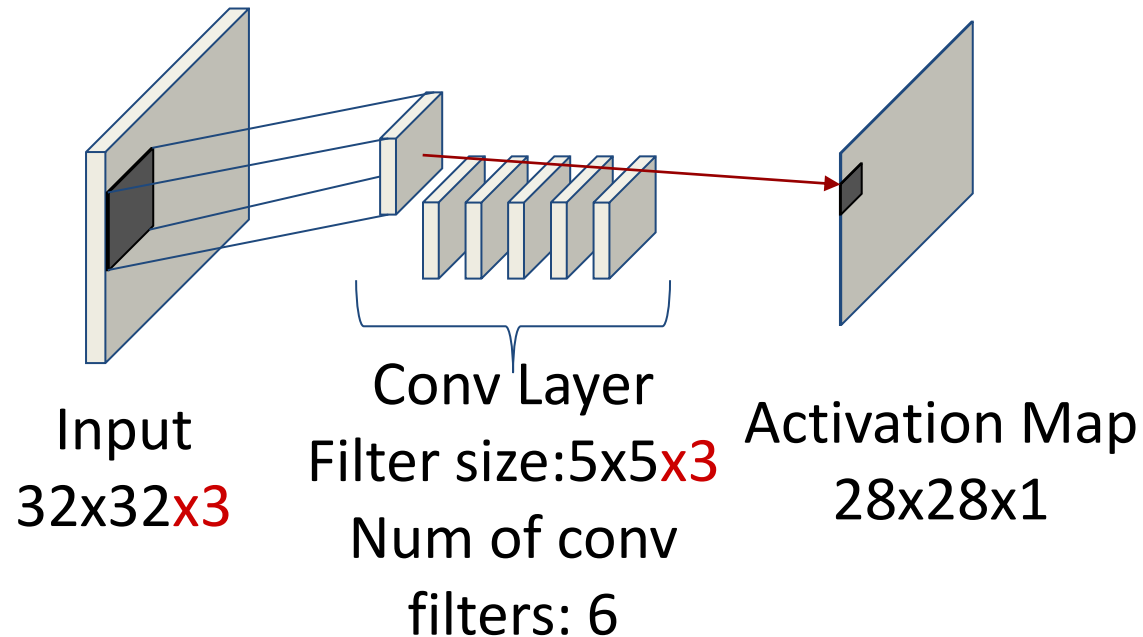
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Activations maps are generated by convolving conv filters with the input volume. That is to slide the filter over the input volume and performing a dot product.

Convolution Layer

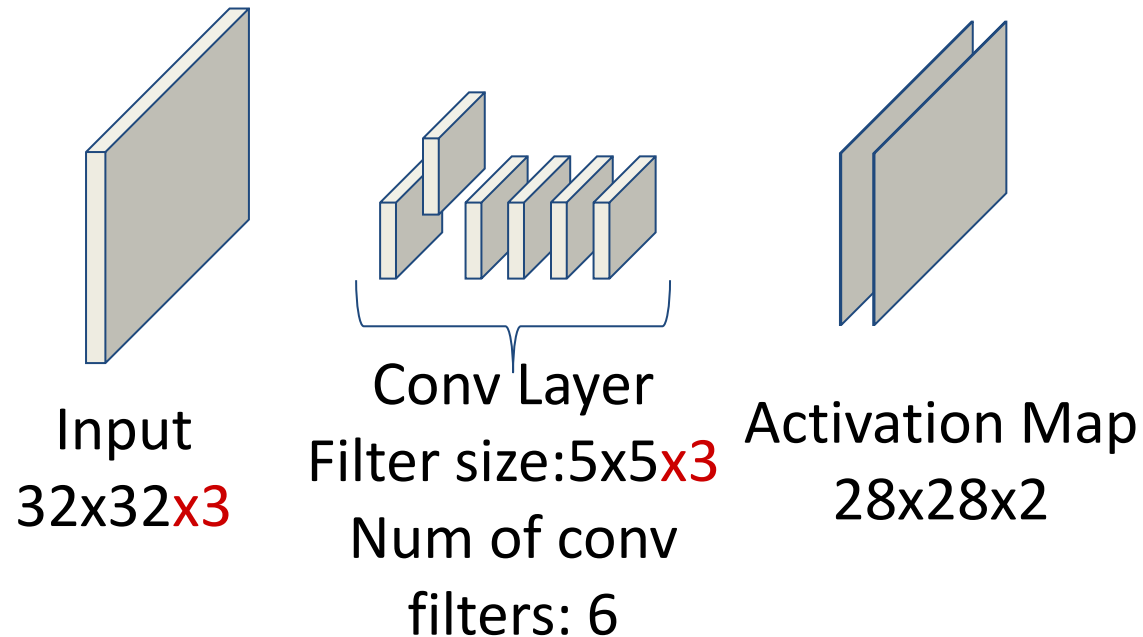
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Activations maps are generated by convolving conv filters with the input volume. That is to slide the filter over the input volume and performing a dot product.

Convolution Layer

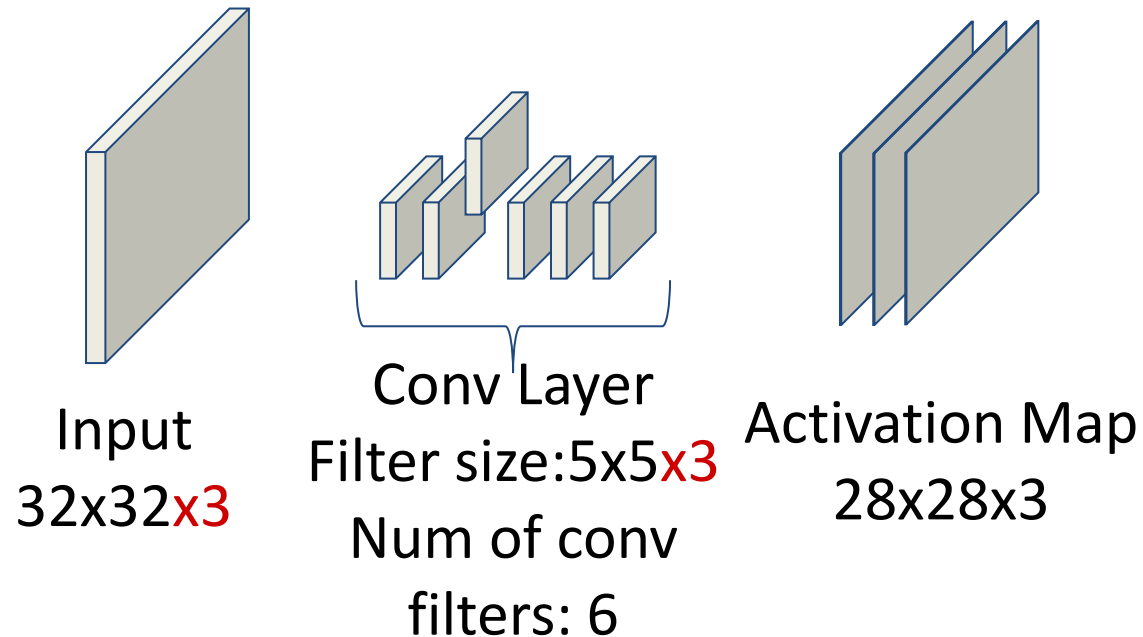
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Each convolution filter creates a separate activation map.

Convolution Layer

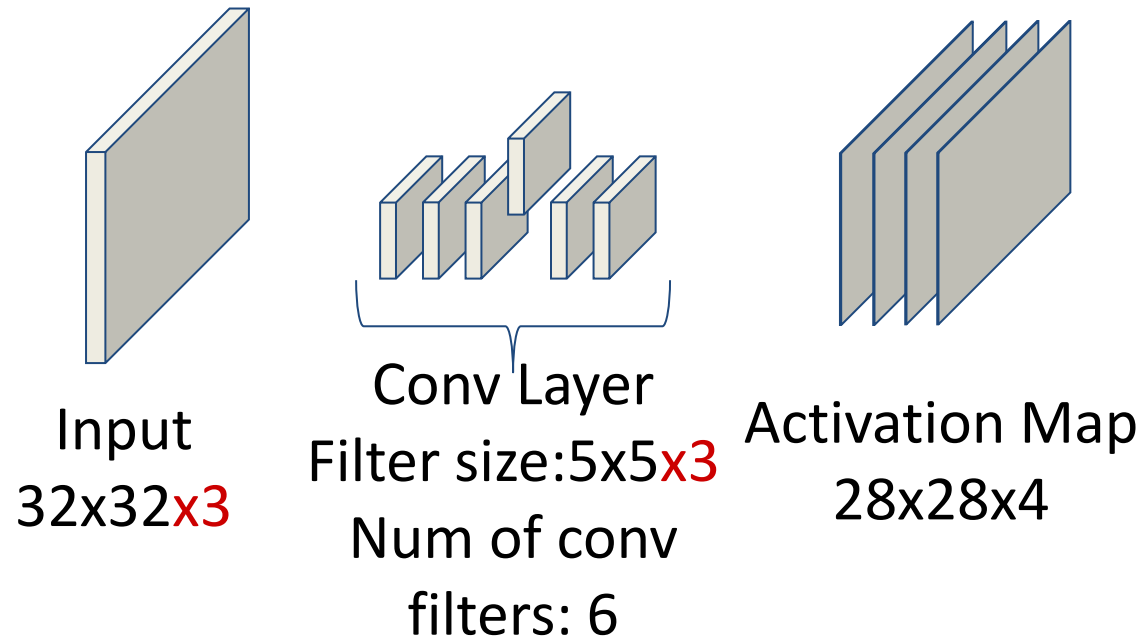
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Each convolution filter creates a separate activation map.

Convolution Layer

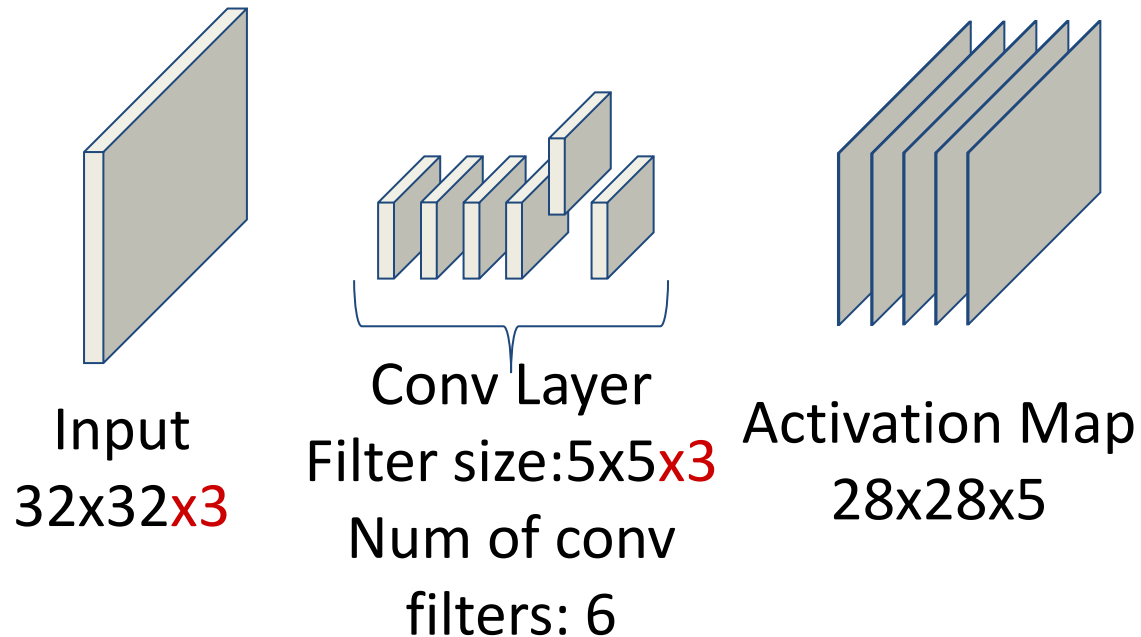
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Each convolution filter creates a separate activation map.

Convolution Layer

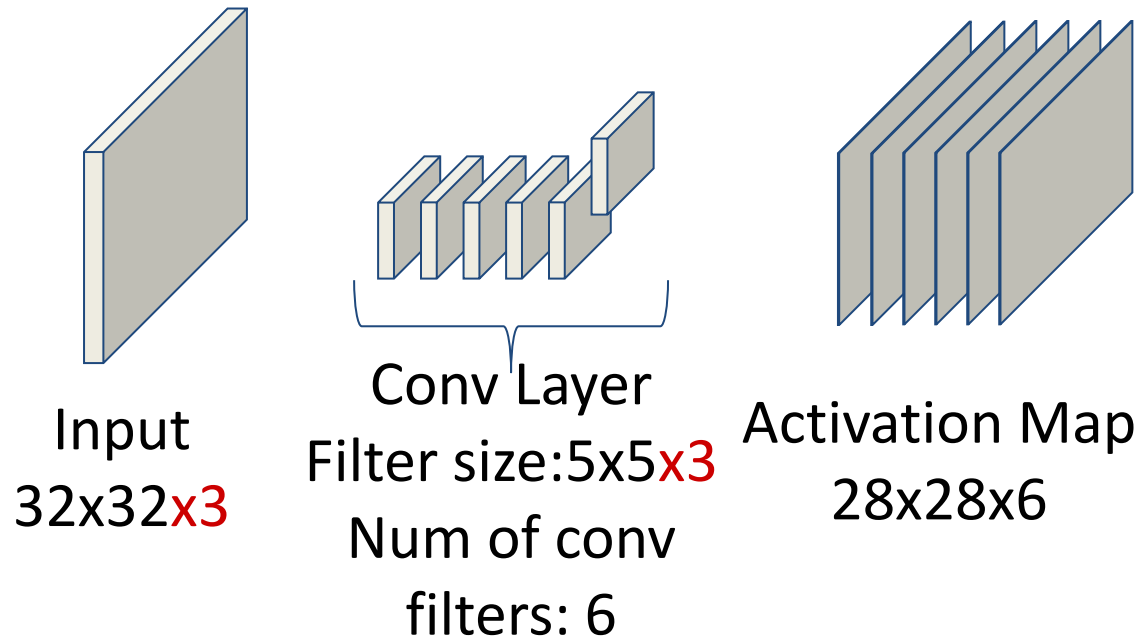
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



Each convolution filter creates a separate activation map.

Convolution Layer

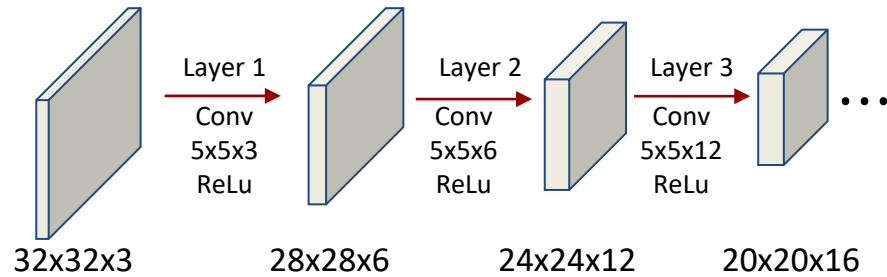
- A convolution layer is defined by
 - Number of convolution filters
 - Convolution Filter size
 - Padding
 - Stride



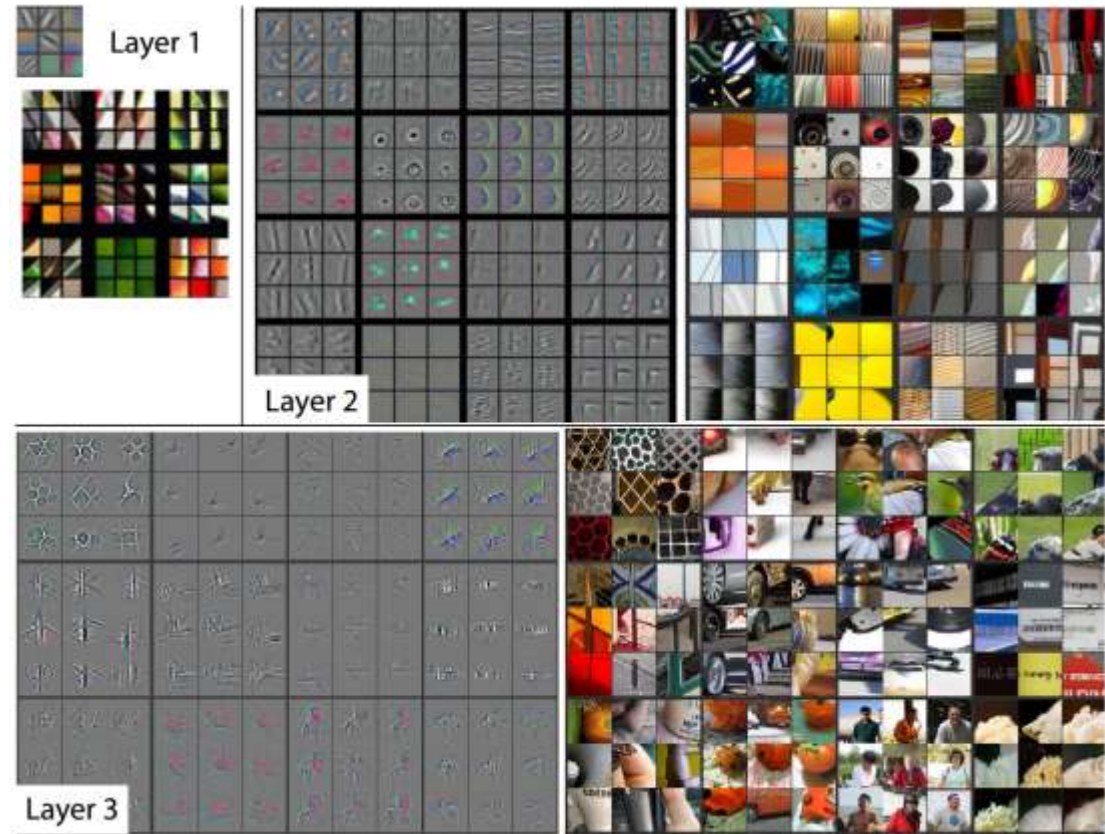
The activation maps “tensor” will be the input volume for the next layer in the CNN.

Feature Hierarchy

- The convolution filters are network weights, i.e., they are initialized with random numbers in the beginning and get updated via backpropagation.

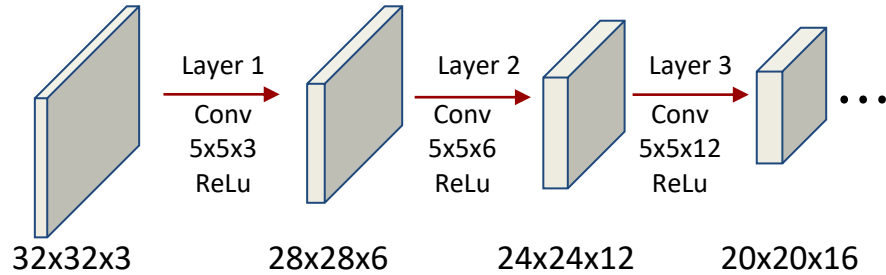


- CNN learns suitable filters for convolution
- Higher layers can capture more complex features
- As we go deeper in the network, the receptive field of neurons is increased.



Source: Zeiler, Matthew D., and Rob Fergus. "Visualizing and understanding convolutional networks." *European conference on computer vision*. Springer, Cham, 2014.

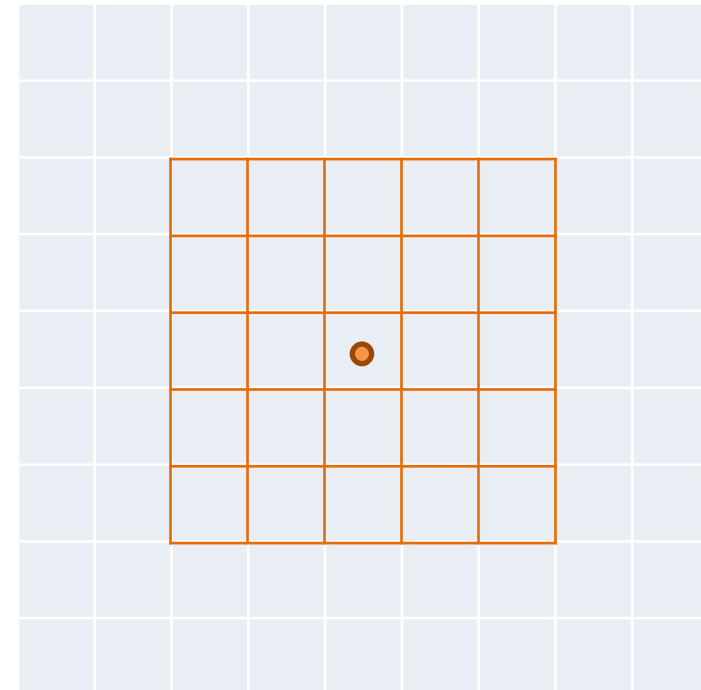
Receptive Field



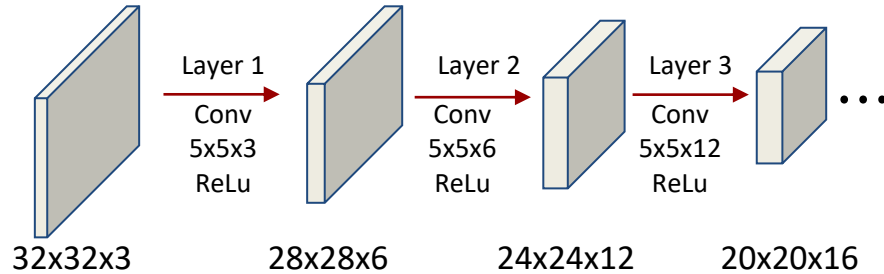
The receptive field of a neuron is defined as the region in the input volume that the neuron is looking at (i.e. is affected by).

The receptive field in Layer 1 is 5x5.
How about in Layer 2 and Layer 3?

1st convolution layer
2nd convolution layer



Receptive Field



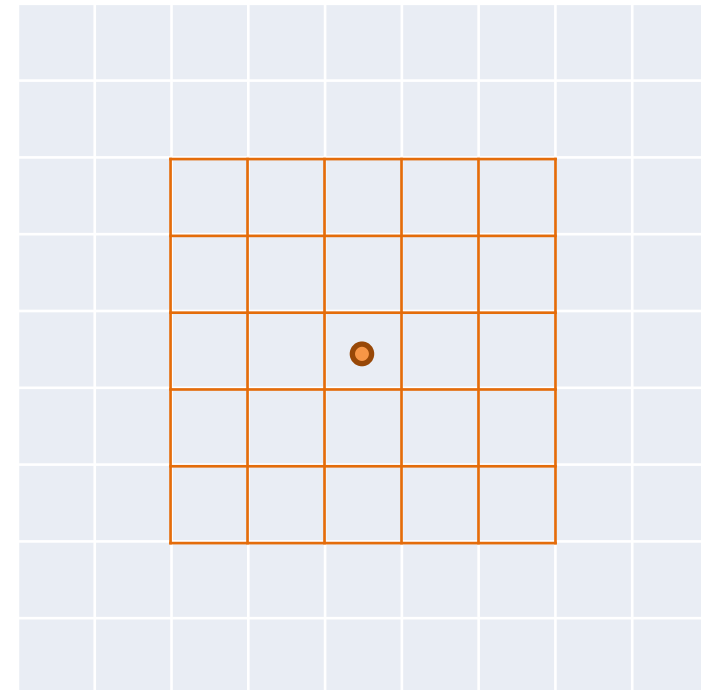
The receptive field of a neuron is defined as the region in the input volume that the neuron is looking at (i.e. is affected by).

The receptive field in Layer 1 is 5x5.
How about in Layer 2 and Layer 3?
9x9 and 13x13 respectively

Striding or pooling make the receptive field grow more quickly.

Padding avoids losing pixels on the boundary.

1st convolution layer
2nd convolution layer



Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

*

1	-1	0
-1	0	0
1	0	1

3x3

=

Stride=2
Padding=0

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4		

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4	3	

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4	3	0

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4	3	0
3		

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4	3	0
3	0	-2
-5	-4	2

Stride=2
Padding=0

Strided convolution can be used to increase the receptive field in subsequent layers

Convolution Operation

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

*

1	-1	0
-1	0	0
1	0	1

3x3

Quiz: What will be the size of output if stride=1? **5x5**

Convolution with Padding

0	0	0	0	0	0	0	0	0
0	1	0	2	1	1	2	2	0
0	1	0	1	0	1	0	1	0
0	2	1	2	1	1	1	1	0
0	-1	0	1	2	1	2	2	0
0	1	2	0	-1	0	1	-1	0
0	6	1	7	-1	0	1	0	0
0	1	1	1	2	1	0	2	0
0	0	0	0	0	0	0	0	0

*

1	-1	0
-1	0	0
1	0	1

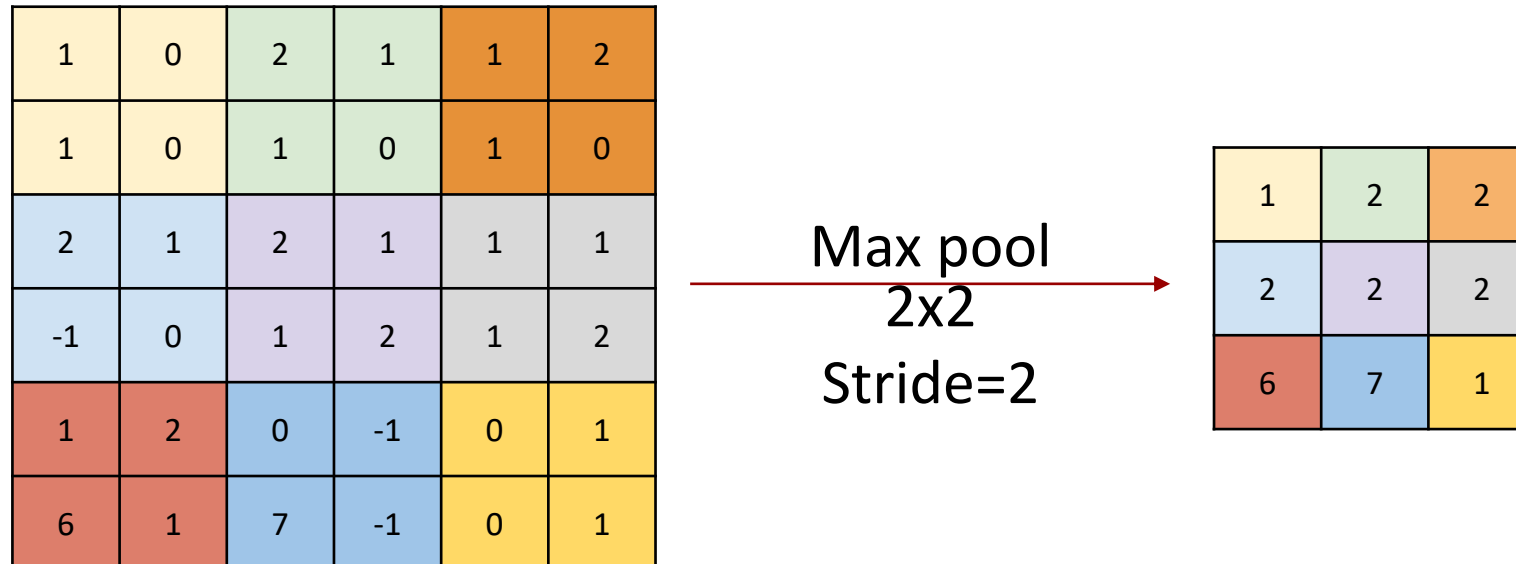
3x3

Quiz: What will be the size of output if stride=1? **5x5**

What will be the size of output if padding=1 and stride=1? **7x7**

Pooling Layer

- **Pooling layers** downsample their input
 - Alternative way to achieve large enough receptive fields with a manageable number of parameters and computational cost
- A pooling layer is defined by
 1. Pooling size
 2. Pooling type (Maximum, Average, Stochastic)
 3. Stride
- Max pooling requires storing location of maxima ("*switches*") for backprop



ImageNet Challenge (ILSVRC)

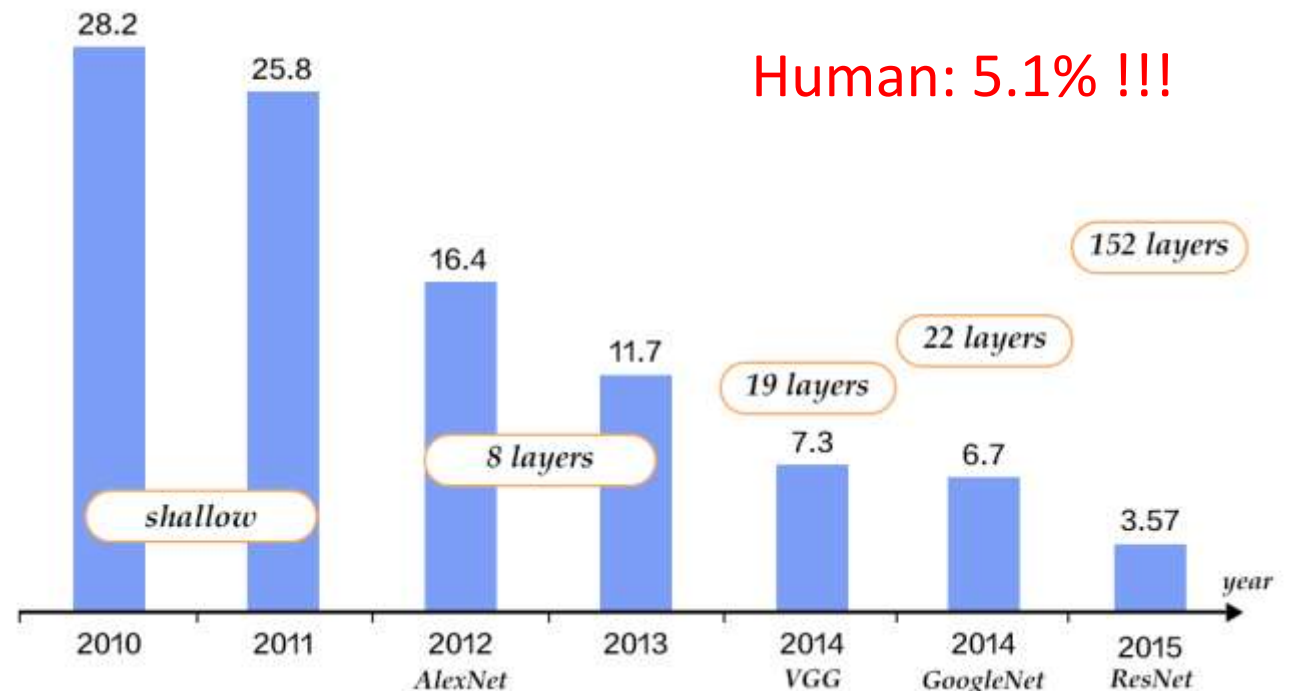
- **ImageNet** is a database with more than 14M images each of which belongs to one of 20.000 categories
- A subset of 1.5M images from 1000 classes form the basis of the **ImageNet Large Scale Visual Recognition Challenge**
 - Took place yearly 2010-2017
 - Includes relatively subtle differences, e.g., 90 different breeds of dogs
 - Top-5-error counts it as success if the correct class is among the 5 most probable



The Triumph of CNN Architectures

- First version of LeNet: 1989
- Main reasons for the breakthrough of CNNs since 2012:
 - Ideas on how to successfully train very **deep networks**
 - GPUs (graphics processing units) provide sufficiently strong and affordable **computational power**
 - Availability of very large **training datasets**

The diagram shows top-5 classification error in Large Scale Visual Recognition Challenge on *ImageNet* dataset.



Source: http://paddlepaddle.org/docs/develop/book/03.image_classification/index.html

Summary: Convolutional Neural Networks

- CNNs use **convolution layers** to reduce the number of parameters and enforce translation invariance.
- **Convolution filters** learn simple, local features in earlier layers; more complex, larger-scale features as the receptive field increases in deeper layers.
- **Pooling layers** are used to further reduce the number of parameters and to increase translation invariance.
- Different **specific architectures** have been shown to be successful, and can be fine tuned for different applications.

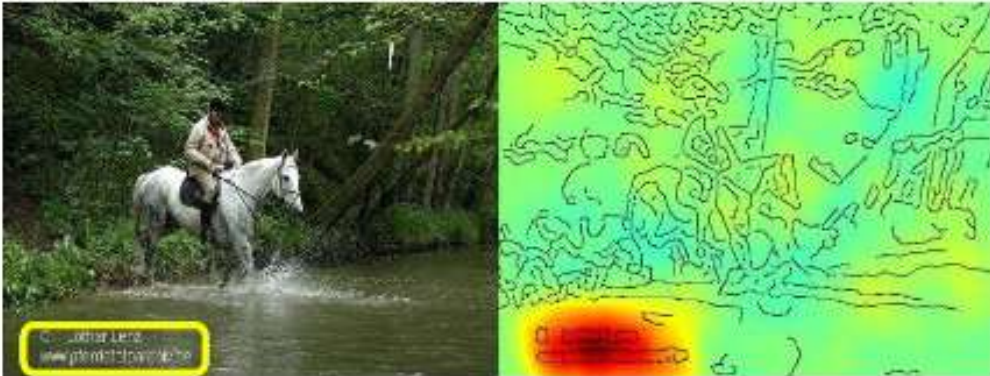
Section 16.3:

Visualizing Input-Output Relationship

Motivation

- **Question:** Which image region(s) are especially relevant for the network's decision?

Horse-picture from Pascal VOC data set

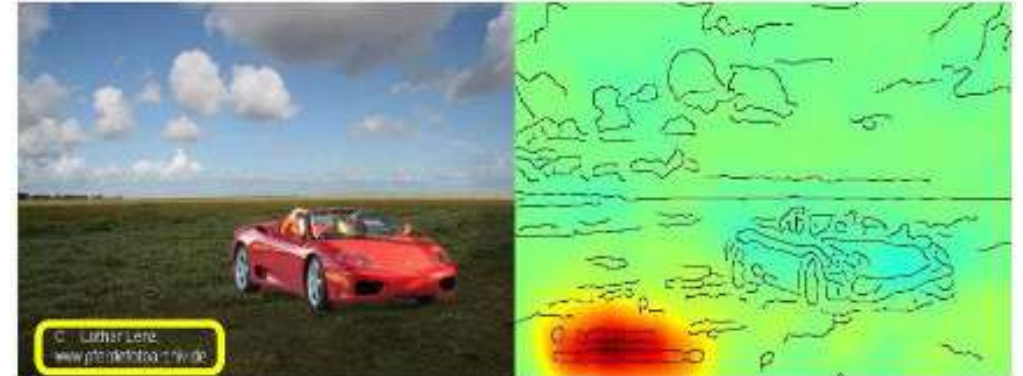


Source tag
present



Classified
as horse

Artificial picture of a car



No source
tag present

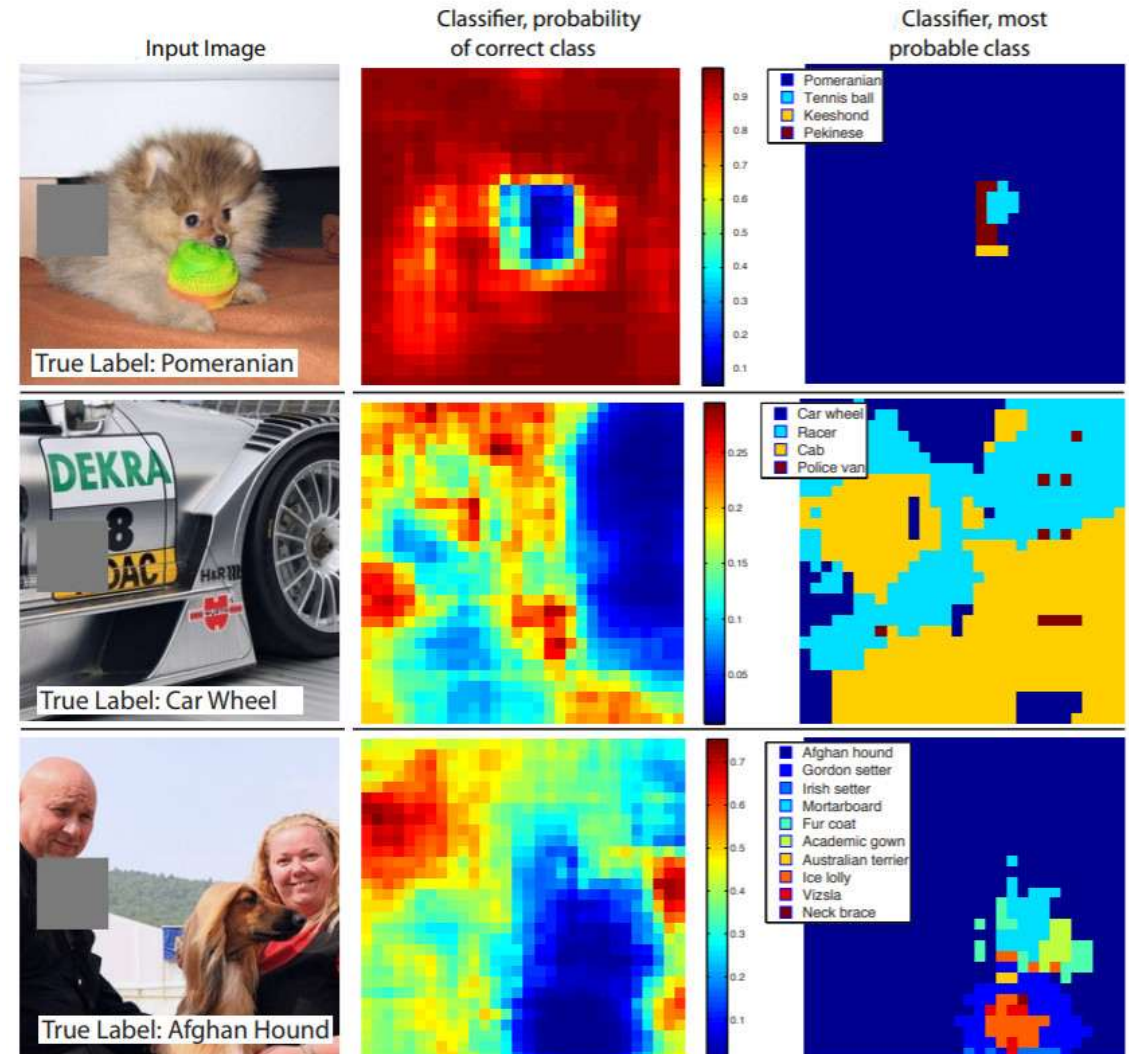


Not classified
as horse



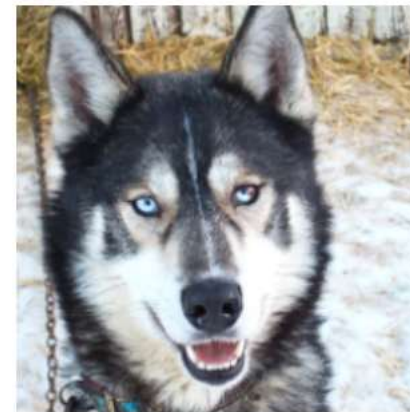
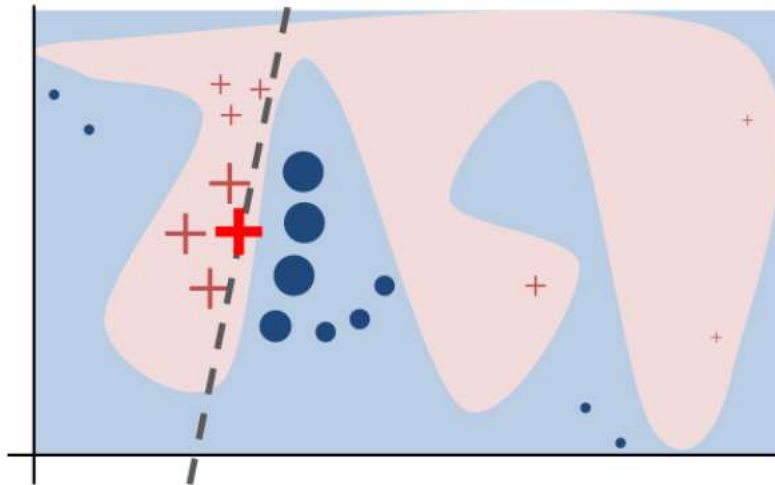
Occlusion Sensitivity

- **Occlusion sensitivity** studies how hiding parts of the input impacts the network's confidence
 - Might decrease or increase it
 - Computation involves a large number of forward passes
 - depends on desired resolution
 - Results depend on size and shape of the occluder

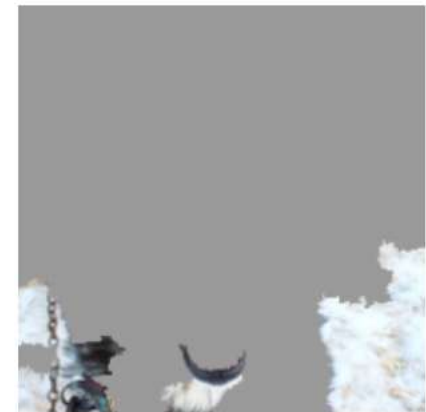


LIME: Main Idea

- **Local Interpretable Model-Agnostic Explanations (LIME)**
 - decomposes the image into superpixels and observes the effects of masking out different combinations of them
 - Fits an **interpretable** (sparse linear) **regression model** to results and visualizes the superpixels that received the largest positive weights
 - Perturbation-based – requires a large number of forward passes



(a) Husky classified as wolf



(b) Explanation

LIME: Further Details

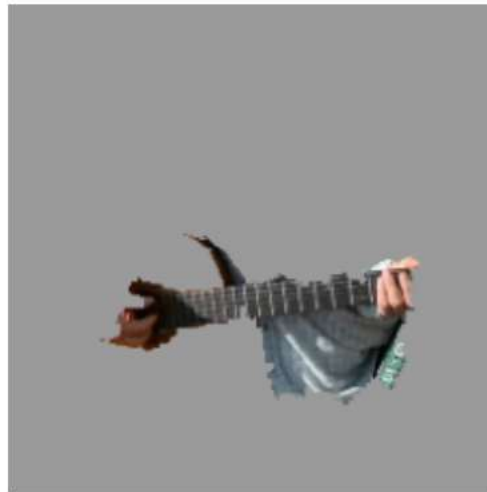
- LIME regression loss:

$$\mathcal{L}(f, g, \pi) = \sum_{z, z' \in \mathcal{Z}} \pi_x(z) (f(z) - g(z'))^2$$

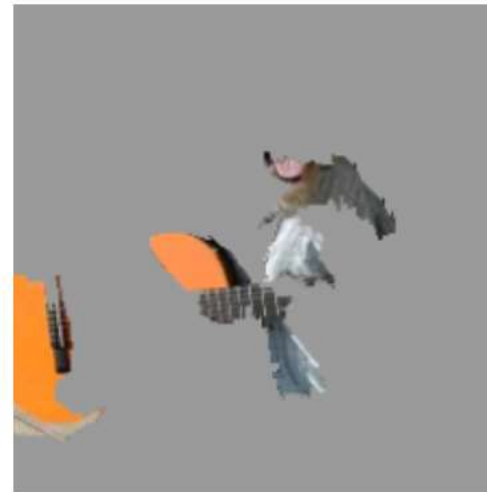
- z' denotes binary vector (superpixel on/off), z the corresponding image
- Class probability $f(z)$, linear model $g(z')$, proximity weight $\pi_x(z)$
- *Consequence*: Do not mis-interpret images as a “sufficient subset”!
 - Classifier might not even have been evaluated on the visual explanation!



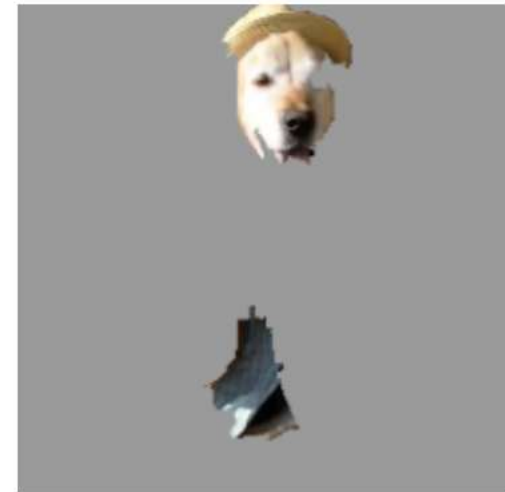
(a) Original Image



(b) Explaining *Electric guitar*



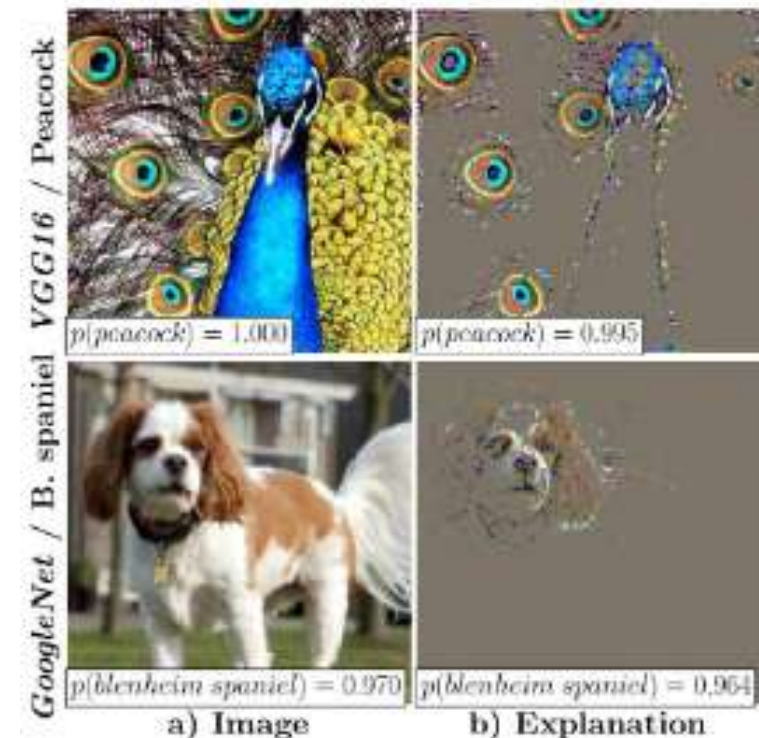
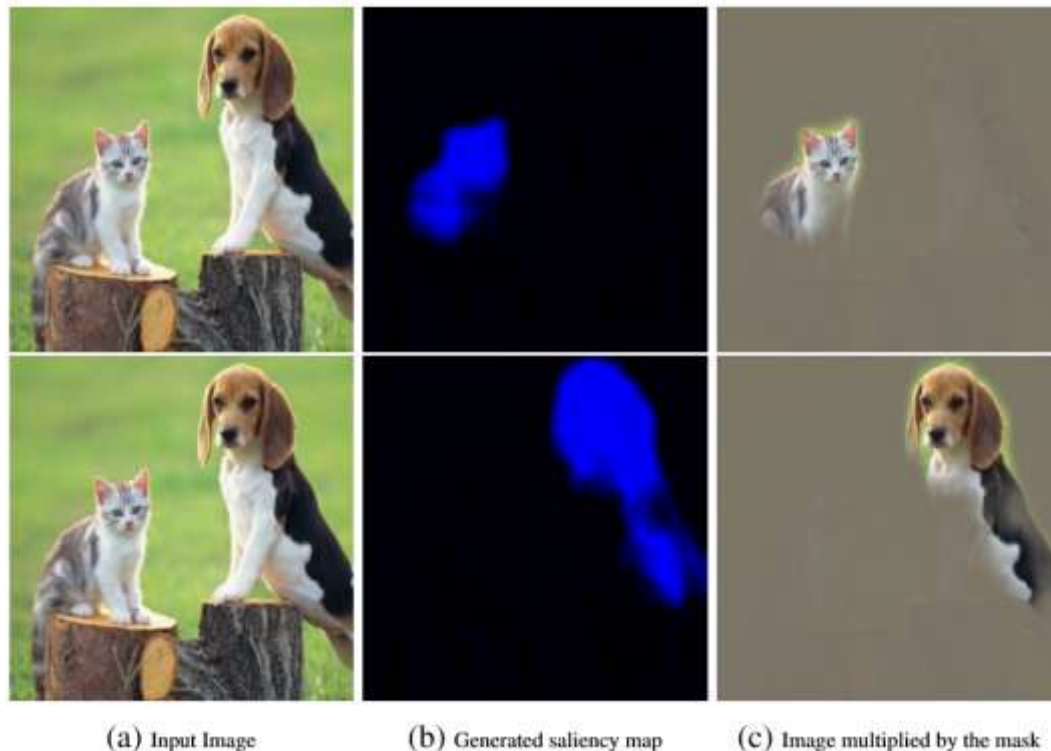
(c) Explaining *Acoustic guitar*



(d) Explaining *Labrador*

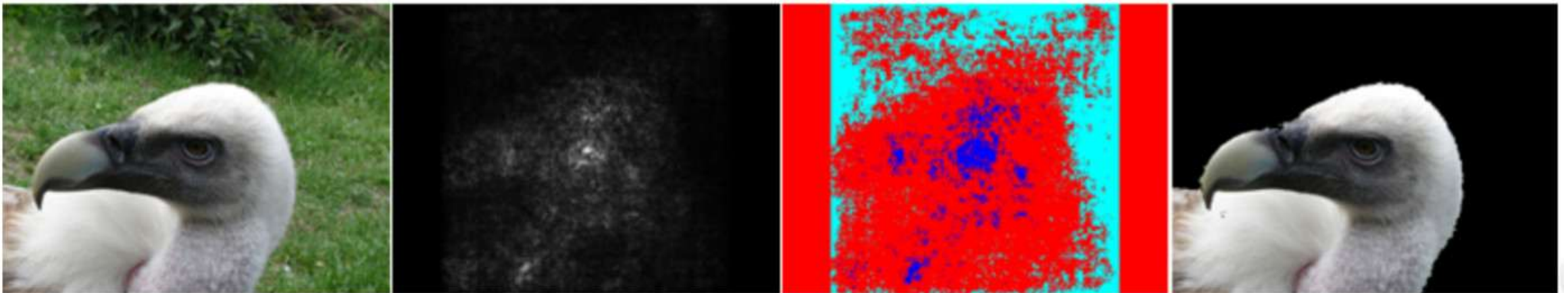
Targeted Deletion / Preservation

- Optimization-based perturbation approaches
 - **Preserve** the smallest image region that maintains original output
 - **Remove** the smallest image region that changes the output
- Masks depend on exact formalization (incl. regularization)



Saliency Maps / Backpropagation

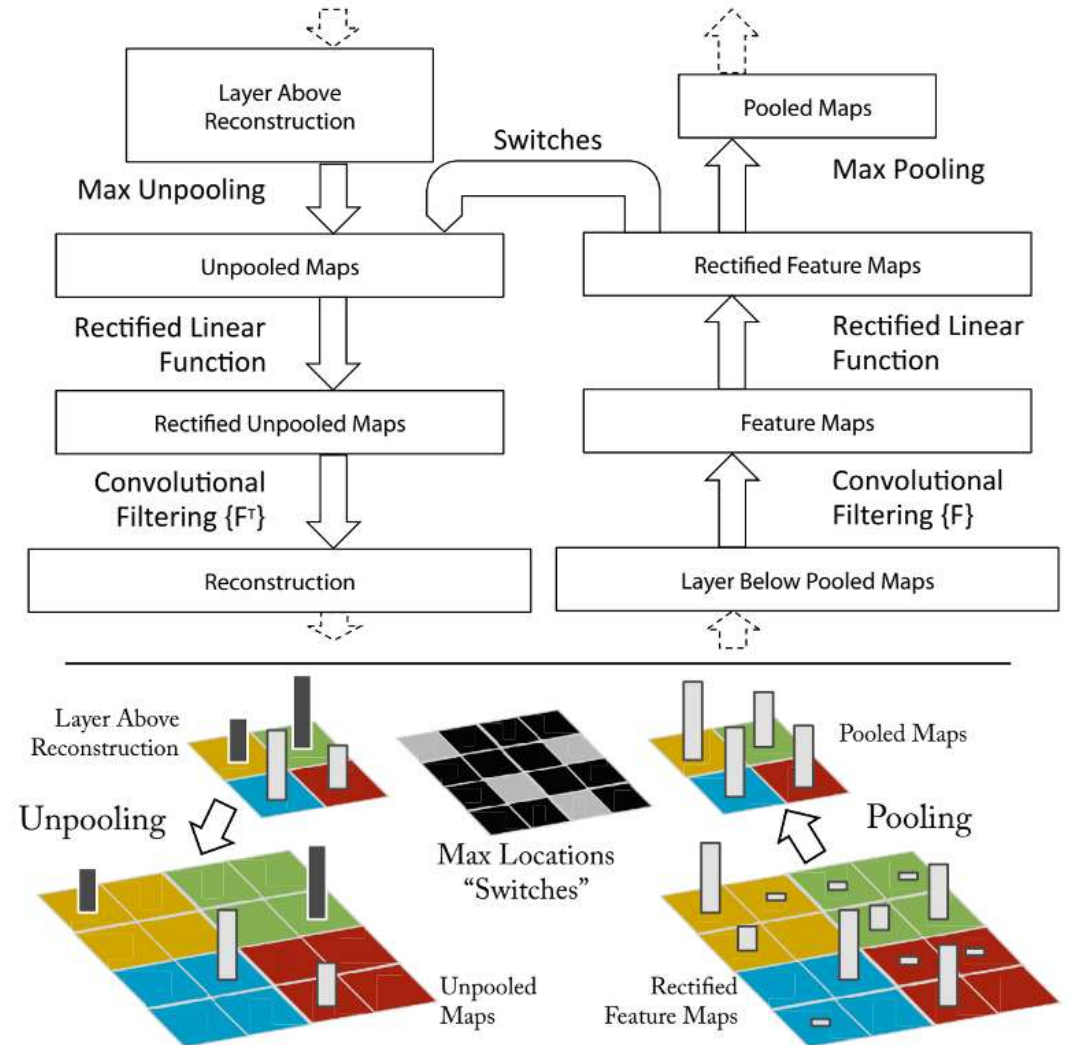
- **Saliency maps** answer the following question:
Which pixels would we have to perturb the least to affect the network's prediction the most?
 - Achieved by backpropagating the class score (not probability!) to the input $\nabla_{\mathbf{x}} S_{\text{vulture}}$ and taking the maximum over color channels
 - Has been used as a basis for segmentation



Example from [Simonyan et al., arXiv 2013]

Deconvolutional Networks

- **Deconvolutional networks** have been designed to map network outputs or activations back to the input space
- Attempt to reverse all steps, to the extent possible:
 - Pooling replaced by unpooling
 - Rectification (*not* based on mask from forward pass)
 - Filtering via transposed convolution



Transposed Convolution

- In **transposed convolution**, the input map elements multiply the “up convolution filter” and are added together in the output:

<table border="1"><tr><td>5</td><td>-2</td></tr><tr><td>3</td><td>1</td></tr></table>	5	-2	3	1	$\ast T$	<table border="1"><tr><td>1</td><td>0</td></tr><tr><td>-1</td><td>2</td></tr></table>	1	0	-1	2	$=$	<table border="1"><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>										• 2x2 up conv. filter • Stride=1
5	-2																					
3	1																					
1	0																					
-1	2																					
Input map		Up conv. filter		Output map																		

Transposed Convolution

- In **transposed convolution**, the input map elements multiply the “up convolution filter” and are added together in the output:

5	-2
3	1

Input map

*T

1	0
-1	2

Up conv. filter

=

5	0	
-5	10	

Output map

- 2x2 up conv. filter
- Stride=1

Transposed Convolution

- In **transposed convolution**, the input map elements multiply the “up convolution filter” and are added together in the output:

5	-2
3	1

Input map

*T

1	0
-1	2

Up conv. filter

=

5	-2	0
-5	12	-4

Output map

- 2x2 up conv. filter
- Stride=1

Transposed Convolution

- In **transposed convolution**, the input map elements multiply the “up convolution filter” and are added together in the output:

5	-2
3	1

Input map

*T

1	0
-1	2

Up conv. filter

=

5	-2	0
-2	12	-4
-3	6	

Output map

- 2x2 up conv. filter
- Stride=1

Transposed Convolution

- In **transposed convolution**, the input map elements multiply the “up convolution filter” and are added together in the output:

<table border="1"><tr><td>5</td><td>-2</td></tr><tr><td>3</td><td>1</td></tr></table>	5	-2	3	1	$\ast T$	<table border="1"><tr><td>1</td><td>0</td></tr><tr><td>-1</td><td>2</td></tr></table>	1	0	-1	2	$=$	<table border="1"><tr><td>5</td><td>-2</td><td>0</td></tr><tr><td>-2</td><td>13</td><td>-4</td></tr><tr><td>-3</td><td>5</td><td>2</td></tr></table>	5	-2	0	-2	13	-4	-3	5	2	• 2x2 up conv. filter • Stride=1
5	-2																					
3	1																					
1	0																					
-1	2																					
5	-2	0																				
-2	13	-4																				
-3	5	2																				
Input map		Up conv. filter		Output map																		

Transposed Convolution

- In **transposed convolution**, the input map elements multiply the “up convolution filter” and are added together in the output:

<table border="1"><tr><td>5</td><td>-2</td></tr><tr><td>3</td><td>1</td></tr></table>	5	-2	3	1	$\ast T$	<table border="1"><tr><td>1</td><td>0</td></tr><tr><td>-1</td><td>2</td></tr></table>	1	0	-1	2	$=$	<table border="1"><tr><td>5</td><td>-2</td><td>0</td></tr><tr><td>-2</td><td>13</td><td>-4</td></tr><tr><td>-3</td><td>5</td><td>2</td></tr></table>	5	-2	0	-2	13	-4	-3	5	2	• 2x2 up conv. filter • Stride=1
5	-2																					
3	1																					
1	0																					
-1	2																					
5	-2	0																				
-2	13	-4																				
-3	5	2																				
Input map		Up conv. filter		Output map																		

- Deconvnet flips the filters from forward pass (horizontally and vertically)
 - Transposed convolution with flipped filters also occurs when computing gradients w.r.t. input via backpropagation through convolutional layers
 - Deconvnets and backpropagation are similar except for their handling of ReLUs

Idea of Guided Backpropagation

- **Backpropagation** through ReLU involves setting derivatives to zero wherever a *negative input* was present in the forward pass
- The **Deconvnet** sets *negative derivatives* to zero
 - *Intuition*: Visualize what has a positive effect on the class/neuron of interest
- **Guided backpropagation** sets derivatives to zero in either case

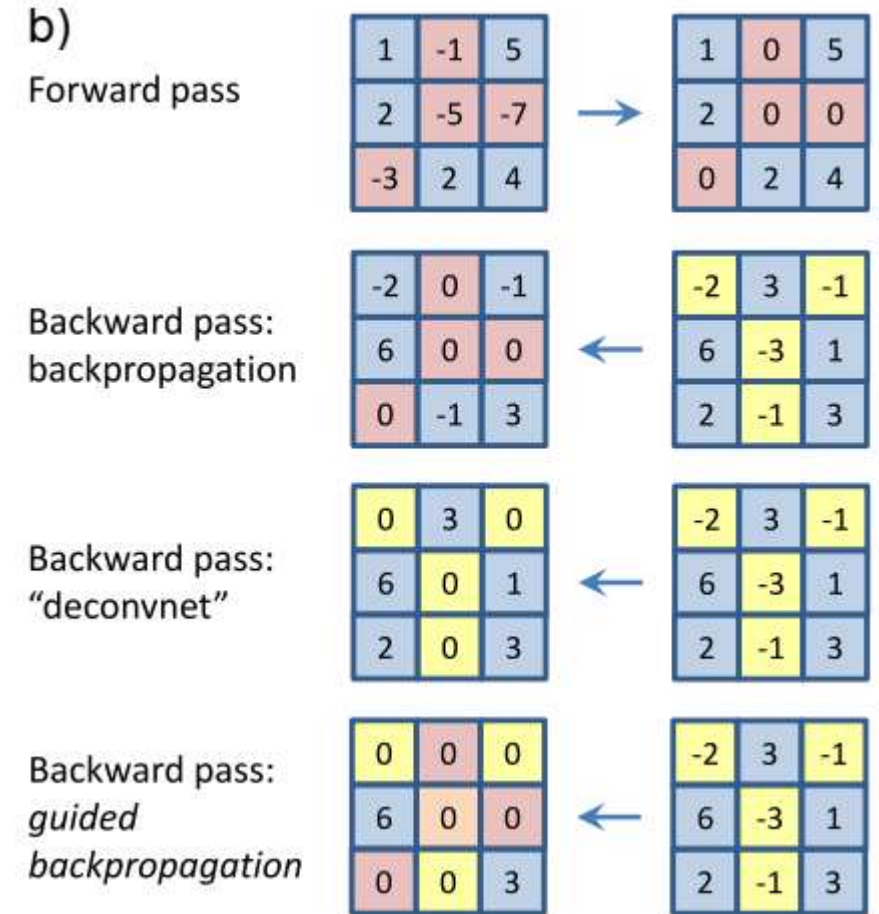
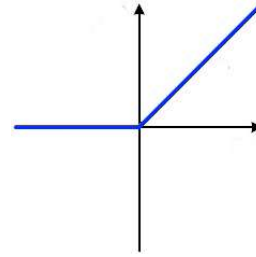
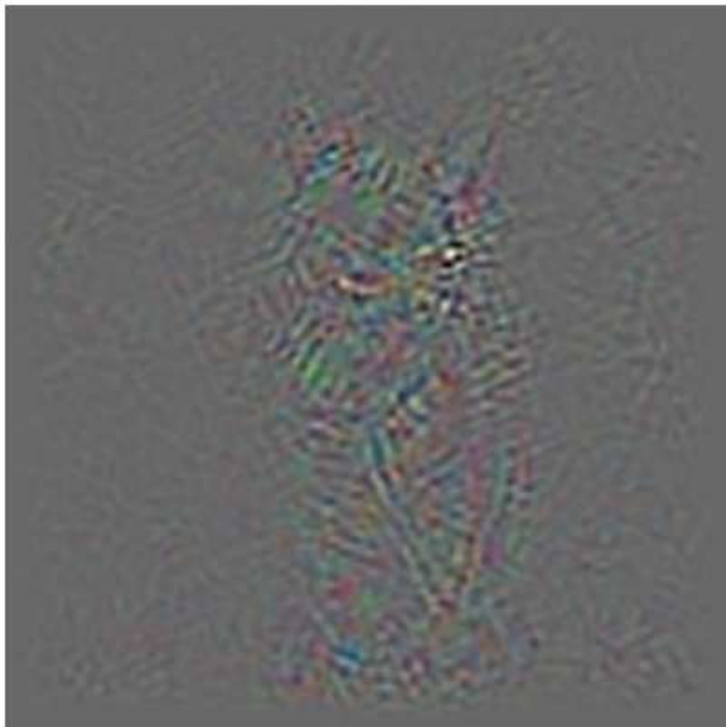


Image from [Springenberg et al. 2015]

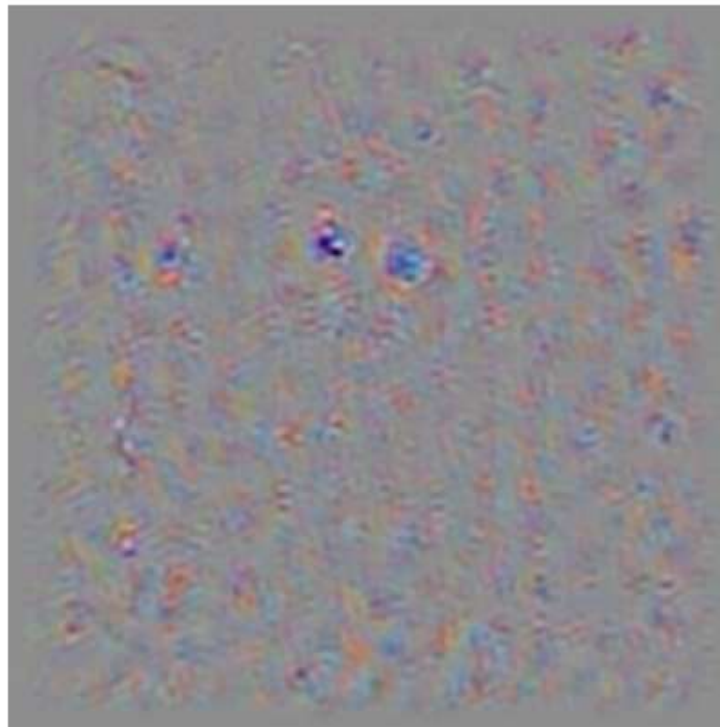
Results from Guided Backpropagation

- Guided backpropagation often produces nicer-looking results than standard backpropagation

backpropagation



'deconvnet'



guided backpropagation



Integrated Gradients: Main Idea

- **Problem:** (Guided) backpropagation misses relevant parts
 - **Local slope $\neq$ Importance:** Pixels might matter even if small changes to their precise value do not (e.g., due to gradient saturation)
- **Sensitivity axiom** [Sundararajan et al. 2017]:
 - A feature must get non-zero attribution if prediction differs between input and baseline in which the feature differs (e.g., pixel is black)
- **Integrated gradients** are based on a path integral between input x and a chosen baseline x' (e.g., black image):
 - i refers to an individual feature (one color channel of one pixel)

$$\text{IG}_i(x) := (x_i - x'_i) \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

Integrated Gradients: Further Axioms, Implementation

- Integrated gradients satisfy a **completeness axiom**:

$$\sum_i \text{IG}_i(x) = f(x) - f(x')$$

- “Attributions add up to the difference between image and baseline”
 - Fundamental theorem of calculus
- Integrated gradients satisfy **implementation invariance**
 - Functionally identical networks get identical attributions
 - Chain rule
- Integrated gradients are computed as **Riemann sums**
 - 20-300 steps, paper recommends verifying completeness

Integrated Gradients: Results and Caveats

- Integrated gradients
 - provide plausibly more complete attribution
 - depend on the choice of baseline
 - Black image as baseline does not reveal relevance of black pixels
 - increase computational expense

Original image



Top label and score

Top label: reflex camera
Score: 0.993755



Top label: fireboat
Score: 0.999961



Top label: school bus
Score: 0.997033

Integrated gradients



Gradients at image



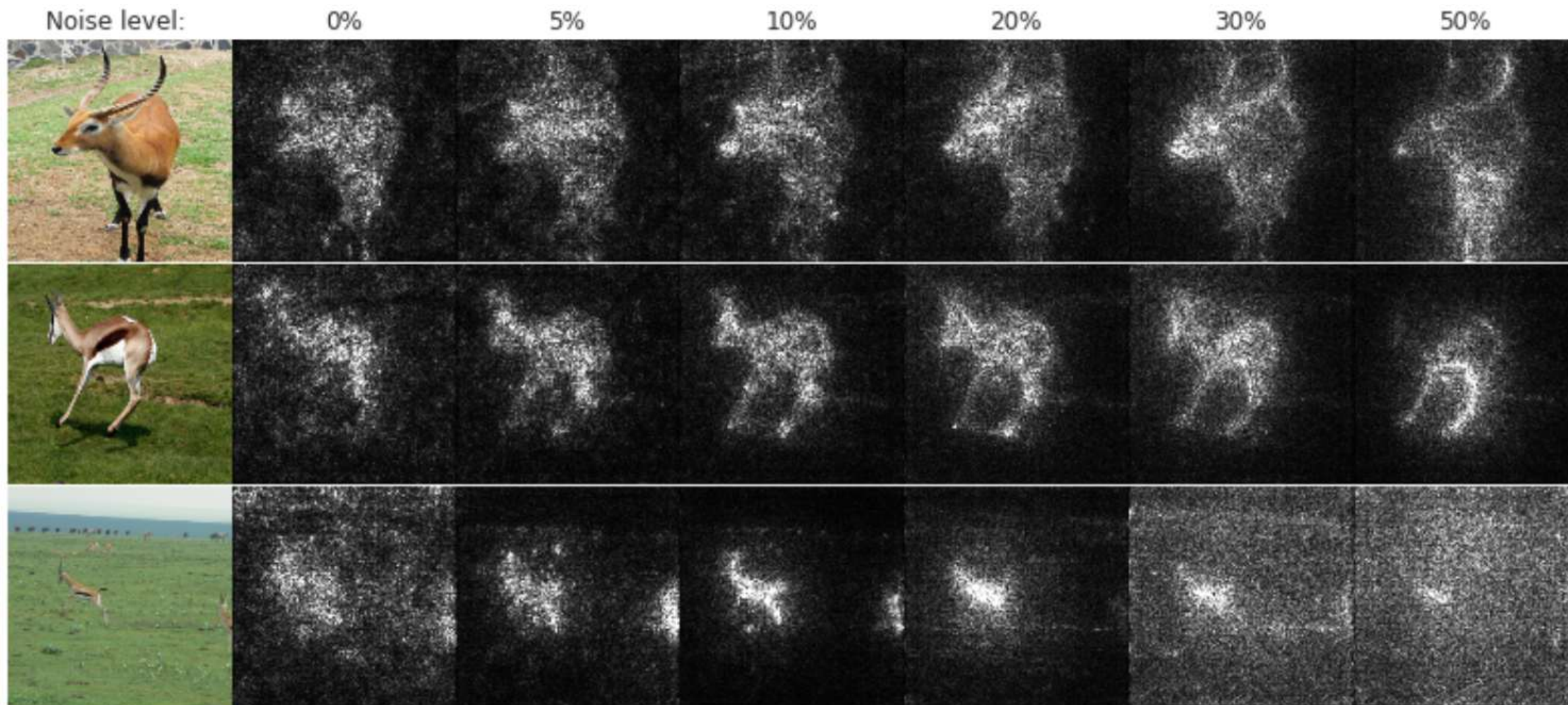
SmoothGrad: Idea

- **Problem:** CNN structure (piecewise linear activations, max pooling) introduces high-frequency gradient noise
- **SmoothGrad** reduces noise by averaging attribution maps $M(x)$ over multiple copies to which noise has been added

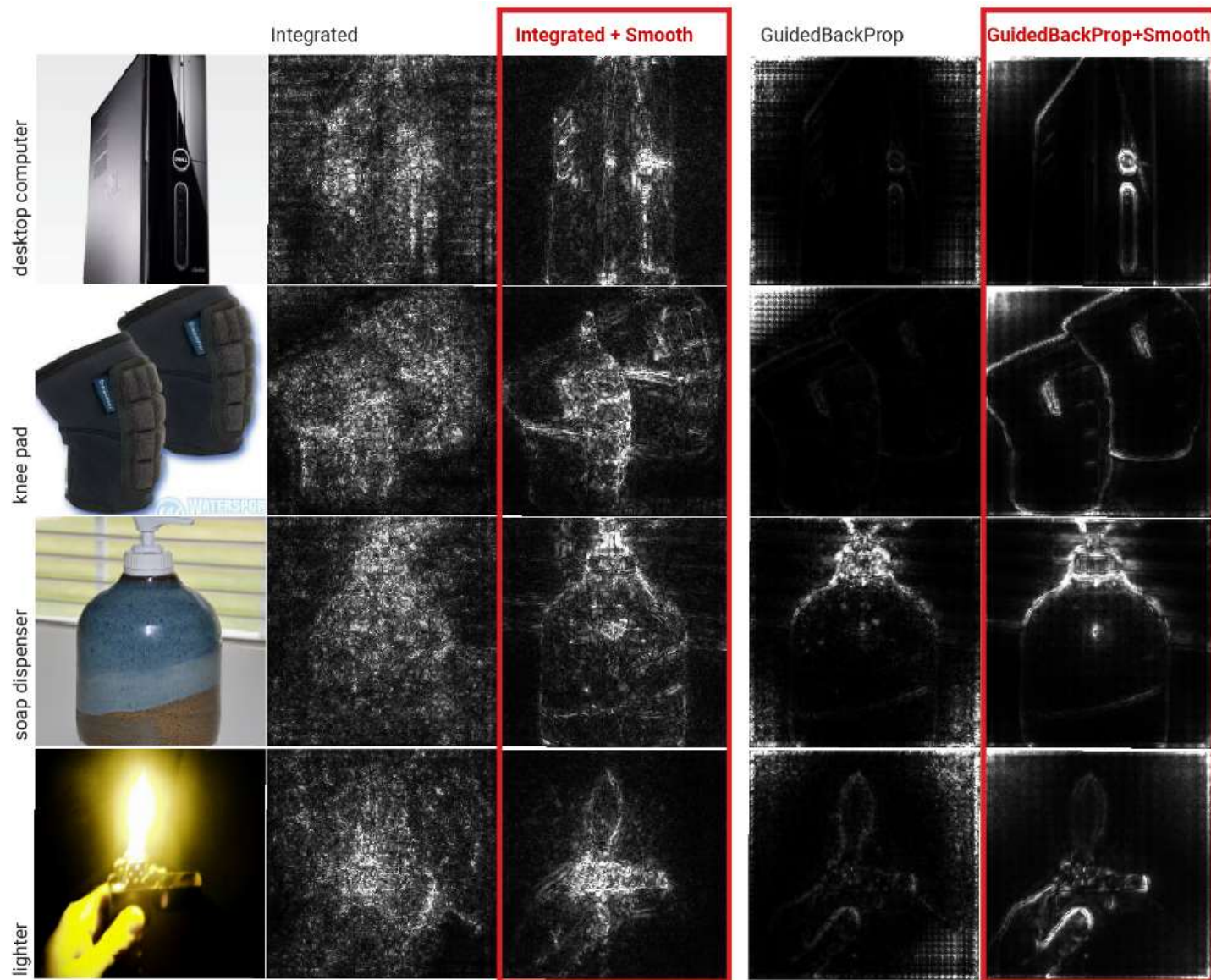
$$\hat{M}(x) = \frac{1}{n} \sum_{1}^n M(x + \mathcal{N}(0, \sigma^2))$$

- Can be combined with any gradient-based attribution method
- Requires tuning σ^2 so that it affects gradients without destroying semantic structure of the image
- Increases computational effort by factor n

SmoothGrad: Impact of Noise Level



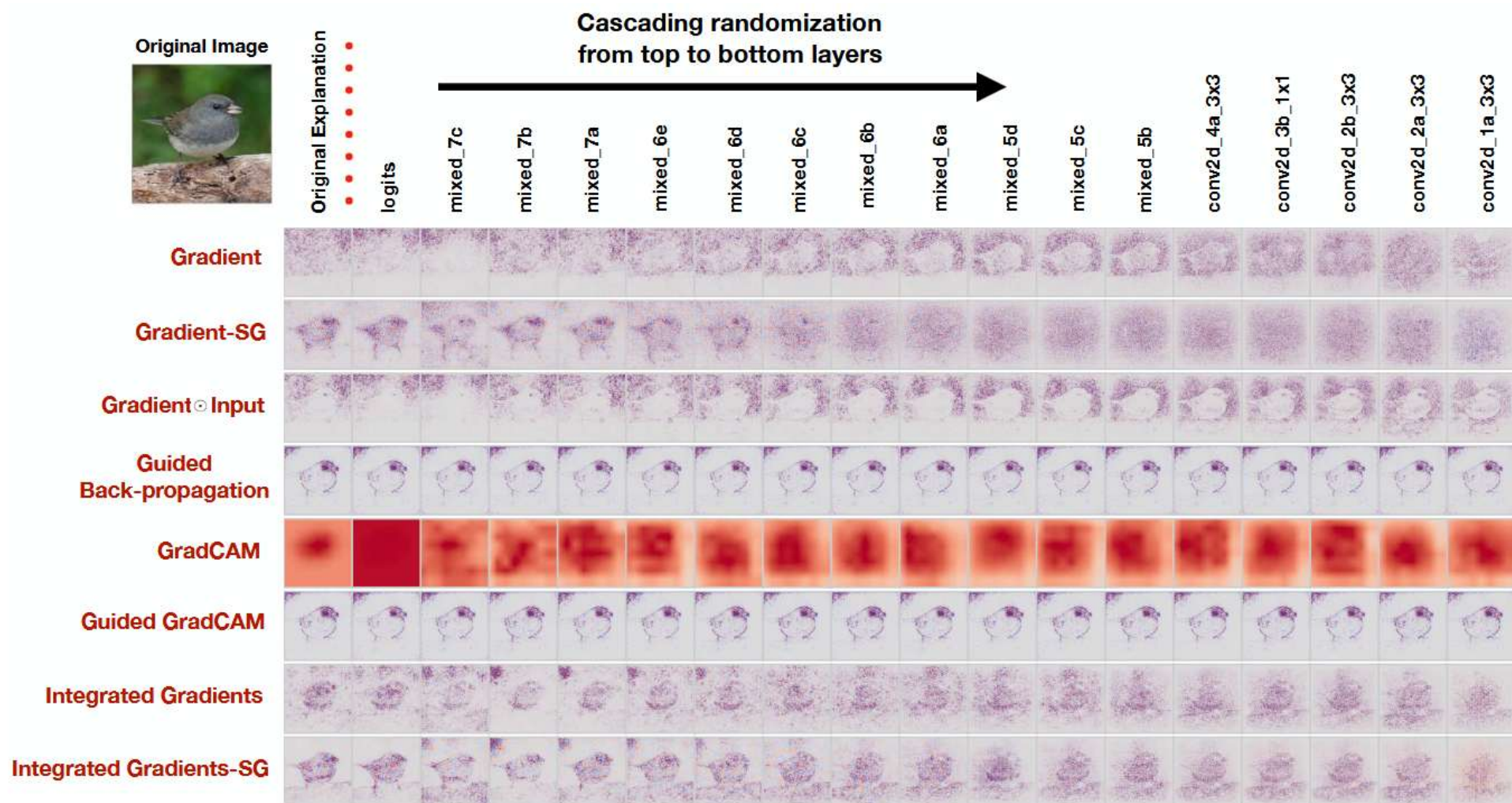
SmoothGrad: Example Results



Sanity Checks for Saliency Maps

- **Plausibility / visual clarity** of attribution maps does not guarantee faithfulness to the underlying model
- **Sanity check** by [Adebayo et al. 2020]:
 - Randomizing weights makes networks unable to perform a meaningful classification
 - Randomization performed in a cascading fashion, from top to bottom
 - If attribution maps reveal anything meaningful about the network's function, they should be affected by such randomization
 - Results of many approaches are suspiciously stable (see next slide)
 - Guided backpropagation in particular resembles edge detection

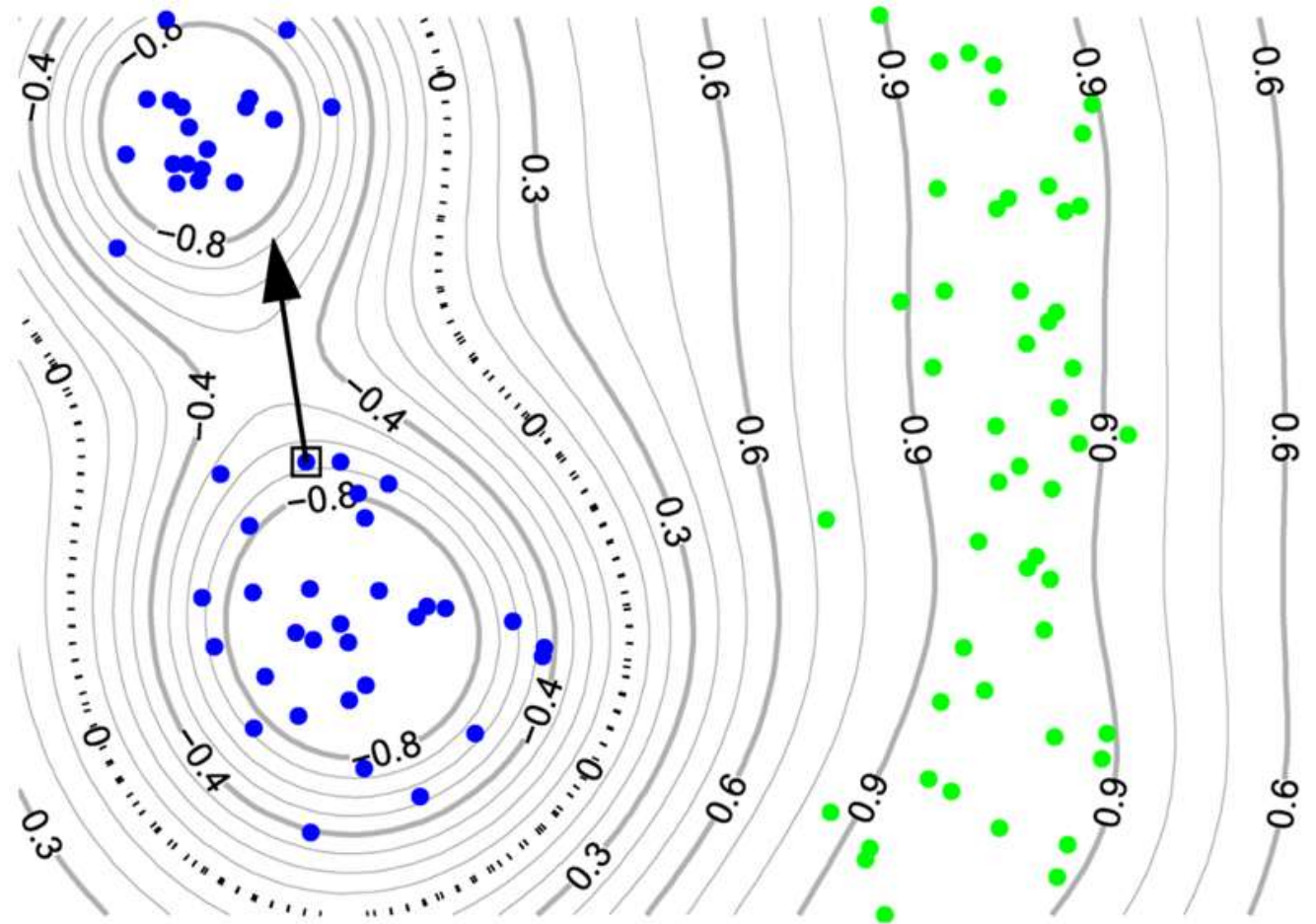
Sanity Checks for Saliency Maps: Outcome



Quiz: Why are integrated gradients relatively stable, despite completeness?

An Objection Against Gradient-based Methods

- **Possible failure case:**
The gradient of the classification function does not necessarily point towards a close point on the decision boundary



Layer-Wise Relevance Propagation

- **Layer-wise relevance propagation** attempts to answer the question “Which pixels contributed most to the outcome?” rather than “Which pixels would we have to perturb to affect the current outcome?”
- Each neuron i (including the inputs, i.e., pixels) is assigned a **relevance score** R_i
 - Relevances are **conserved**: At each layer, the sum of relevance scores of all neurons should be the same, and equal to the output score
- Given some measure q_{ij} of how much input neuron i activates output neuron j , relevance is propagated layer-wise according to

$$R_{i \leftarrow j} = \frac{q_{ij}}{\sum_i q_{ij}} R_j$$

Example: LRP in a CNN

- In a neural network with

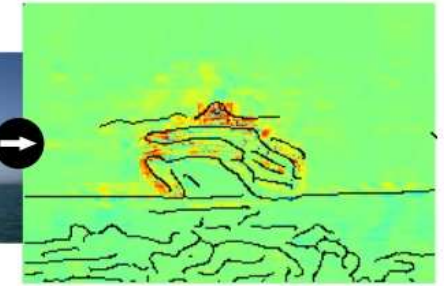
$$y_j = \text{ReLU} \left(\sum_{i=0}^m w_{ij} x_i \right)$$

one way to apply LRP is to set

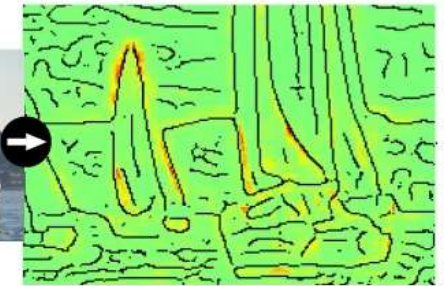
$$q_{ij} = \text{ReLU}(w_{ij} x_i)$$

- In max pooling layers, relevance is only propagated to the pixel that had the maximum activation
 - Leads to sparse high-resolution maps

Important features for individual predictions

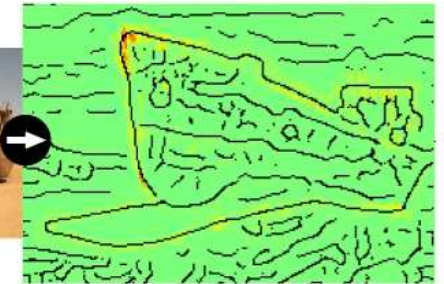


"To detect this boat look at the wheelhouse!"



"To detect this boat look at the sails!"

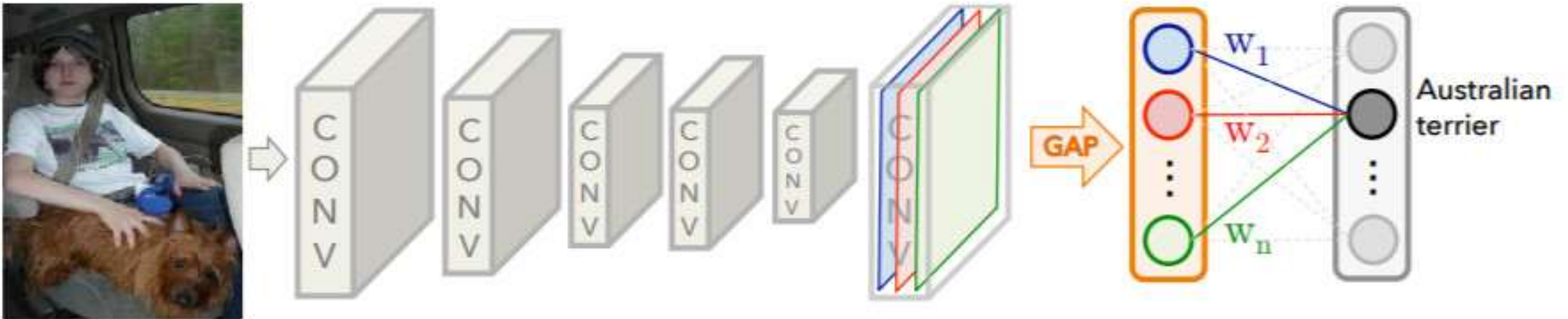
...



"To detect this boat look at the bow!"

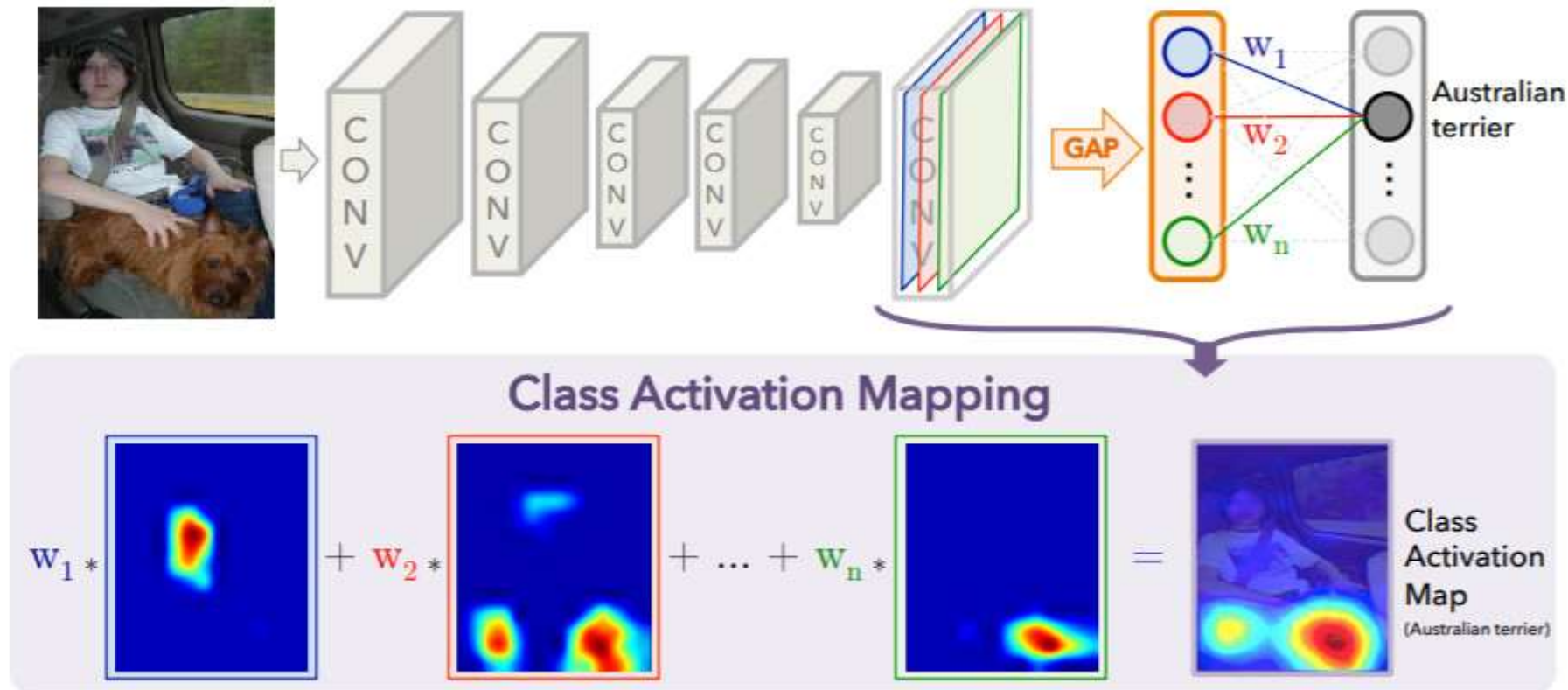
Class Activation Maps: Global Average Pooling

- **Class activation maps** are an alternative for localizing objects using networks trained for classification
 - Neither requires multiple forward passes nor back propagation
 - Assumes that network uses **Global Average Pooling (GAP)**



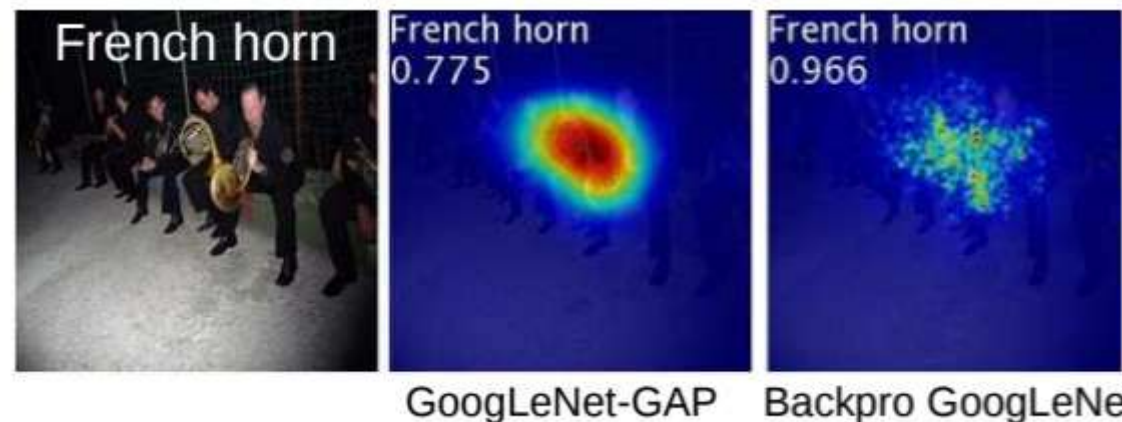
Computing Class Activation Maps

- Class activation maps are simply computed as a **weighted average** of the activations in the final convolution layers
 - Upsampled to original image resolution and overlaid



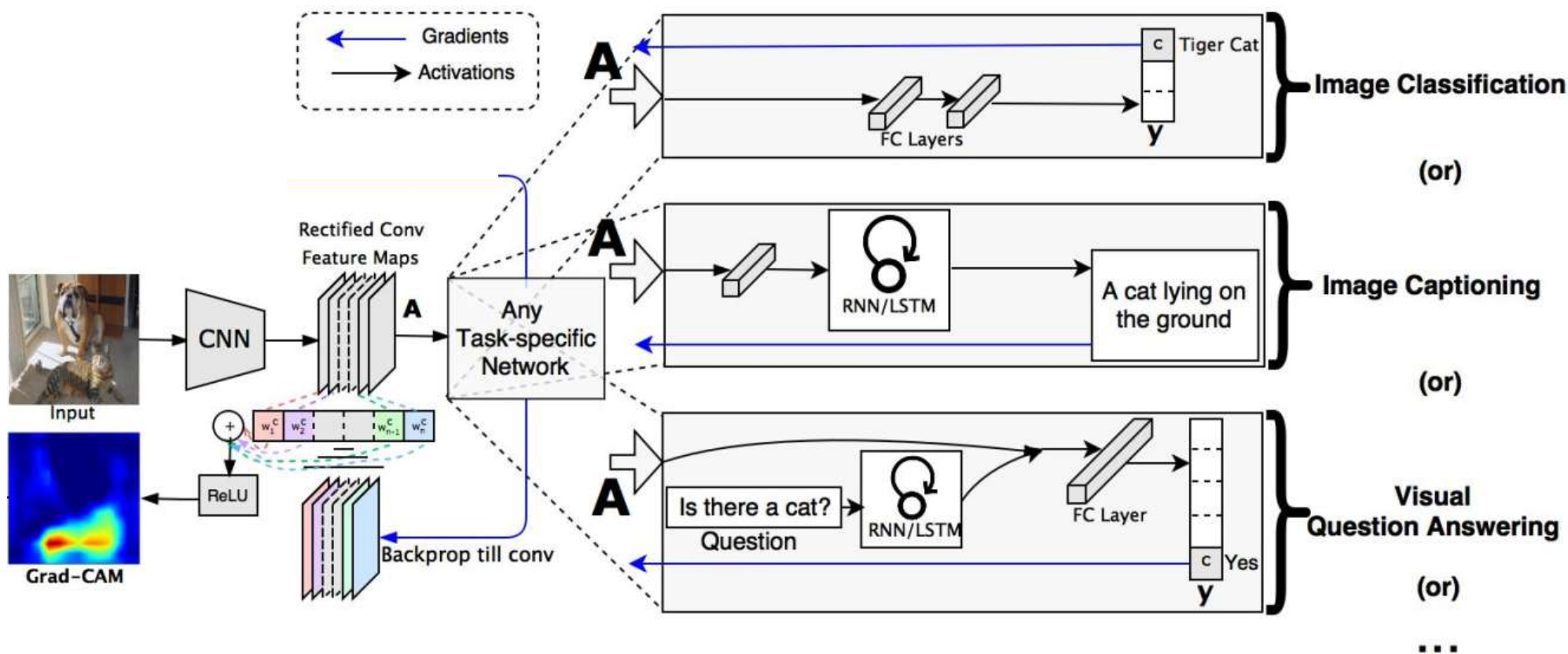
Class Activation Maps: Justification

- **Justification** for why class activation maps directly indicate importance for classification:
 - GAP performs $F_k = \sum_{x,y} f_k(x, y)$
 - Class scores obtained as $S_c = \sum_k w_k^c F_k = \sum_k w_k^c \sum_{x,y} f_k(x, y)$
 - Swapping sums yields $S_c = \sum_{x,y} \sum_k w_k^c f_k(x, y)$
 - Class activation map defined as $M_c(x, y) = \sum_k w_k^c f_k(x, y)$
 - Therefore: $S_c = \sum_{x,y} M_c(x, y)$



Gradient-weighted Class Activation Mapping

- **Grad-CAM** generalizes CAM to arbitrary tasks and architectures
 - Backpropagates output of interest to last convolutional layer
 - Combines activation maps with weights from GAP on gradients



Grad-CAM: Formal Definition

- **Neuron importance weights** α_k^c for activation map k and output score S_c computed via GAP:

$$\alpha_k^c = \frac{1}{Z} \sum_x \sum_y \frac{\partial S_c}{\partial f_k(x, y)}$$

- **Grad-CAM** is computed as the corresponding weighted sum of activation maps, rectified to disregard activations with a negative influence on the desired output:

$$M_{GC}^c = ReLU \left(\sum_k \alpha_k^c f_k \right)$$

- *Note:* Grad-CAM is a proper generalization of CAM

Example Result: Visual Question Answering

In Visual Question Answering, Grad-CAM provides insight on why the network gave a specific answer and why multiple answers might be plausible

- This example uses a 200 layer ResNet



What is the man doing?



Surfing



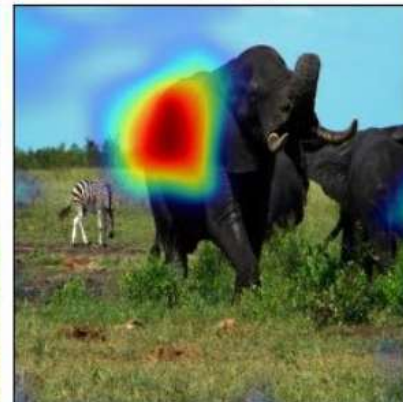
What is the she holding?



Baseball bat



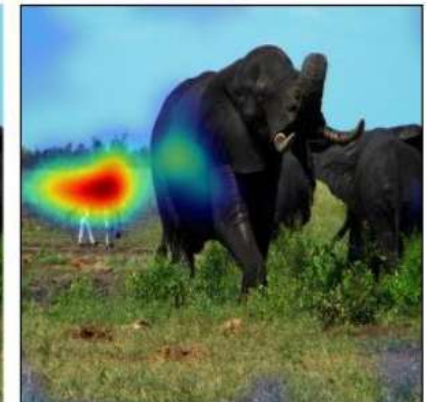
What is that?



Elephant



What is that?



Zebra

Guided Grad-CAM

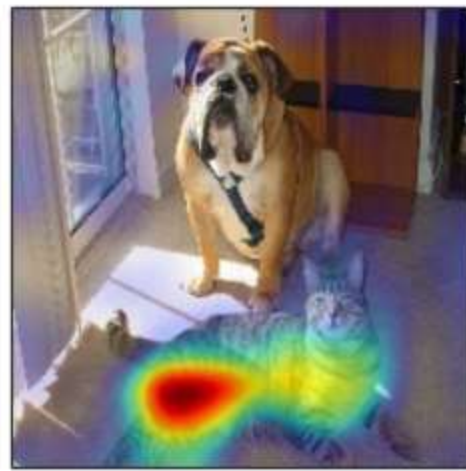
- **Guided Grad-CAM** multiplies the Grad-CAM and guided backpropagation outputs to reveal fine details while increasing class specificity compared to pure guided backpropagation



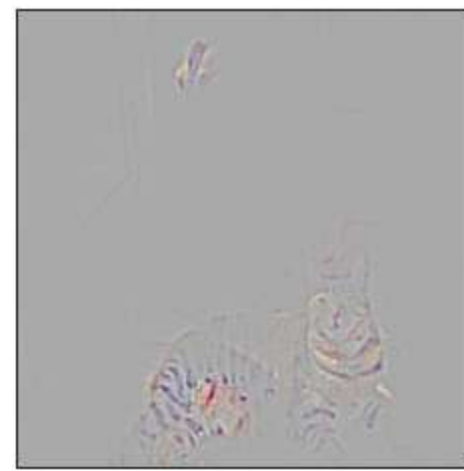
(a) Original Image



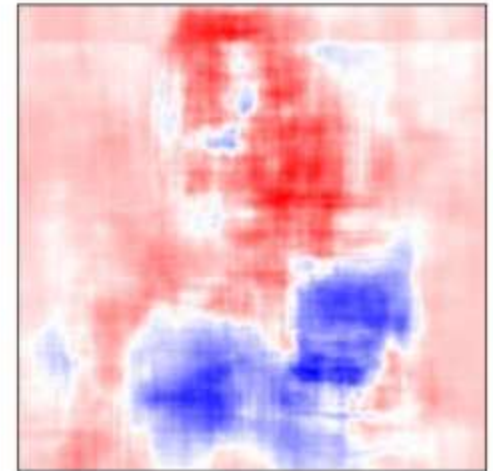
(b) Guided Backprop 'Cat'



(c) Grad-CAM 'Cat'



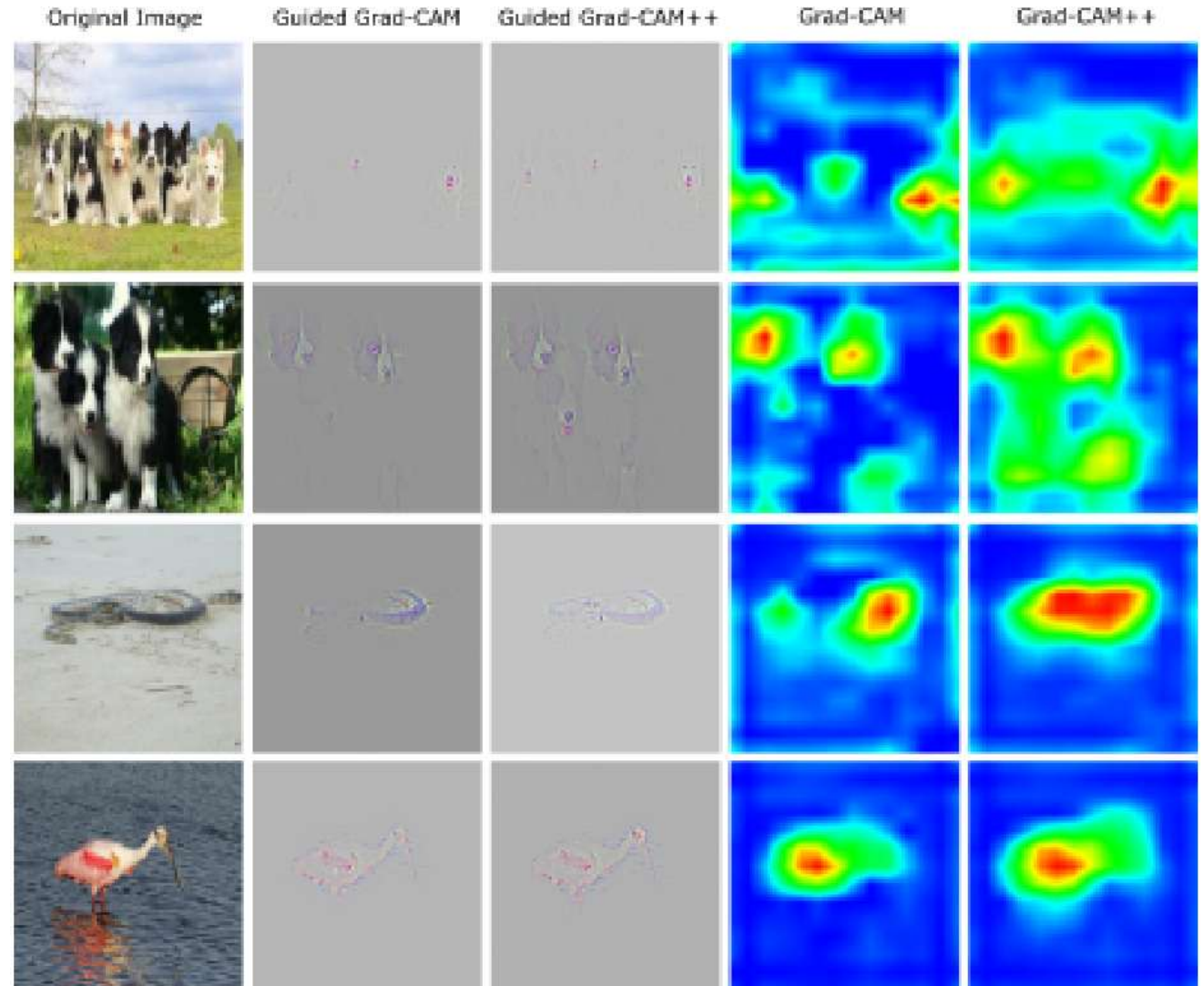
(d) Guided Grad-CAM 'Cat'



(e) Occlusion map 'Cat'

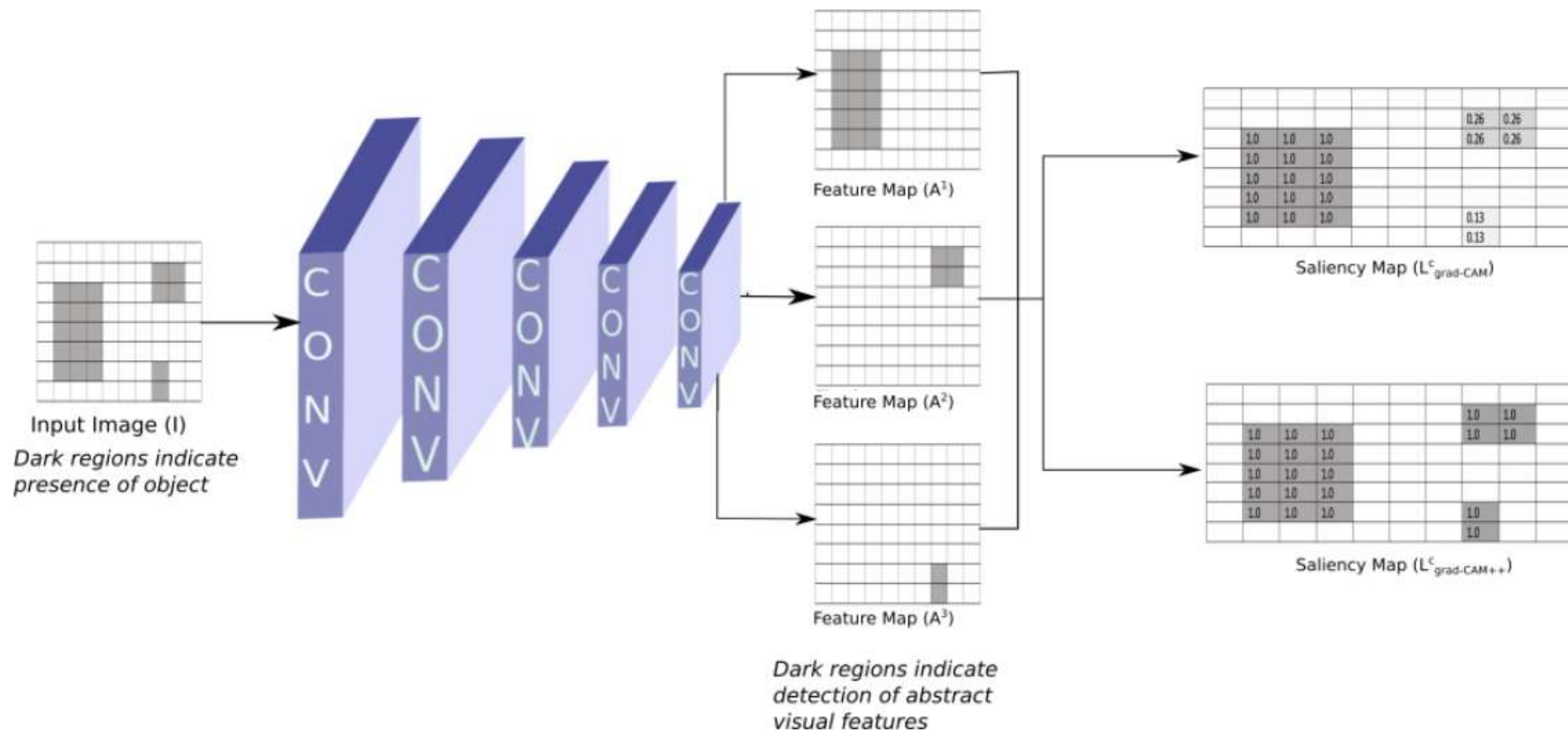
Motivating Grad-CAM++

- **Observation:** Grad-CAM often highlights objects incompletely, or only some of the objects when the same class appears more than once



Intuition: Grad-CAM++

- **Observation:** Computing neuron importance weights via global averaging favours activation maps that include large regions
 - **Idea:** Introduce pixel weights that compensate for this



Grad-CAM++: Main Idea

- Grad-CAM++ aims to restore the interpretation that the spatial sum over an attribution map yields the corresponding output:

$$Y_c = \sum_x \sum_y M_{GC++}^c(x, y) \text{ with } M_{GC++}^c = \sum_k \omega_k^c f_k$$

- Replaces class score S_c with $Y_c = \exp(S_c)$
 - Derivation (next slide) involves repeated differentiation
- Replaces neuron importance α_k^c with a neuron weight

$$\omega_k^c = \sum_x \sum_y p_k^c(x, y) \text{ReLU} \left(\frac{\partial Y_c}{\partial f_k(x, y)} \right)$$

that accounts for per-pixel weighting coefficients $p_k^c(x, y)$

Grad-CAM++: Derivation of Pixel Weights

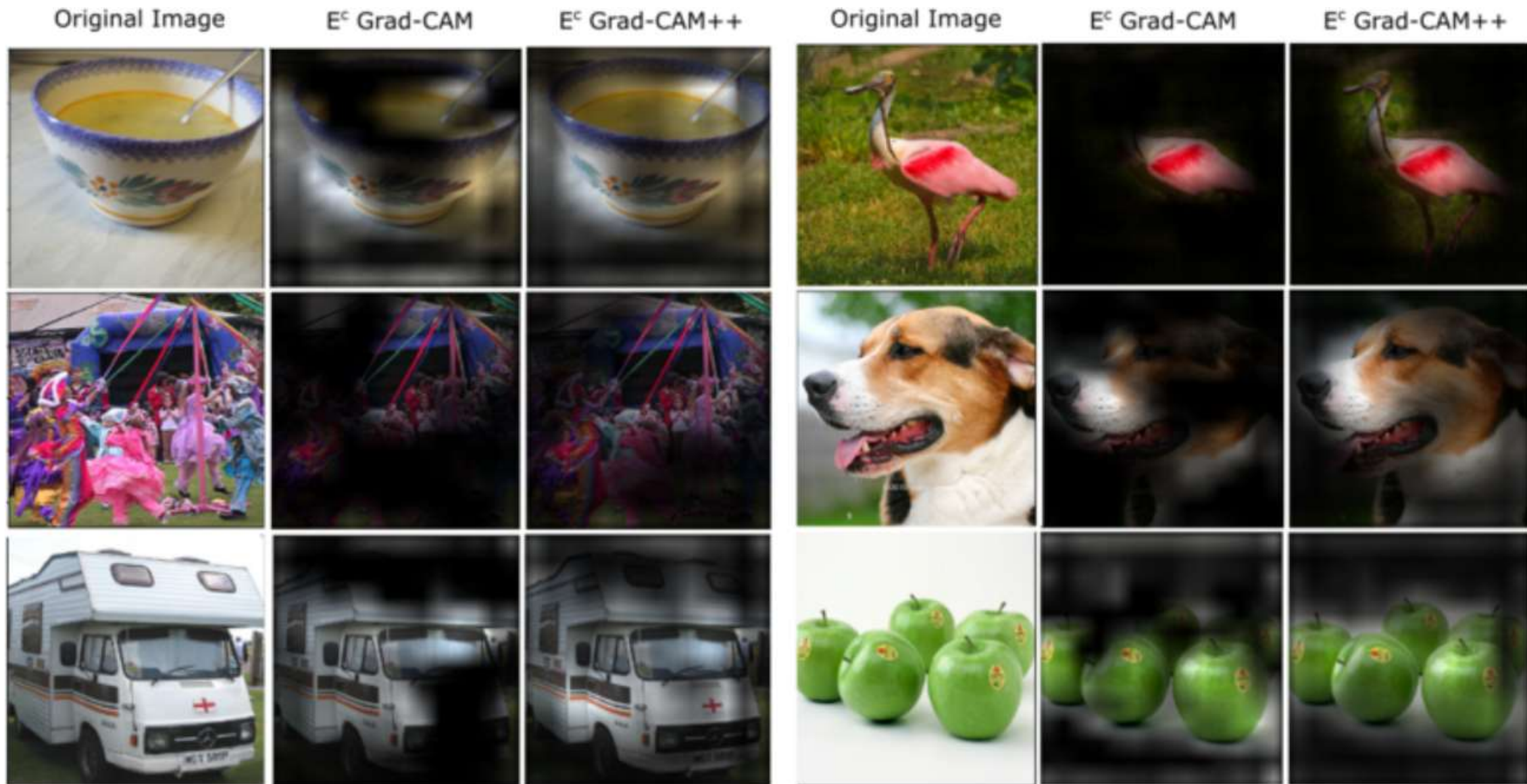
- After combining all equations from the previous slide

$$Y_c = \sum_k \left[\sum_a \sum_b p_k^c(a, b) \text{ReLU} \left(\frac{\partial Y_c}{\partial f_k(a, b)} \right) \right] \sum_x \sum_y f_k(x, y)$$

ignoring the ReLU, and taking partial derivatives w.r.t. $f_k(x, y)$ twice, we obtain

$$p_k^c(x, y) = \frac{\frac{\partial^2 Y_c}{(\partial f_k(x, y))^2}}{2 \frac{\partial^2 Y_c}{(\partial f_k(x, y))^2} + \sum_a \sum_b f_k(a, b) \left[\frac{\partial^3 Y_c}{(\partial f_k(x, y))^3} \right]}$$

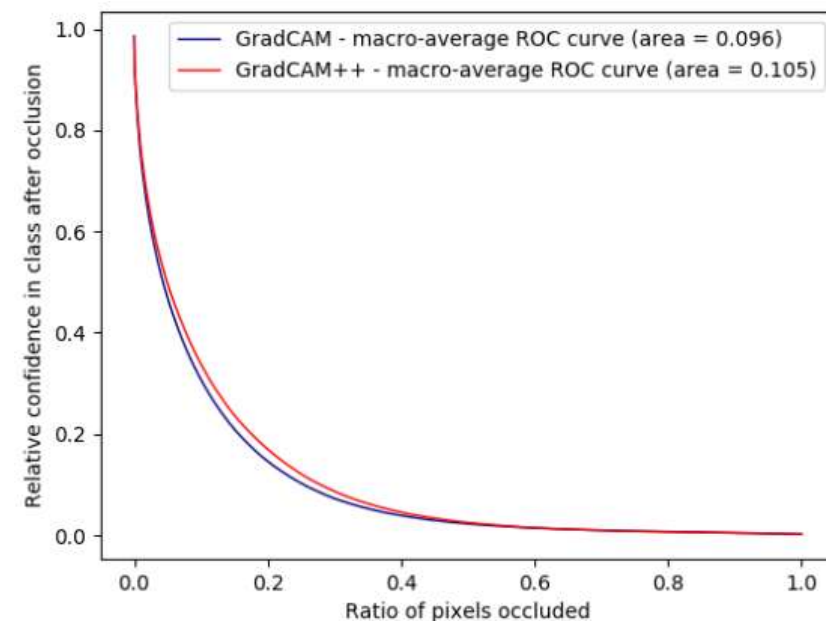
Grad-CAM++: Example Results



Grad-CAM++: Objective Evaluation

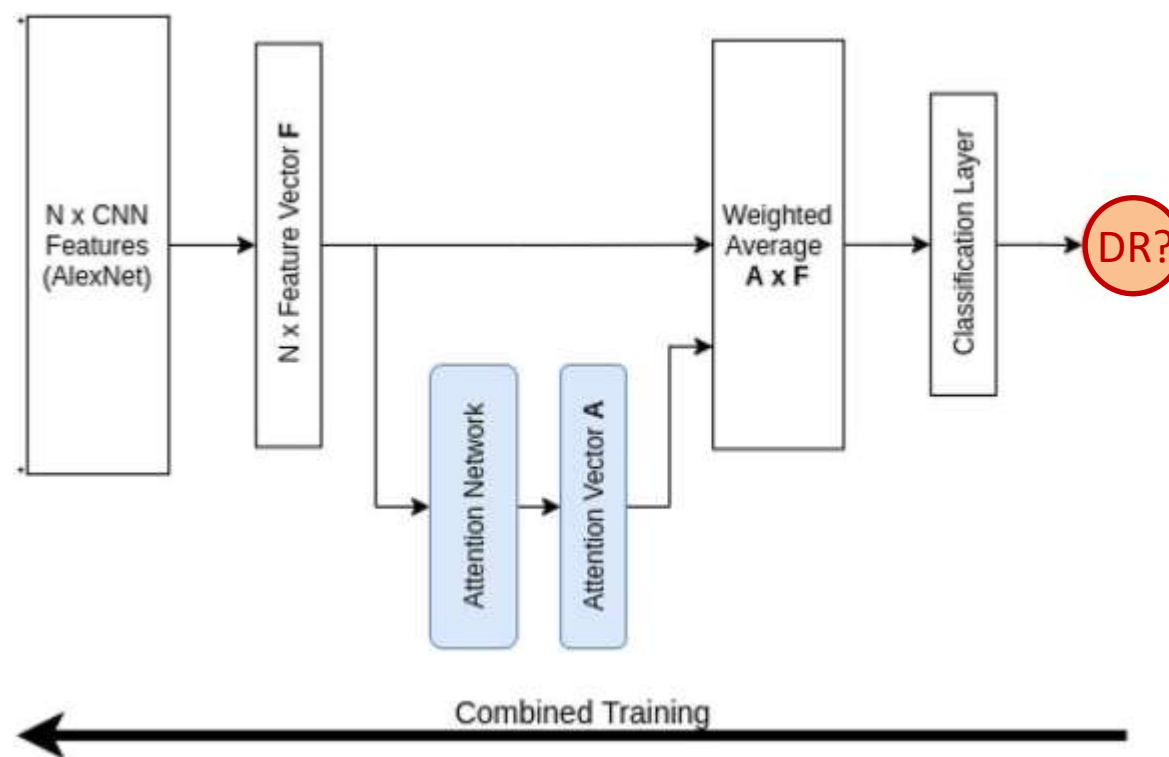
- **Hypothesis:** If attribution map correctly captures the relevant region, point-wise multiplication with image should
 - Not reduce the confidence of the correct class (a lot)
 - Some drop might be explained by making the image less natural
 - Even increase the confidence in some cases
 - When competing classes are masked out

Method	Grad-CAM++	Grad-CAM
Average Drop % (Lower is better)	36.84	46.56
% Incr. in Confidence (Higher is better)	17.05	13.42
Win % (Higher is better)	70.72	29.28



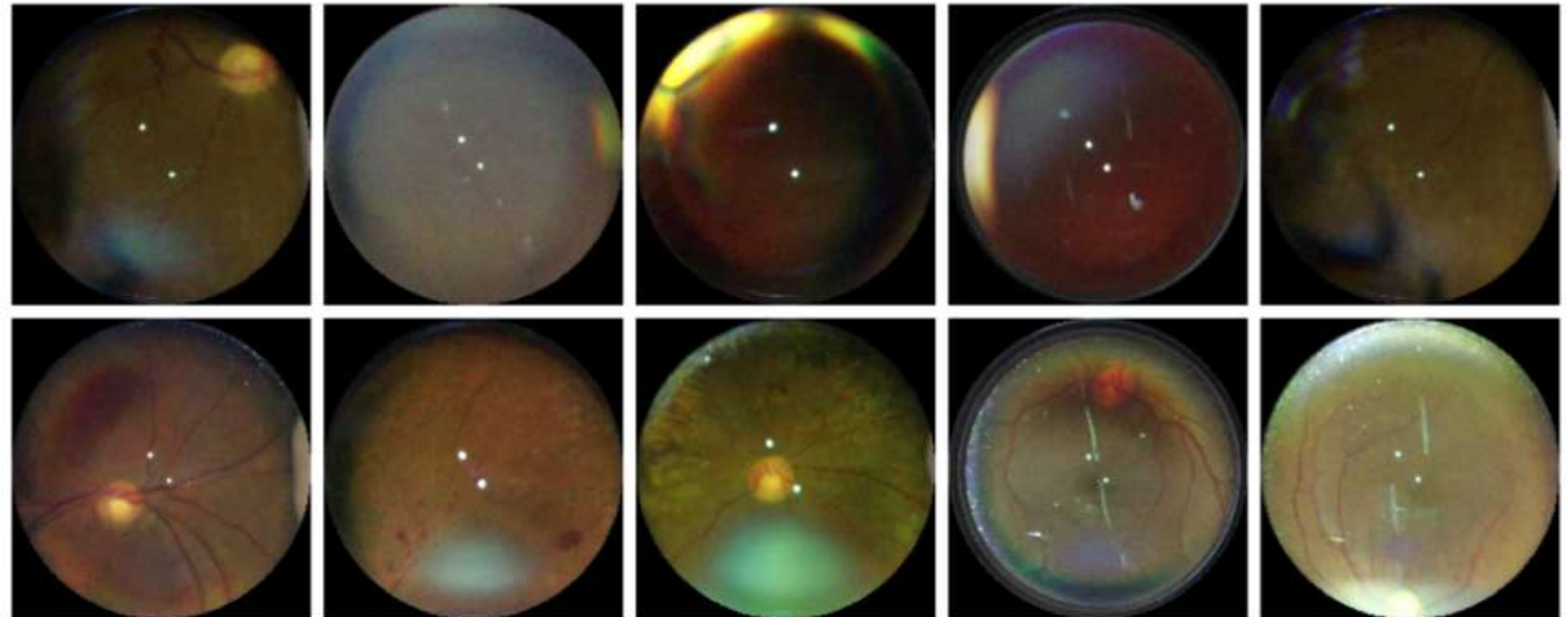
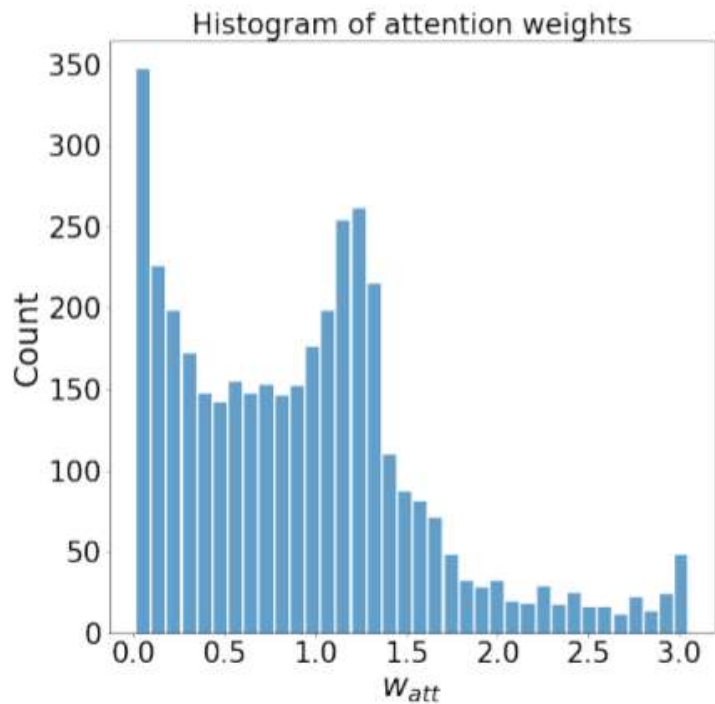
Attention

- **Attention** permits data-adaptive weighting when fusing parts of the input (e.g., frames over time, patches over space)
 - Often increases accuracy by suppressing irrelevant information



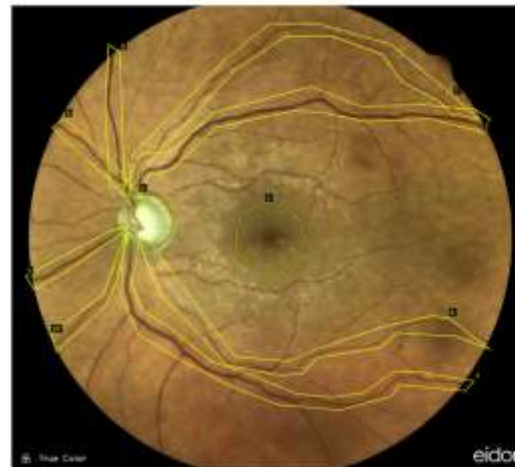
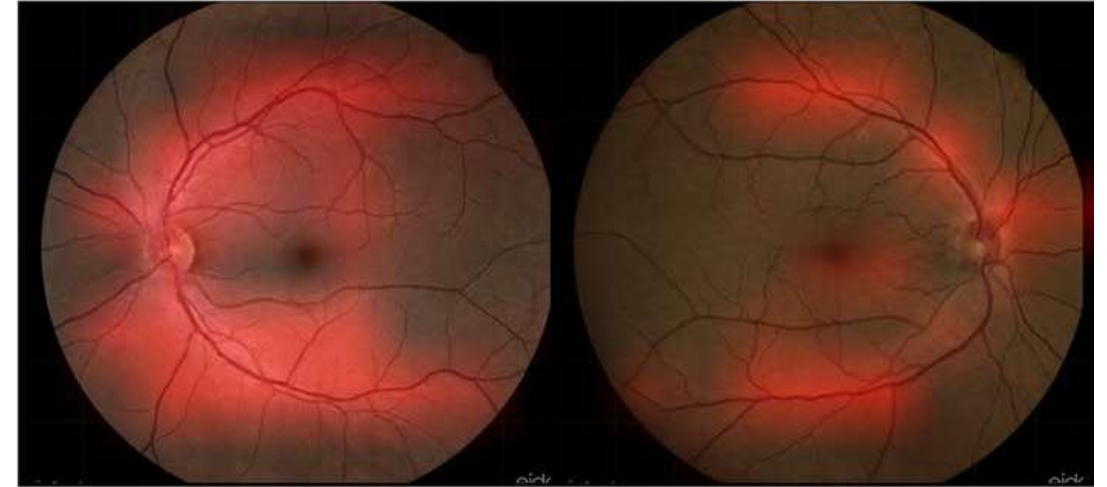
Visualizing Attention: Example

- Attention mechanisms often make neural networks **more interpretable**
- **Visualizing attention** provides insight into input-output relationships

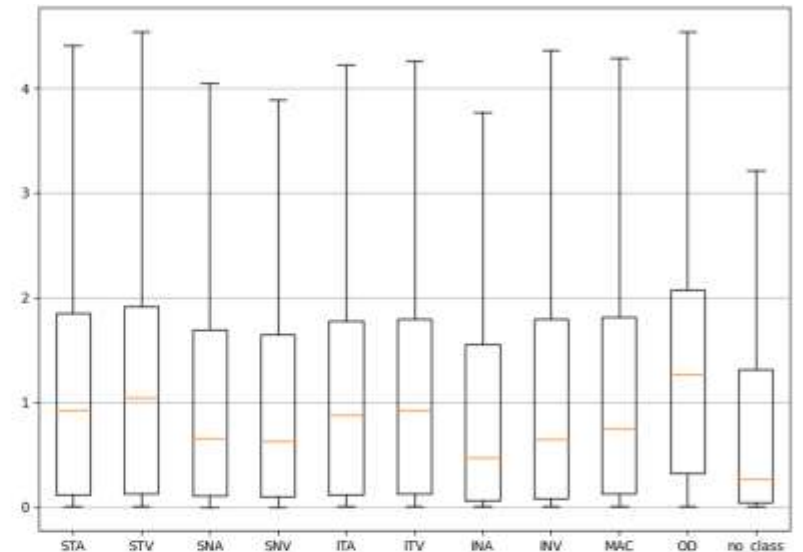


Visualizing Attention: Example 2

- [Mueller et al., Scientific Reports 2022] detect peripheral arterial disease from high-resolution color fundus photography
 - Visualize and statistically test the importance of various anatomical structures based on attention weights



(a) Example of our anatomical annotations



(b) Boxplot of the attention weights

Visualizing Attention: Theoretical Concerns

- **Attention scores** α_i used to aggregate hidden features $\mathbf{h}_i$

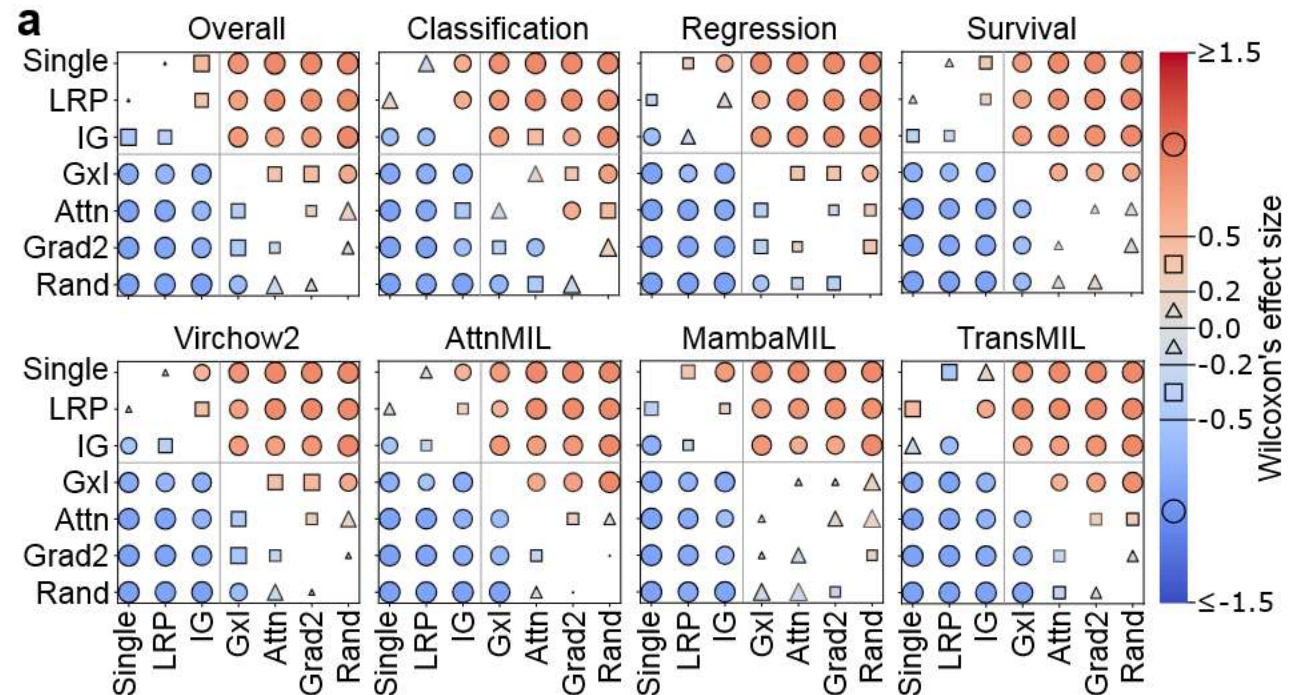
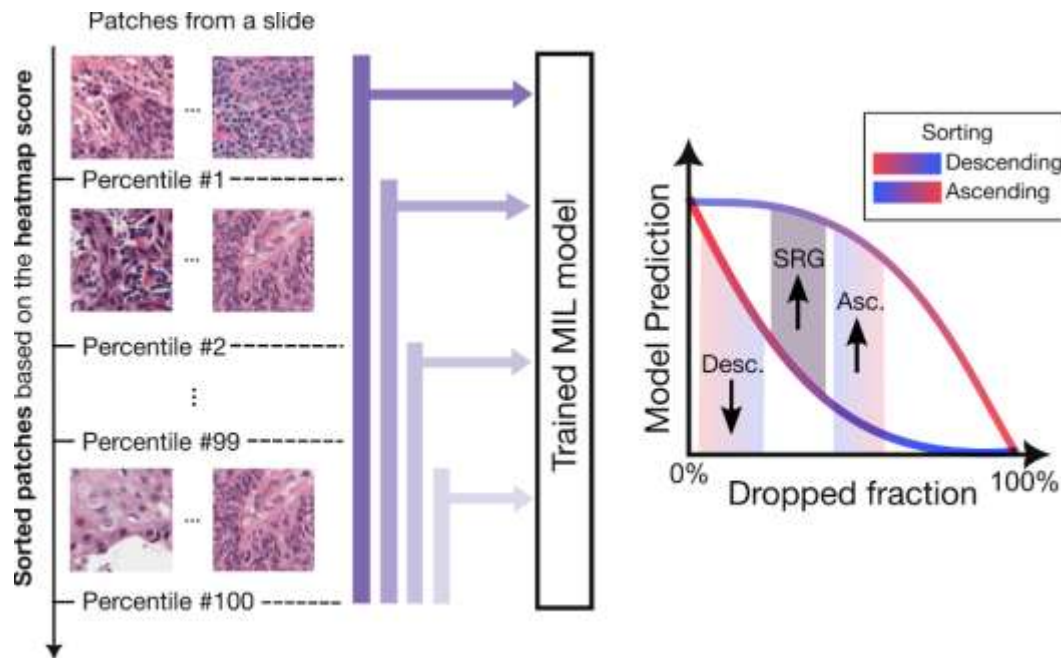
$$\mathbf{h} = \sum_i \alpha_i \mathbf{h}_i$$

are informative as an integral part of the model's computation, but do *not* directly expose impact on the final outcome

- Low α_i might be associated with large $\|\mathbf{h}_i\|$ (or vice versa)
- Instances with high α_i might still have a small effect if $\mathbf{h}_i$ is (near-) orthogonal to downstream weights (e.g., linear classification)

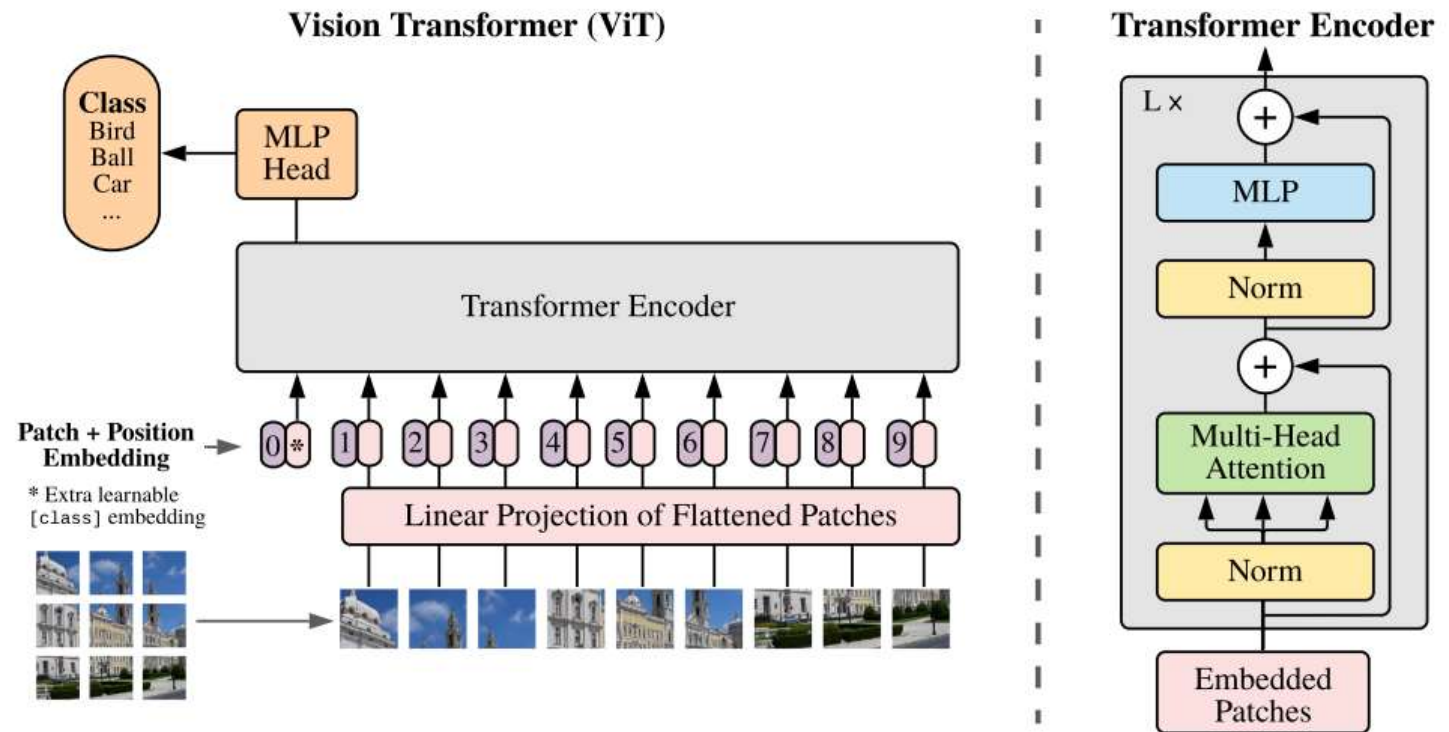
Visualizing Attention: Experimental Evidence

- [Idaji et al. 2026] compute **Symmetric Relevance Gain** as an attribution fidelity score
 - Single-instance scores, LRP, Integrated Gradients are found to be more faithful than raw attention across tasks and architectures



Vision Transformers (ViT)

- **Vision Transformers** are replacing CNNs in many vision tasks
 - Capture global context from the start
 - Well-suited for large-scale (self-supervised) training
 - Unified architecture across modalities
- **Visualization challenges** resulting from extensive use of attention across layers and heads are a topic of ongoing work



Summary: Post-hoc Interpretation

Three ideas for **highlighting important regions (attribution)**:

- **Perturbation approaches** are based on hiding parts of the image
 - Occlusion sensitivity: “Is this image region required?”
 - LIME: Superpixels instead of fixed occluder, accounts for interactions
 - Optimization-based approaches preserve or remove most relevant pixels
- **Gradient-based saliency** makes use of internal network structure
 - Saturation problem can be addressed with Integrated Gradients
 - Gradient noise can be addressed with SmoothGrad
 - Both approaches can be combined, at increased computational cost
 - Guided backpropagation often looks nice, but does not pass sanity check
- **Layer-wise relevance propagation** tries to identify the pixels that contribute most to a given class score

Summary: Interpretable Models

Two ideas for building interpretability into the network

- **Class activation maps** rely on global average pooling of the final convolutional layer
 - **Gradient weighted class activation maps** (Grad-CAM) generalize CAMs to more complex architectures by backpropagating to the last convolutional layer
 - **Grad-CAM++** provides more complete coverage of relevant area
- **Attention mechanisms** provide a built-in opportunity for visualizing what the network focuses on
 - Single-instance scores, LRP, Integrated Gradients increase faithfulness
 - Extensive use of attention in **Vision Transformers** is a Vis challenge

Section 16.4:

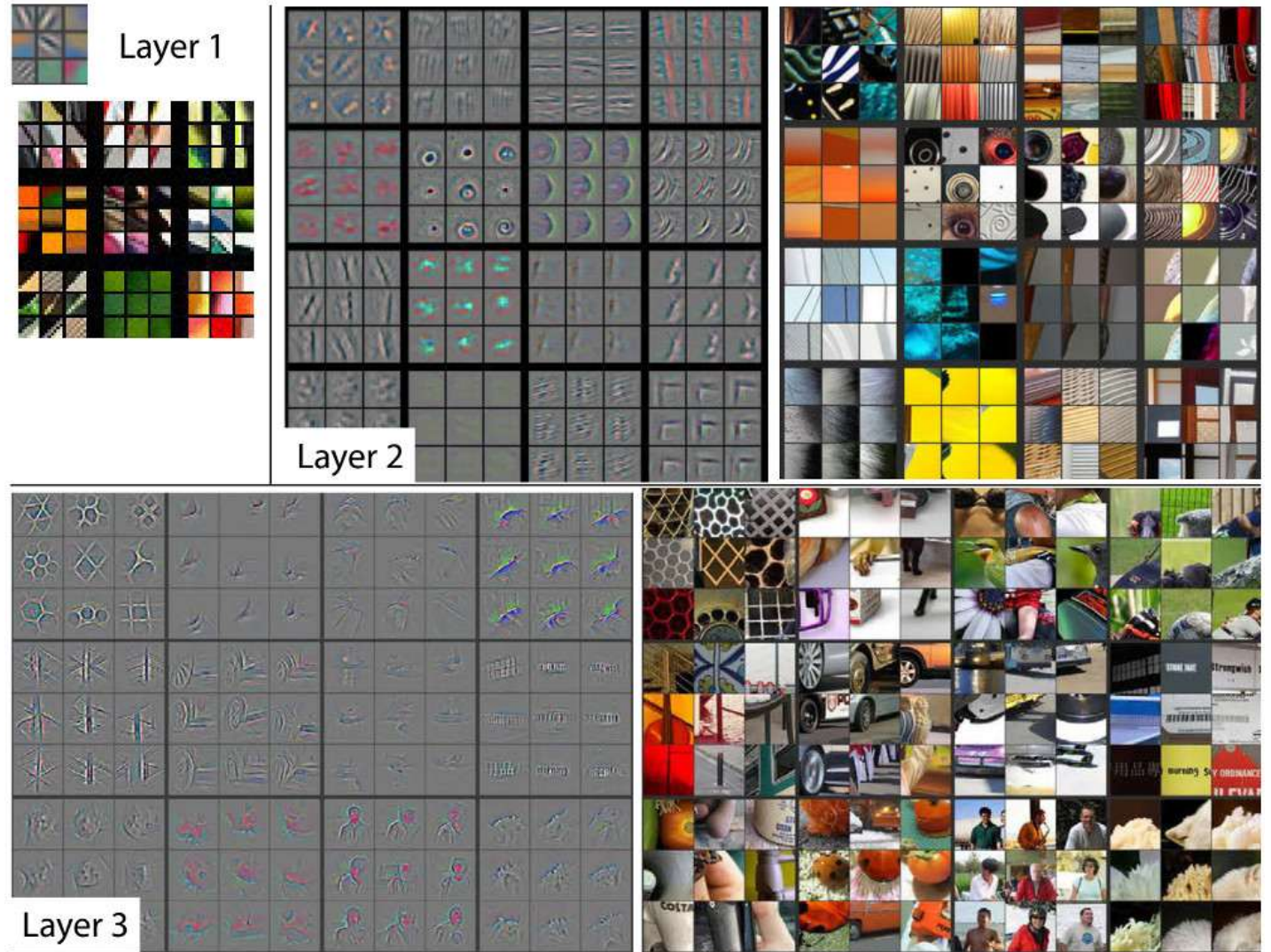
Understanding Hidden Layers

Motivation

- The power of deep neural networks partly derives from their ability to **learn suitable representations** of complex data:
Can we understand what these (learned) features represent?
 - Instead of trying to explain outcome for a specific single input, asks about general role of neurons (which concept is represented?)
- **Dataset search** and **saliency** are basic ideas
- **Optimization-based approaches** synthesize images that maximize selected hidden activations
- **Embedding visualization** reveals which inputs are judged to be similar at different network layers
- Interactive tools like **DeepVis** often combine several approaches

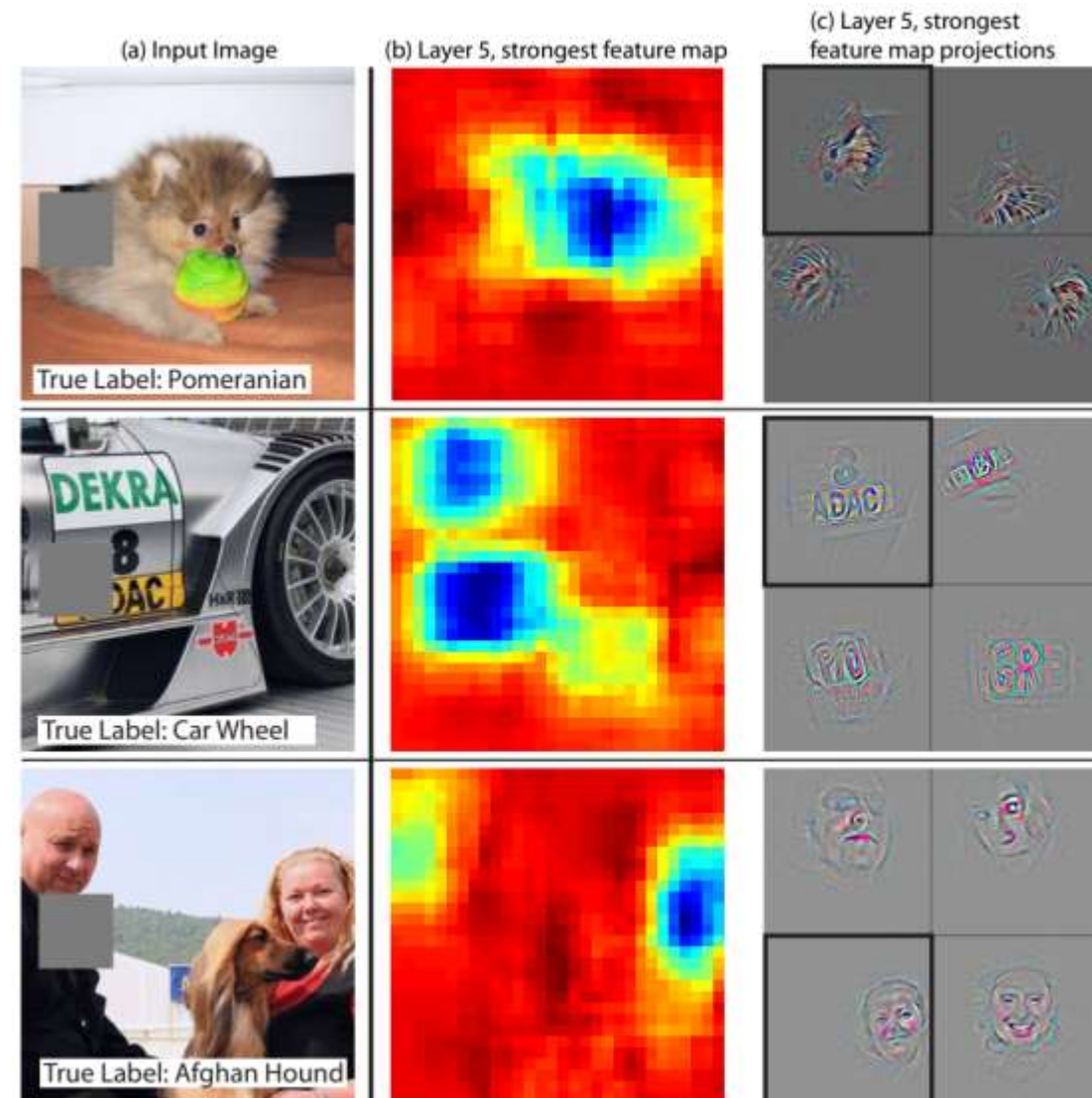
Dataset Search

- It is simple and informative to answer the question:
Which image patches led to the strongest activation of a unit?



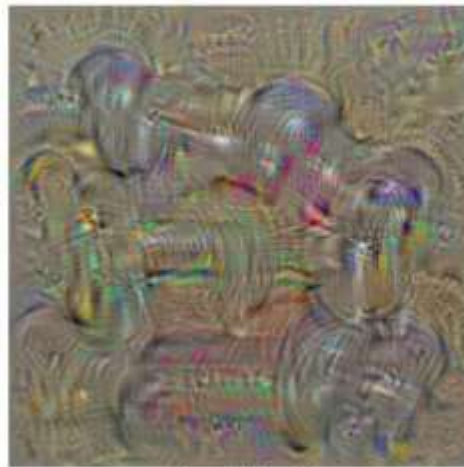
Saliency Methods for Hidden Layer Visualization

- We can apply **occlusion** or the **saliency methods** from Section 16.3 also to activations within the network
 - (Guided) backpropagation, deconvolutional networks
- Figure on the right illustrates **empirical agreement** between occlusion mapping and deconvolutional networks
 - DeconvNets have much lower cost



Visualizing Class Models

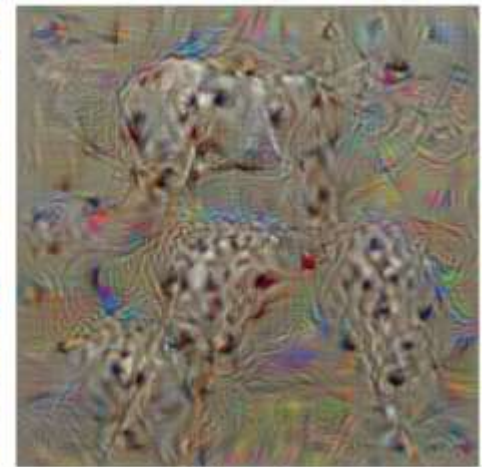
- Visualizing the **class model** addresses the following question:
How does the network think a specific class looks like?
 - Answered by generating an image that maximizes the corresponding class score S_c (with some regularization)
$$\mathbf{x} = \operatorname{argmax} L(\mathbf{x}) \text{ with } L(\mathbf{x}) = S_c(\mathbf{x}) - \lambda \|\mathbf{x}\|_2^2$$
 - Solved by gradient *ascent* via backpropagation
 - Analogous to training the network, but iteratively modifying input instead of weights



dumbbell



cup

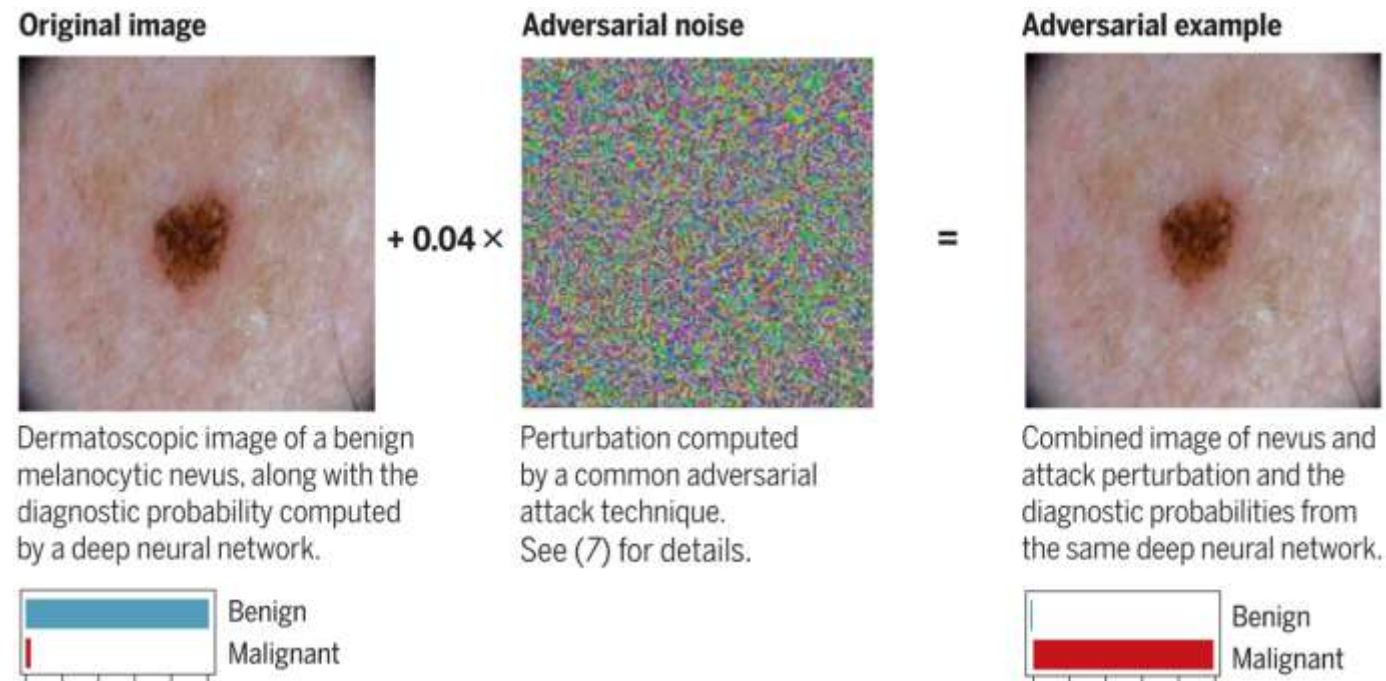


dalmatian

Aside: Adversarial Attacks

Neural networks can be misled by adding specific high-frequency patterns, even if they might not be visible to the human eye

- Certain high-frequency patterns cause strong activations, basic class score optimization tends to generate such difficult to interpret patterns

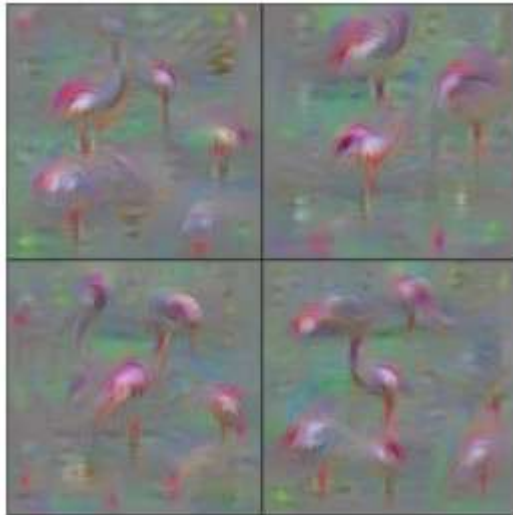


Class Models with Improved Regularization

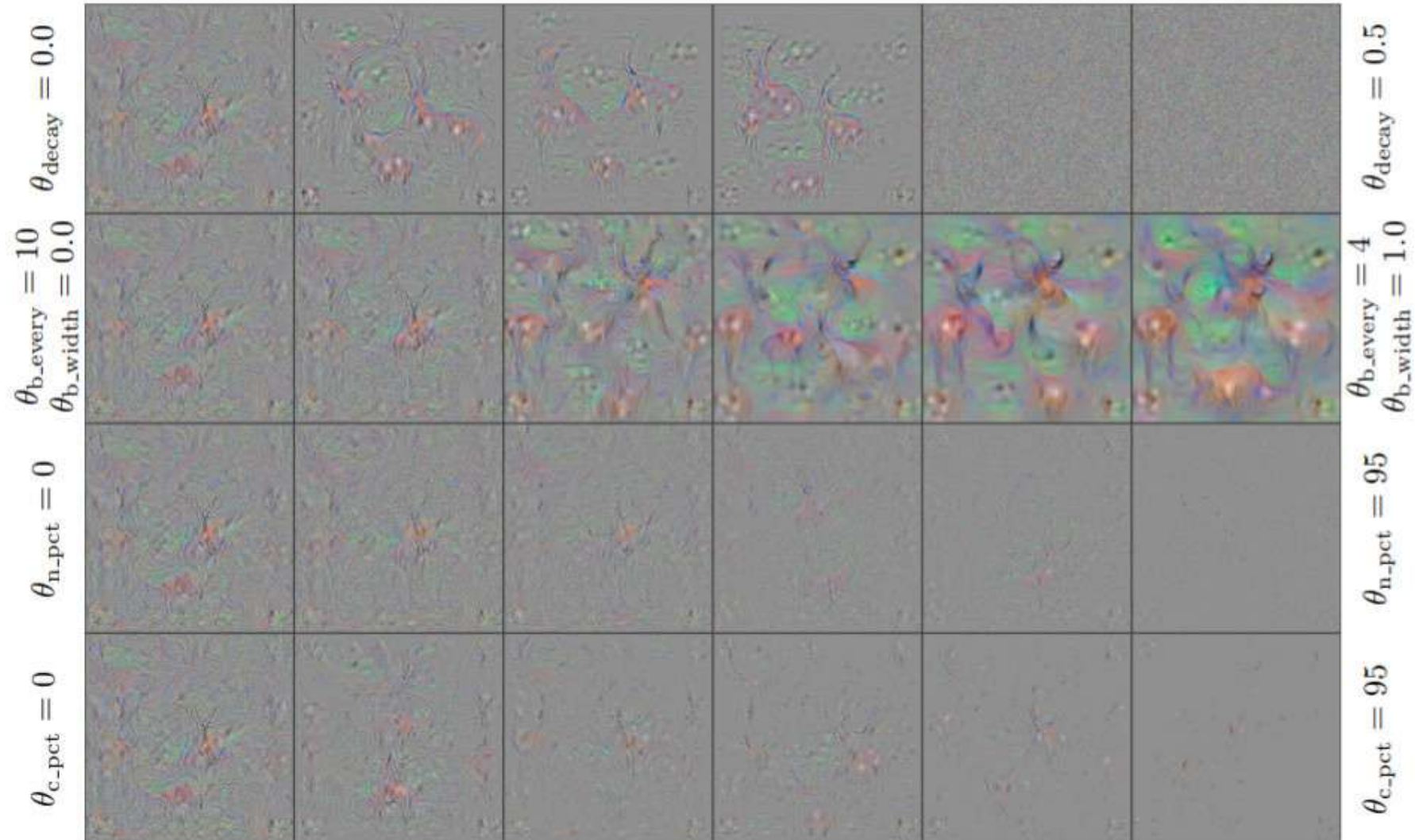
- **Goal:** Produce more natural and recognizable images by imposing a regularization that is more sophisticated than L_2
 - Difficult to express powerful regularization as differentiable loss
 - *Alternative:* Define regularization operator $r_\theta(\mathbf{x})$ that maps $\mathbf{x}$ to a more regularized version of itself; replace gradient ascent step with
$$\mathbf{x}^{(i+1)} = r_\theta(\mathbf{x}^{(i)} + \eta \nabla S_c(\mathbf{x}^{(i)}))$$
 - [Yosinski et al. 2015] define $r_\theta(\mathbf{x})$ to perform
 1. **L_2 decay** to avoid strongly dominating pixels
 2. **Gaussian blur** to avoid high-frequency patterns
 3. **Clipping pixels with low norm** to focus on main objects
 4. **Clipping pixels with low contribution** (i.e., product of value and gradient)

Illustration: Class Models with Improved Regularization

- **Right:** Impact of individual regularizers
- **Bottom:** Result achieved with a combination

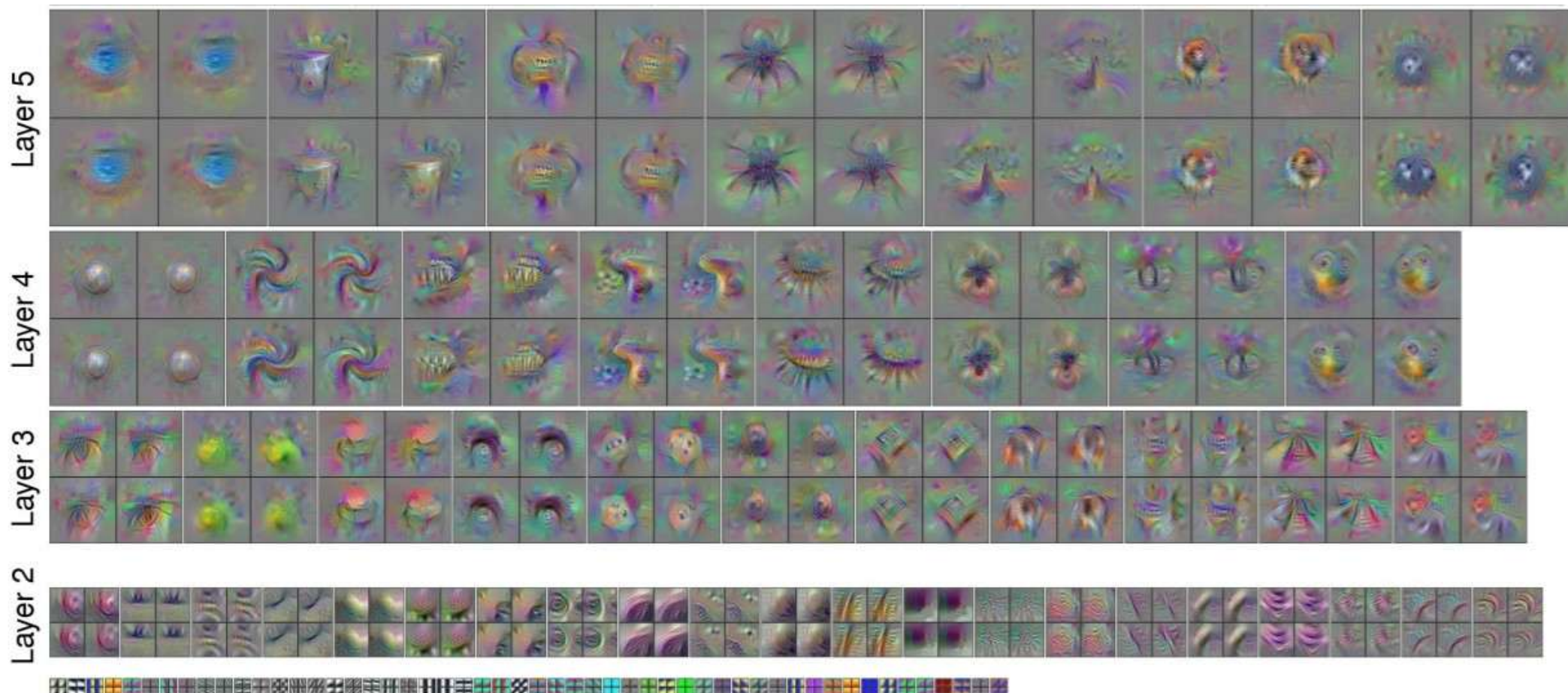


Flamingo



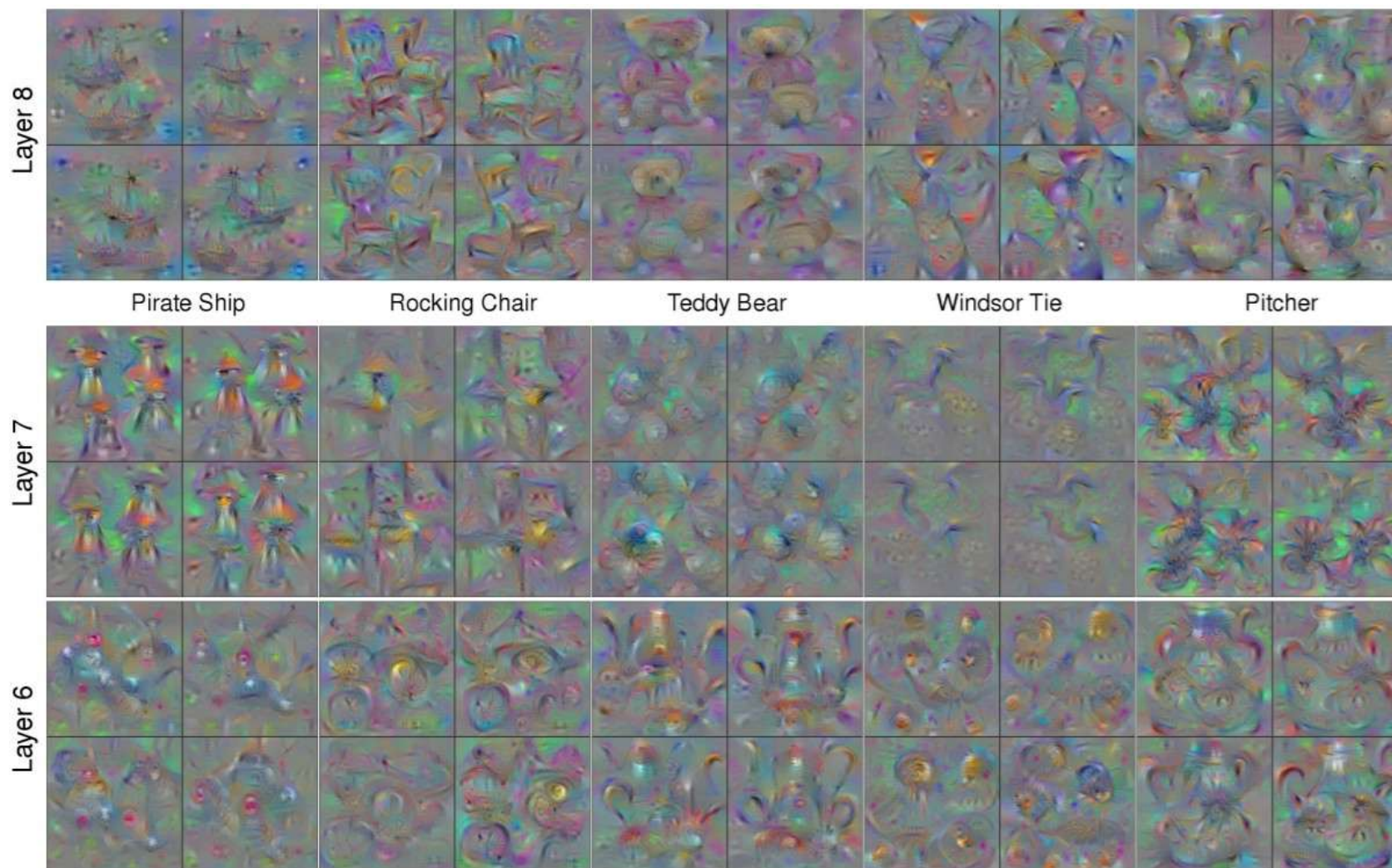
Gradient Ascent on Individual Activations

- To understand what is captured by an activation $a_j^{[l]}$ in a hidden layer, simply replace S_c with it



Results on Later Layers

- Variation in results increases with first fully connected Layer 6



Diagnostic Value of Class Models

- **Class model visualizations** might contain elements that are not part of the desired object itself
 - Suggests that the model has not fully captured the essence of the class; indicates a need to modify training data



“Dumbbell” class



“Saxophone” class

Deep Dreaming

- A variant of gradient ascent is “**deep dreaming**”. Iteratively:
 1. Feed image into CNN until dreaming layer
 2. Let gradients of dreaming layer equal its activations
 3. Backpropagate gradients and apply to input



"Admiral Dog!"



"The Pig-Snail"



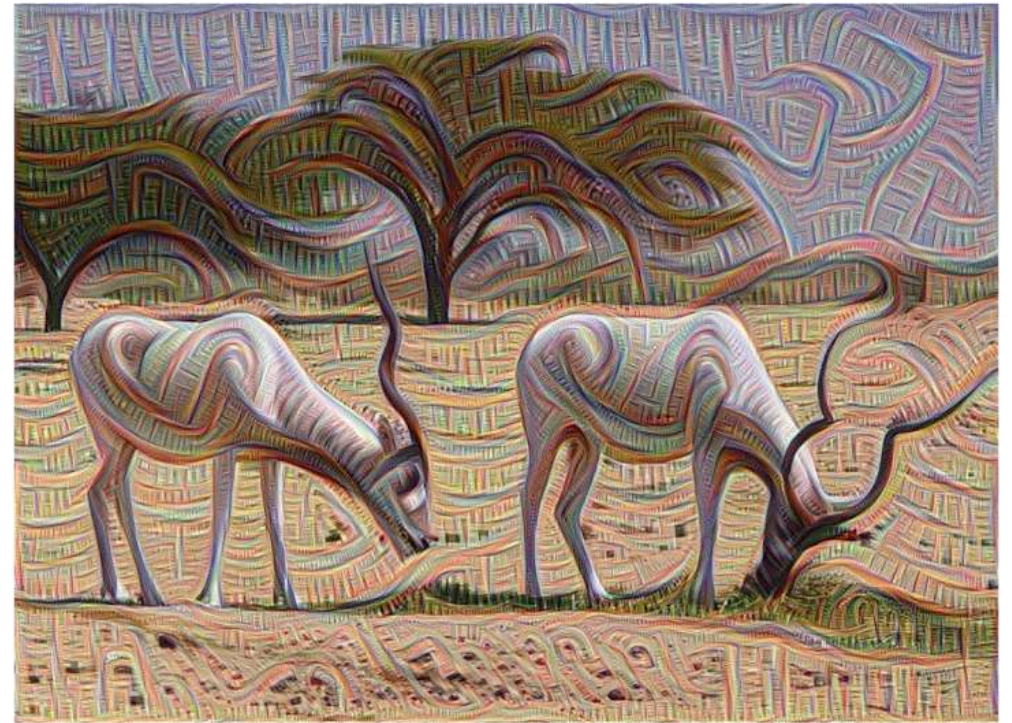
"The Camel-Bird"



"The Dog-Fish"

Deep Dreaming in Lower Layers

- Since lower layers detect low-level features such as edges and coherent structures, selecting them as dreaming layers produces strokes and ornament-like patterns



Are Neurons Interpretable?

- *Controversy*: It is not obvious why individual neurons should be more interpretable than random directions in feature space
 - *Intuition*: Rotating feature vectors can be compensated by rotating weights in the next layer



Mixed4e, Unit 55



mixed3a, random direction



mixed4c, random direction



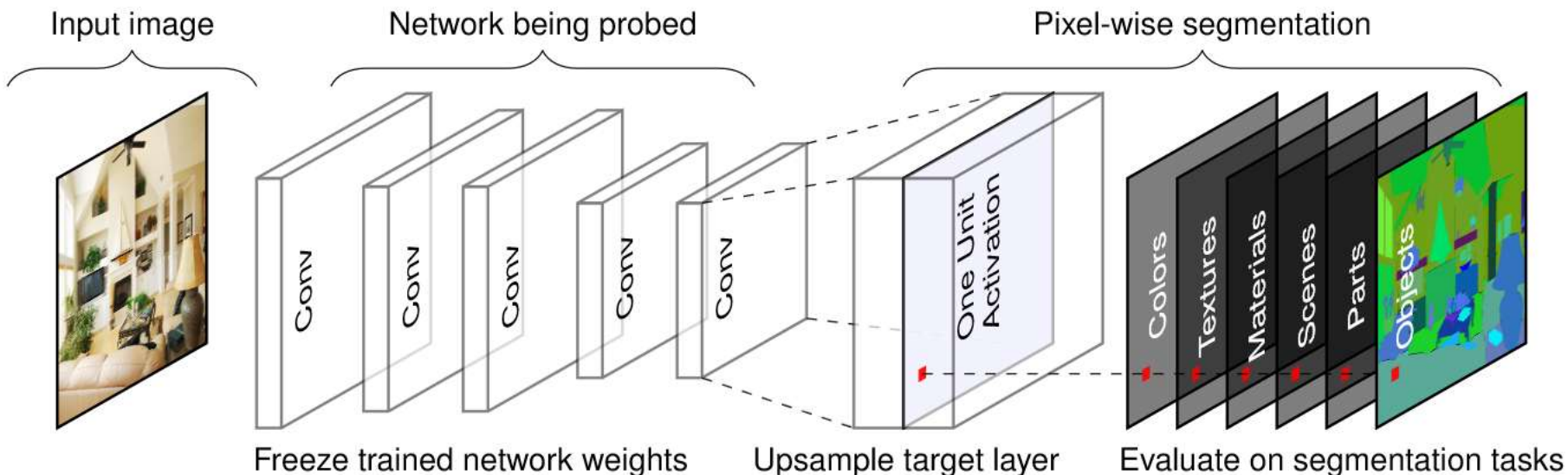
mixed4d, random direction



Network Dissection: Setup

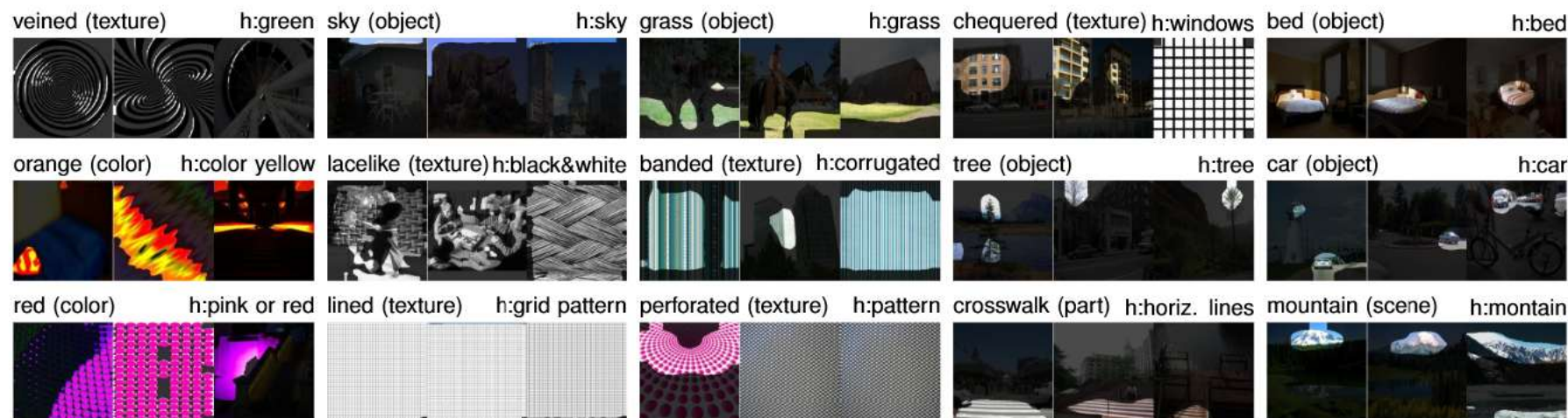
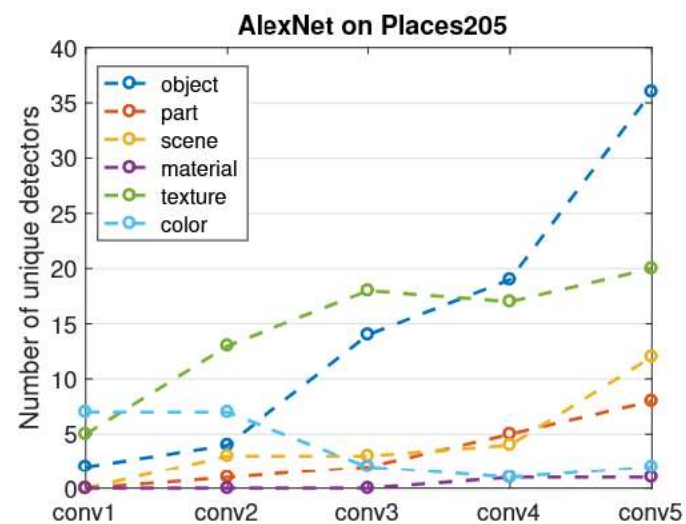
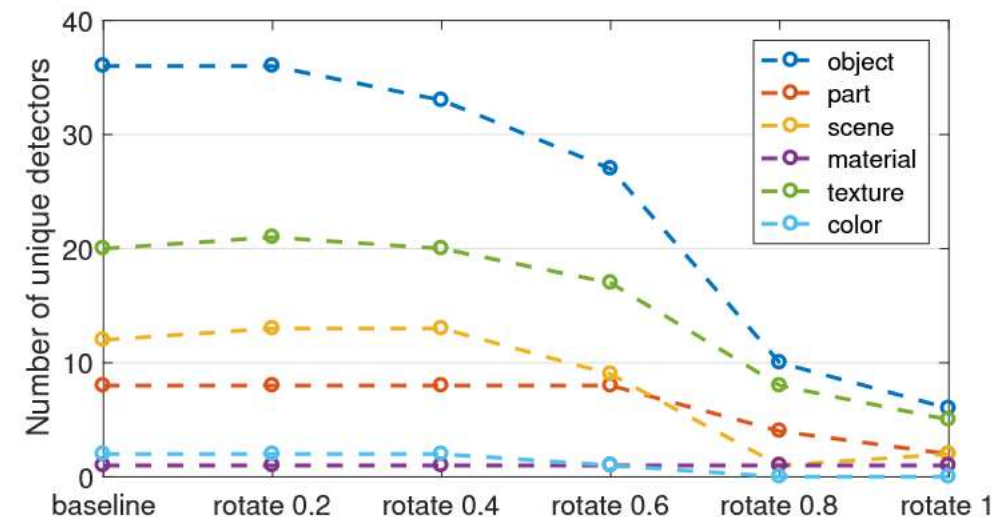
[Bau et al. 2017] quantify interpretability in CNNs using a broadly and densely labelled (Broden) dataset

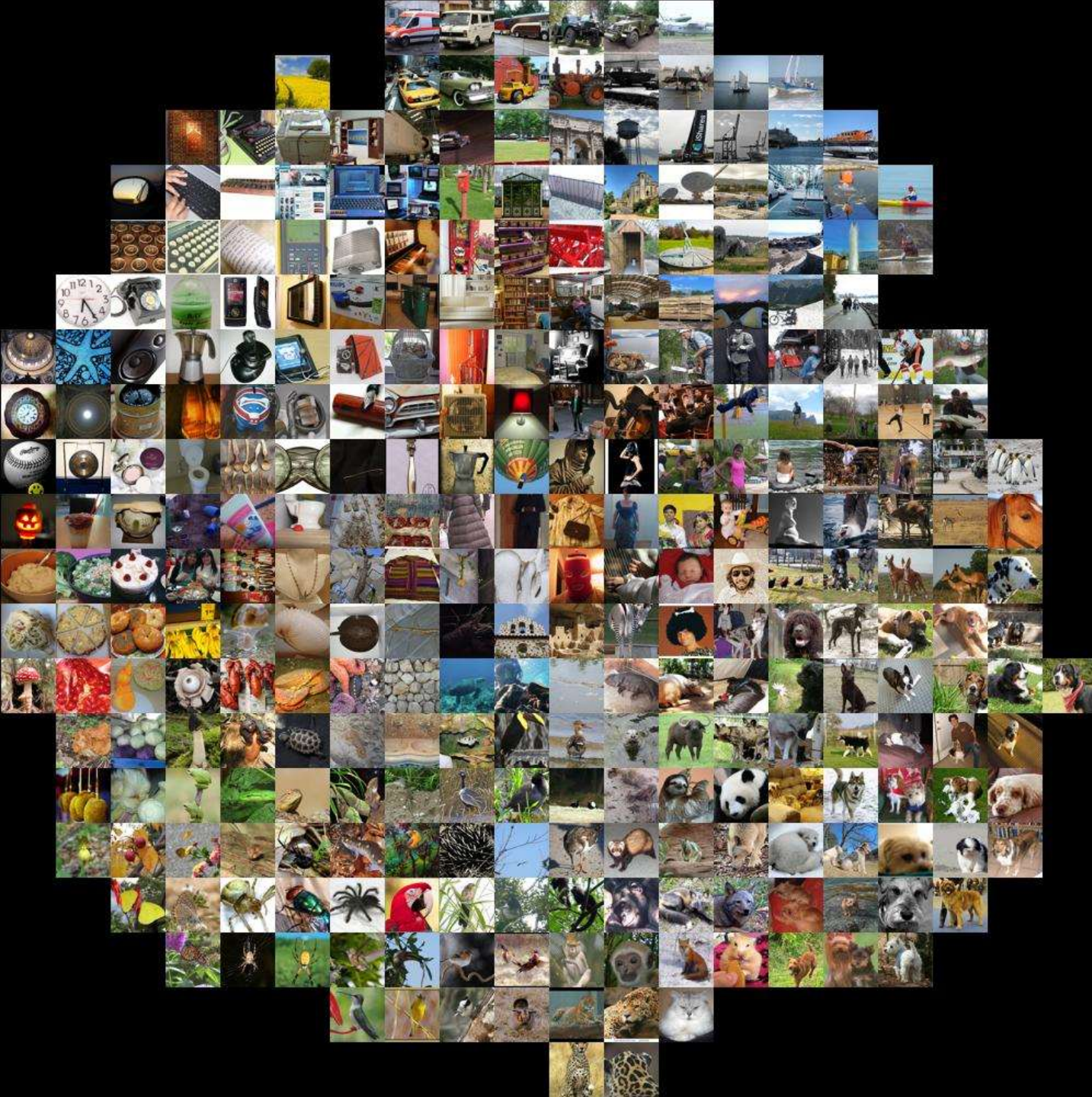
- Overlap between thresholded activation maps and human-interpretable concepts



Network Dissection: Experimental Results

- **Experimental outcomes** include:
 - Neurons are interpretable at a higher rate compared to random directions
 - Color / texture detectors dominate initially, object / scene detectors emerge in later layers





Dimensionality Reduction

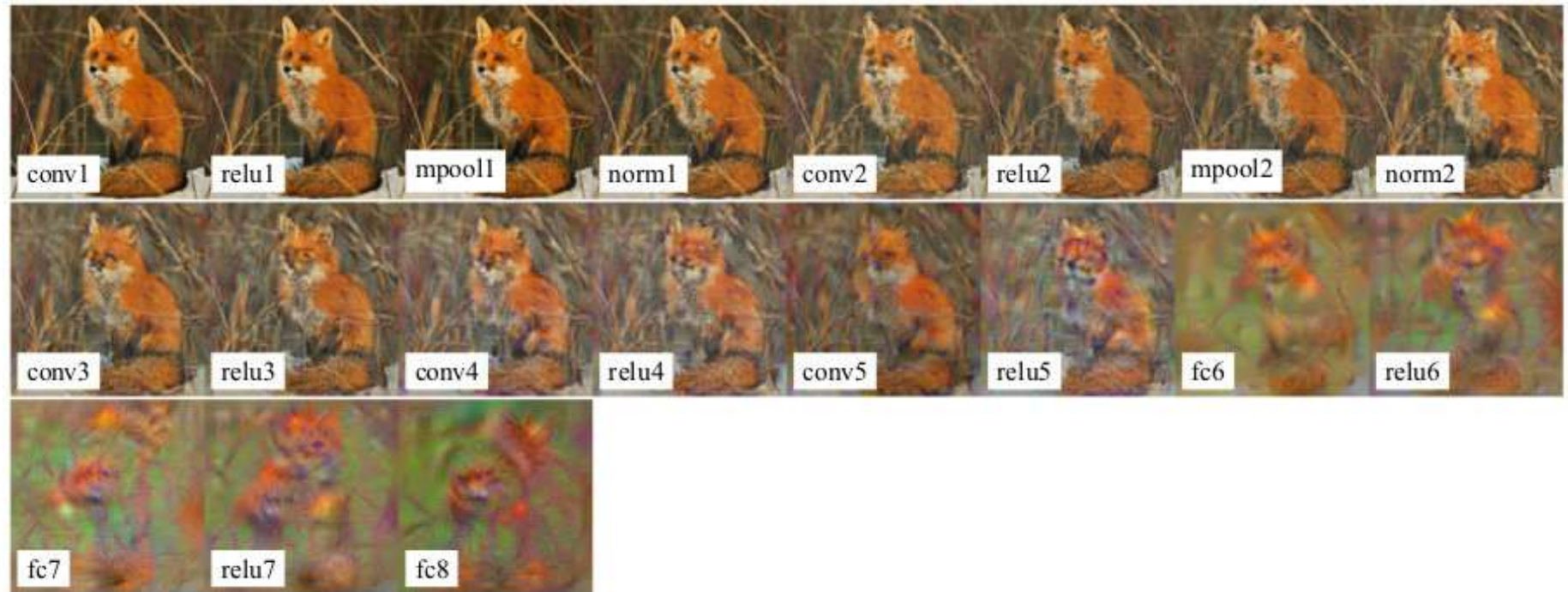
t-SNE is a popular choice for visualizing learned feature spaces

- *Example:* Embedding of 4096-dimensional features from ImageNet
 - Grouping is semantic, not based on per-pixel RGB similarity

Image from Andrej Karpathy
cs.stanford.edu/people/karpathy/cnnembed/

Pre-Images / Inversion

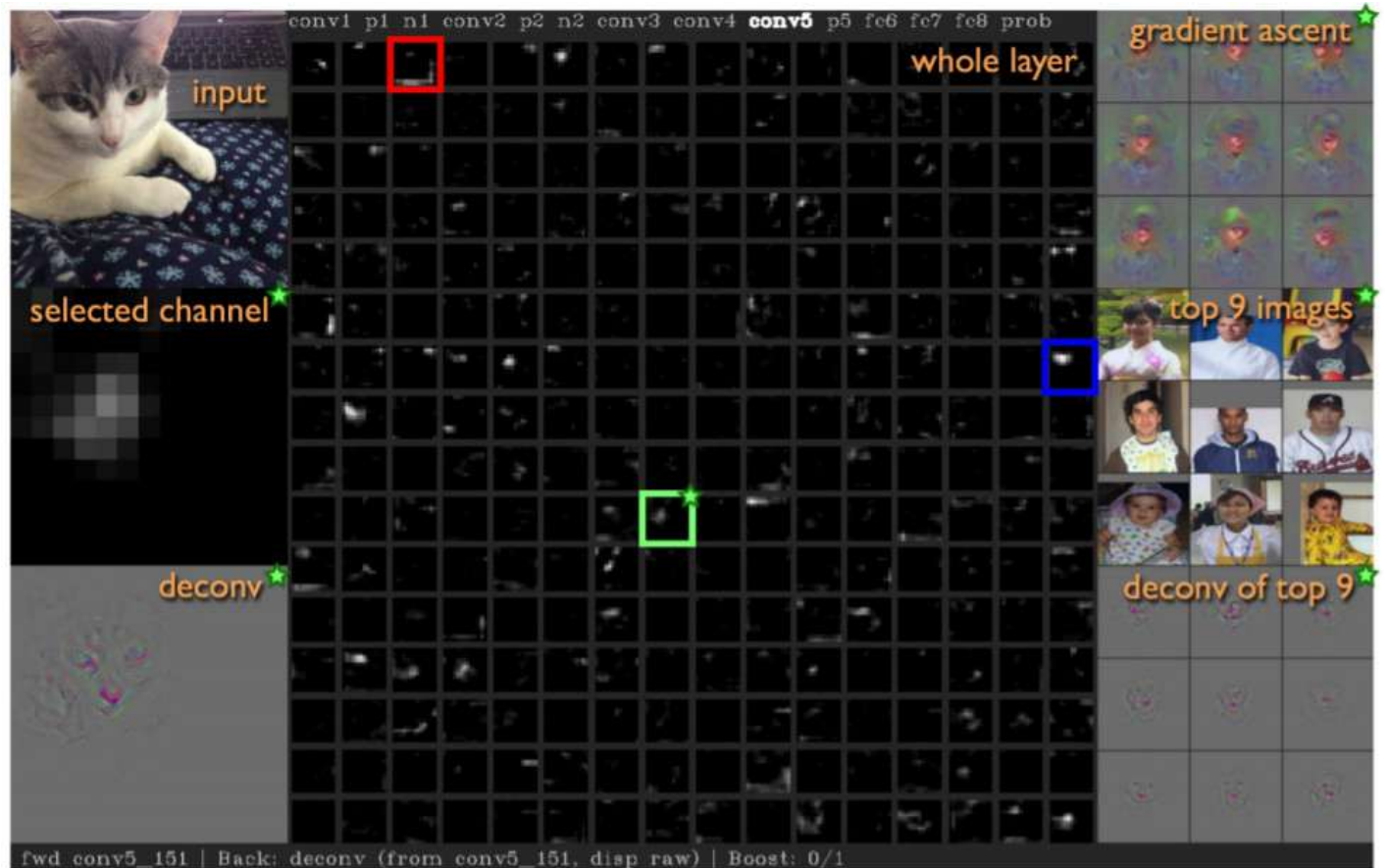
- **Inversion** attempts to reconstruct an input image $\mathbf{x}_0$ from the (vectorized) activations $\phi_0 := \phi(\mathbf{x}_0)$ at a given layer
 - Minimizes $\|\phi(\mathbf{x}) - \phi_0\|^2$ starting from a randomly initialized $\mathbf{x}$, with regularization to achieve a more natural $\mathbf{x}$
 - Visualizes what information is preserved at that layer



DeepVis Visualization Tool

[Yosinski et al. 2015] provide an interactive tool for exploring CNNs with the above-described methods

- Real-time:
 - Activations from forward pass
 - Deconvolution results for selected feature
- Interactively selected:
 - Class/activation model
 - Top 9 images
 - Deconv of top 9 images



[also show video]

Screenshot from <https://microscope.openai.com/models>

**OpenAI
Microscope**
permitted (?)
browsing
feature
visualizations
of several
widely used
CNN
architectures

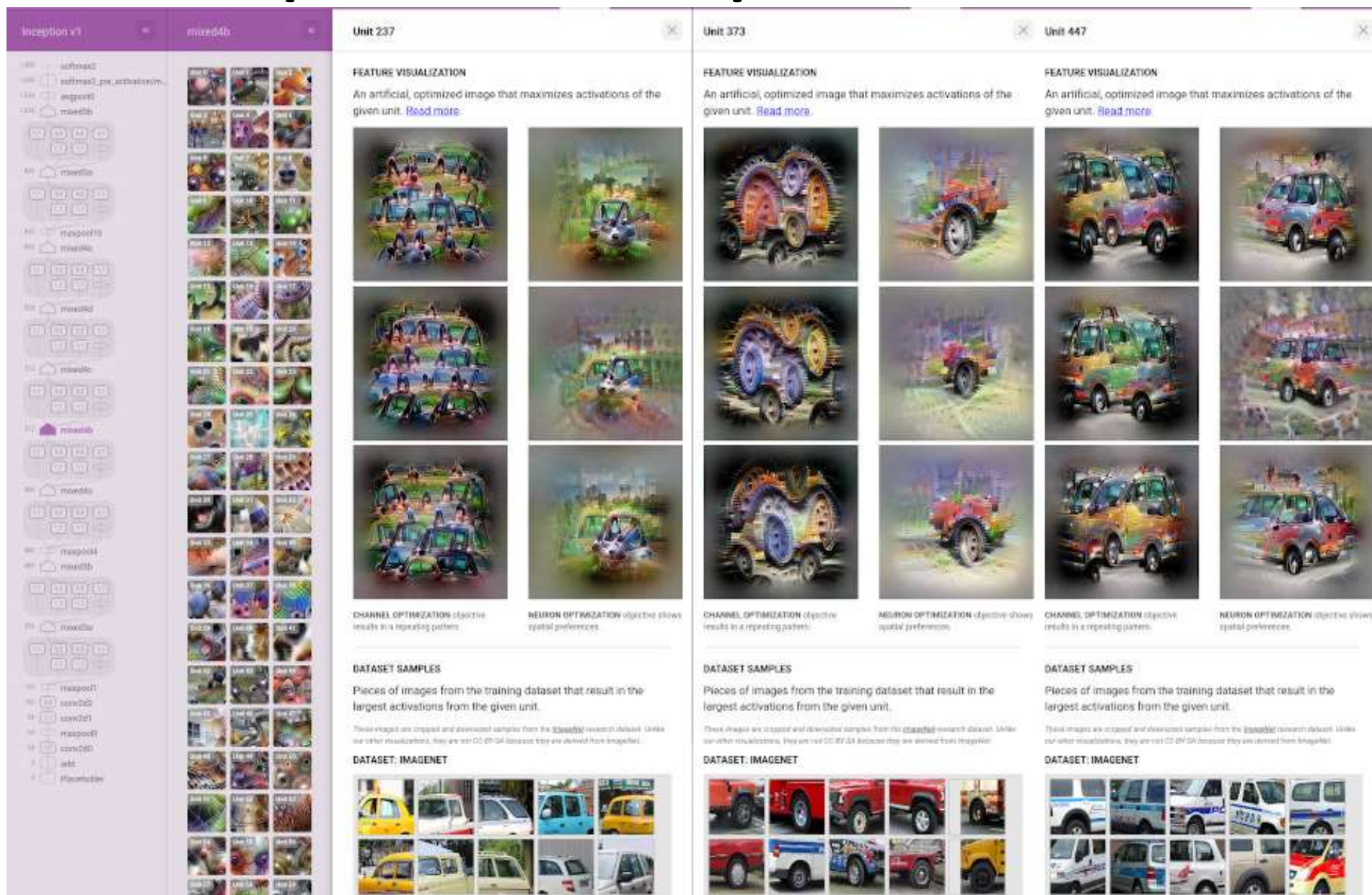


Exhibit at Deutsches Museum Bonn

An interactive video wall that visualizes CNN activations in real time is part of the AI exhibition at **Deutsches Museum Bonn**



Summary

Activations in hidden layers can be visualized

- **Directly** in response to a video stream or to different images
- By **searching** a given dataset (e.g., training or validation) **for examples** that maximize activations
- By applying **saliency** techniques (Section 16.3) to neurons
- By **synthesizing images** that maximize activations
 - **Regularization** is important to obtain interpretable results
 - **Deep dreaming** and **inversion** are variants of this
- By **embedding** them to two dimensions (e.g., using t-SNE)

Advertisement

Related Lab and
Thesis Topics
Available

The image displays a software interface for deep learning-based medical image segmentation, likely for cardiac MRI. The interface is divided into several sections:

- Architecture Diagram:** Shows a sequence of layers: Conv 1, Norm 1, LeakyReLU, Conv 2, Norm 2, LeakyReLU, and Transp Conv.
- Encoder-Decoder Architecture:** Illustrates a U-Net style architecture with three encoders (Encoder 1, Encoder 2, Encoder 3) and three decoders (Decoder 1, Decoder 2, Decoder 3). A Bottleneck layer connects the encoders. The output is labeled 'Output'.
- Region Selector:** Displays a grayscale MRI slice with a bright circular region. Below it are four buttons: 'Eliminate Region', 'Smart Contour', 'Invert Mask', and 'Reset Selection'.
- Segmentation with Ablation:** Shows a large MRI slice with a segmented region (yellow) and a reference region (blue). To the right, text indicates: 'Segmentation with Ablation', 'Active Ablations', 'Layer: upsamples.1.conv_block.norm2', and 'Channels: [0, 6, 12, 23, 26, 28]'. Below this, it compares Dice-Score: 'Dice-Score Original vs GT: 0.9284' and 'Dice-Score Ablation vs GT: 0.5251'. At the bottom are buttons for 'Show Differences' and 'Quantification'.
- Channel Selection:** A grid of four small MRI slices labeled 'Channel 7', 'Channel 31', 'Channel 12', and 'Channel 28'. Below them is a button 'Eliminate Selected Feature Maps' and a 'Show All' button.
- Navigation:** Includes buttons for '↑ Slice', '↓ Slice', '3', 'ED', '← MRI', '→ MRI', and '202'.

References

- Ian Goodfellow, Yoshua Bengio, Aaron Courville: “Deep Learning.” MIT Press, 2016 Accessible online: <https://www.deeplearningbook.org/>
- Matthew D. Zeiler and Rob Fergus. "Visualizing and understanding convolutional networks." *European conference on computer vision*. Springer, Cham, 2014.
- Sebastian Bach et al., “On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation.” PLOS ONE, 2015
- Jason Yosinski et al. “Understanding Neural Networks Through Deep Visualization.” ICML Deep Learning Workshop, 2015

References

- Julius Adebayo et al.: “Sanity Checks for Saliency Maps” NeurIPS 2018, correction as arXiv 1810.03292v3 (Nov 2020)
- Bolei Zhou et al.: “Learning Deep Features for Discriminative Localization” CVPR 2016
- Ramprasaath R. Selvaraju et al.: “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization” Int’l J. of Computer Vision, 2020

Announcements

- Discussion of trial exam on July 15 @ 14 c.t.
 - Several students still need to present to be admitted to the exam.
 - Please contact me TODAY if this affects you!
- Q&A session during lecture slot on Wed, July 22 @ 10 c.t.
- **Final exam on Monday, August 3 @ 12:00 sharp in Meckenheimer Allee 176, HS IV**
 - An A4 cheat sheet (both sides, in your own handwriting) is permitted
- Repeat exam on Mon, September 21 @ 11:00 in Meckenheimer Allee 176, HS II

Lecture Evaluation

- Please participate in the lecture evaluation, now available via <https://limesurvey.informatik.uni-bonn.de/index.php/599793?lang=en>